



GaiaCom

Benutzer-, Betreiber- und Entwicklerhandbuch

Sovereign Beta v2 · System & Architektur · September 2026

Post-Quantum Communication Infrastructure · Zero Server Trust · Federated by Design

VISIONGAIATECHNOLOGY

GaiaCom – Benutzer-, Betreiber- und Entwicklerhandbuch

Dokumentationsstand: Sovereign Beta v2 / Repository-Snapshot September 2026

Produkt: GaiaCom

Dokumenttyp: Benutzerhandbuch · Node-Operator-Handbuch · Deployment-Guide · Entwicklerreferenz

Basis: Quellcode und Repository-Dokumentation des bereitgestellten `GaiaCom-main`-Snapshots sowie die separat bereitgestellte ausführliche Architekturkarte.

Wichtig: Dieses Handbuch beschreibt den tatsächlich vorliegenden Repository-Stand. Wo Quellcode, README, ältere Dokumente und Architekturkarte voneinander abweichen, wird das ausdrücklich markiert. Die separat bereitgestellte Architekturkarte ist in **Anhang A vollständig und unverändert** eingebettet.

Inhaltsverzeichnis

1. Über dieses Handbuch
2. GaiaCom in einem Satz
3. Produktstatus und Release-Grenzen
4. Source of Truth und bekannte Dokumentationsdrift
5. Sicherheitsmodell und Vertrauensgrenzen
6. Grundbegriffe

Teil I – Benutzerhandbuch

7. Erster Start, Registrierung und Onboarding
8. Login, lokale Sperre und Wiederherstellung
9. Ein neues Gerät koppeln
10. Navigation und Dashboard
11. Identität, GaiaID, Kontakte und Trust Passport
12. Quanten-Chat / Direct Messaging
13. GaiaMail
14. SMTP Legacy Bridge
15. Gruppen, Räume und Unterkanäle
16. Public Channels
17. GSN – GaiaSocialNetwork
18. GaiaDrive
19. GaiaDrop
20. Network Health
21. Security Center / GaiaShield
22. Abuse Center, Governance und Gaia's Eyes
23. Profil und persönliche Einstellungen
24. Tastaturkürzel

Teil II – Node-Operator- und Deployment-Handbuch

25. Betriebsmodell
26. Voraussetzungen
27. Lokale Entwicklungsumgebung
28. Produktionsbuild
29. Linux-Server vorbereiten
30. Produktionskonfiguration mit `gaiacom setup`
31. `gaiacom doctor` und `gaiacom version`
32. Governance-Bootstrap

- 33. Systemd-Härtung
- 34. NGINX, TLS und Reverse Proxy
- 35. Firewall und Netzwerkexposition
- 36. Node Registry und kontrollierte Föderation
- 37. Speicher: lokales Object Store und S3/MinIO
- 38. SMTP produktiv konfigurieren
- 39. Healthchecks, Metrics und Logs
- 40. Backups und Wiederherstellung
- 41. Updates und Rollback
- 42. Produktions-Checkliste

Teil III – Entwicklerhandbuch

- 43. Repository-Struktur
- 44. Frontend-Architektur
- 45. Backend-Architektur
- 46. Persistenz und SQLite
- 47. Sovereign Crypto Core, WASM und Desktop
- 48. Android-Plattform
- 49. API-Übersicht
- 50. Security- und Quality-Gates
- 51. Sicher entwickeln und erweitern

Teil IV – Betriebshilfe

- 52. Troubleshooting
- 53. Bekannte Grenzen der Sovereign Beta v2
- 54. Glossar
- 55. Dokumentationspflege

Anhang

- A. Vollständige Architekturkarte
-

1. Über dieses Handbuch

GaiaCom besteht nicht aus einer einzelnen Webanwendung. Der bereitgestellte Stand umfasst eine React/Vite-Oberfläche, ein Go-Backend, SQLite-Persistenz, eine Rust-basierte Sovereign-Crypto-Schicht, WebAssembly, einen Tauri-Desktop-Host, eine modulare Android-Plattform, NGINX-/systemd-Deploymentdateien, Föderationslogik, Governance- und Security-Subsysteme sowie adversariale Test-Suites.

Dieses Handbuch richtet sich deshalb an vier Lesergruppen:

- **Endanwender**, die GaiaCom bedienen.
- **Administratoren und Node-Operatoren**, die einen GaiaCom-Knoten installieren, absichern und betreiben.
- **Entwickler**, die das Repository verstehen oder erweitern.
- **Security-Reviewer**, die Sicherheitsgrenzen, Test-Gates und bekannte Einschränkungen nachvollziehen müssen.

Die Architekturkarte in Anhang A dient als technische Tiefenreferenz. Der Hauptteil dieses Handbuchs legt den Schwerpunkt auf **Benutzung, Einrichtung, Betrieb und praxisnahe Abläufe**.

2. GaiaCom in einem Satz

GaiaCom ist eine **föderierte Kommunikationsplattform mit clientseitiger Ende-zu-Ende-Verschlüsselung und post-quanten-orientierter Hybridkryptografie**, die Direct Messaging, Mail, Gruppenräume, öffentliche Kanäle, Social-Funktionen, verschlüsselten Dateispeicher, anonyme bzw. pseudonyme Texteingaben, Governance und Node-Föderation in einer Plattform zusammenführt.

Der zentrale Designgedanke lautet **Zero Server Trust**: Ein korrekt betriebener GaiaCom-Server soll vertrauliche Nutzdaten nicht im Klartext benötigen, um Transport, Zustellung, Föderation und Speicherung zu ermöglichen.

3. Produktstatus und Release-Grenzen

Der vorliegende Snapshot ist als **Sovereign Beta v2** zu behandeln. Nicht jede im Quellcode sichtbare UI oder experimentelle Komponente ist automatisch ein freigegebenes Produktionsfeature.

3.1 Im Beta-Stand aktiv bzw. vorgesehen

Bereich	Status im bereitgestellten Stand
Native E2EE Direct Messages	Aktiv
Gruppenräume und Unterkanäle	Aktiv
GaiaMail	Aktiv
SMTP Bridge	Aktiv, aber expliziter Security-Downgrade
GaiaDrive	Aktiv
GaiaDrop Text-Einreichungen	Aktiv
GaiaProof	Aktiv
TrustMesh	Aktiv, lokal/policy-basiert
Trust Passport / Gaia Passport	Aktiv
Key-Change-Warnungen	Aktiv
Public Channels	Aktiv
GSN Social Layer	Aktiv
Controlled Federation	Aktiv, bekannte/zugelassene Nodes
Node Registry MVP	Aktiv
Governance / Reviewer	Aktiv
GaiaShield / Security Center	Aktiv
Sovereign Accelerated	Aktiv
Sovereign Top Secret	Aktiv, nur bei erfüllten Fähigkeiten
Browser-HQC-Provider	Aktiv über isolierten Worker/WASM
Tauri Native Crypto Bridge	Aktiv
S3/MinIO Object Store Adapter	Optional aktiv
Android modulare Plattformbasis	Technische Beta

3.2 Absichtlich noch nicht als vollständig freigegeben behandeln

Die Release-Dokumentation grenzt unter anderem folgende Punkte aus:

- Datei-, Medien- und Bildanhänge im Chat, solange das Attachment-Audit nicht abgeschlossen ist.
- Offene öffentliche Föderation ohne kontrollierte Zulassung.
- Vollständig offene Node-Registrierung.
- Global synchronisierter Abuse-Consensus.
- Vollständiger Mobile-Sovereign-v1-Nachrichtenpfad.
- Triple Ratchet beziehungsweise ratchet-basierte Forward-/Post-Compromise-Secrecy.
- Identity-signierte per-device Sovereign Prekeys als vollständig abgeschlossener Pfad.
- Attachment-basierter GaiaDrop.
- Vollautomatische Update-Pipeline.

Praxisregel: Wenn eine UI bereits einen Button für ein experimentelles Feature zeigt, die Release-Matrix es aber noch als deaktiviert oder nicht auditiert markiert, gilt die **Release-Grenze**.

4. Source of Truth und bekannte Dokumentationsdrift

Bei der Erstellung dieses Handbuchs wurden README, Quellcode, Deploymentdateien und die separat gelieferte Architekturkarte gegeneinander geprüft. Dabei sind einige Abweichungen sichtbar, die für Installation und Security wichtig sind.

4.1 Passwort-Hashing: Argon2id

GaiaCom verwendet Argon2id als verbindlichen serverseitigen Passwort-Hashing-Standard. Registrierung, Login, Passwortänderung und Kontolöschung laufen konsistent über denselben zentralen Argon2id-Pfad.

Neue Passworthashes werden ausschließlich mit Argon2id erzeugt. Noch vorhandene Legacy-Hashes sind im Zuge der laufenden Umstellung kontrolliert zu migrieren. Die Architekturkarte in Anhang A entspricht damit dem vorgesehenen Zielstand.

4.2 Vite-Konfigurationsdatei

Die Architekturkarte referenziert teilweise `vite.config.js`. Im bereitgestellten Repository lautet die Datei:

```
Frontend/frontend/vite.config.mjs
```

4.3 Lizenz: GNU AGPLv3

GaiaCom wird einheitlich unter der GNU Affero General Public License v3 (AGPLv3 / AGPL-3.0-only) veröffentlicht. LICENSE, README, Paketmetadaten und Quellcode-Hinweise sind entsprechend konsistent auf AGPLv3 zu führen.

4.4 `OPEN_REGISTRATION=false` nicht als sichere User-Registration-Sperre voraussetzen

`gaiacom setup` schreibt im Produktionsprofil `OPEN_REGISTRATION=false`, sofern kein Wert vorhanden ist. Im geprüften Auth-Registrierungspfad ist diese Variable jedoch nicht als eindeutiger serverseitiger Gatekeeper für `RegisterUser` zu erkennen.

Betriebsempfehlung: Nicht allein auf diese Variable vertrauen, wenn öffentliche Benutzerregistrierung unterbunden werden soll. Den gewünschten Gatekeeper im Auth-Pfad explizit implementieren und testen.

4.5 Lokale Frontend-/Backend-Ports

Das Frontend-Skript startet Vite standardmäßig mit:

```
vite --host 127.0.0.1
```

Vite verwendet dabei typischerweise Port `5173`. Der Backend-Dev-CORS-Pfad ist im Snapshot dagegen auf lokale Origins wie `http://localhost:3000` und `http://localhost:1420` zugeschnitten.

Für reproduzierbare Entwicklung wird in diesem Handbuch daher Port **3000** explizit gesetzt.

4.6 Desktop-API ist im Quellcode auf `beta.gaiacom.de` gepinnt

Der Frontend-API-Client enthält für den Desktop-Modus:

```
https://beta.gaiacom.de
```

Auch die Tauri-CSP ist auf dieses Ziel ausgerichtet. Wer einen Desktop-Build gegen einen eigenen Node ausliefern will, muss **API-Origin und CSP bewusst anpassen und den Client neu bauen**.

4.7 Produktions-Provisionierung: CLI vor UI-Secret-Generator

Einige ältere Operator-Dokumente beschreiben das Erzeugen von Node-Secrets über den Security-Center-/Nodesystem-Bereich. Der aktuelle Produktionspfad im Repository sieht dagegen `gaiacom setup` vor. Dieser erzeugt Secrets atomar, legt die Umgebung mit restriktiven Rechten an und kann anschließend durch `gaiacom doctor` geprüft werden.

Dieses Handbuch behandelt `gaiacom setup` als kanonischen Produktionsweg.

4.8 Backend-Build-Entrypoint

Für den ausführbaren Server ist der tatsächliche Entrypoint:

```
Backend/cmd/gaiacom
```

Daher sollte der Produktionsbuild mit `./cmd/gaiacom` erfolgen, nicht mit einem generischen Build auf dem Backend-Package-Root.

5. Sicherheitsmodell und Vertrauensgrenzen

5.1 Was GaiaCom schützen will

Das System ist darauf ausgelegt, folgende Schutzziele zu unterstützen:

- Vertraulichkeit nativer Nachrichten und Dateien durch clientseitige Verschlüsselung.
- Authentizität und Integrität kryptografischer Envelopes.
- Erschwerung von Downgrade- und Key-Substitution-Angriffen.
- Signierte und replay-resistente Server-zu-Server-Kommunikation.
- Minimierung von serverseitigem Klartext.
- Begrenzung von Operator- und Reviewer-Rechten.
- Nachvollziehbare Security-Events über GaiaShield und Audit-Ketten.
- Kryptografisch abgesicherte Trust-/Disclosure-Workflows.

5.2 Was GaiaCom ausdrücklich nicht automatisch löst

Zero Server Trust bedeutet nicht „Zero Risk“.

Ein kompromittiertes oder entsperartes Endgerät kann Klartext, Sessionmaterial oder Eingaben preisgeben. Ein kompromittiertes Betriebssystem kann Keylogging, Screen-Capture oder Speicherzugriffe ermöglichen. Die Web/PWA-Variante vertraut außerdem darauf, dass die aktuell ausgelieferte JavaScript-Anwendung nicht böartig ersetzt wurde.

Für hohe Assurance ist ein **signierter, reproduzierbar gebauter nativer Client** konzeptionell stärker als ein bei jedem Aufruf neu vom Node geladener Webclient.

GaiaCom ist außerdem **kein Anonymitätsnetzwerk**. Föderations-, Zustell-, Zeit-, Account- und Netzwerkmetadaten können trotz Inhaltsverschlüsselung sensibel sein.

5.3 Native Kommunikation versus SMTP

Native GaiaCom-Kommunikation bleibt innerhalb der GaiaCom-Kryptografie und des Föderationsmodells.

Bei SMTP verlässt die Nachricht diesen Vertrauensraum. Die Bridge muss daher in UI und Betrieb als **bewusster Downgrade** verstanden werden. Nach Übergabe an klassische Mailinfrastruktur gelten deren Transport-, Provider- und Metadatenrisiken.

6. Grundbegriffe

GaiaID

Föderierte Benutzeradresse im Stil:

```
@username:node.example.org
```

Der Node-Teil bestimmt unter anderem, wohin föderierte Kommunikation geroutet werden muss.

Sovereign Accelerated

Der performantere Sovereign-Modus. Er kombiniert im vorliegenden Design eine Hybrid-KEM-Ableitung mit einer zweistufigen authentifizierten Verschlüsselungskaskade.

Sovereign Top Secret

Höhere kryptografische Stufe mit zusätzlicher Cipher-Kaskade und zusätzlichen Signatur-/Capability-Anforderungen. Sie darf nur aktiviert werden, wenn beide Kommunikationsseiten und der Nachrichtenpfad die erforderlichen Fähigkeiten nachweislich unterstützen.

GaiaProof

Nachweisformat, mit dem relevante Kommunikations- oder Missbrauchssachverhalte kryptografisch belegt werden können, ohne einen universellen Admin-Entschlüsselungsschlüssel einzuführen.

TrustMesh

Lokales, föderationsbezogenes Reputations- und Friction-System. Es ist kein globaler „Wahrheitswert“.

Gaia Passport / Trust Passport

Darstellung von Identitäts-, Human-Proof- und Vertrauensinformationen zu einer GaiaID.

Gaia's Eyes

Kontrollierter Offenlegungsworkflow für freiwillig beziehungsweise policy-basiert bereitgestellte Nachweise in Governance-/Reviewer-Prozessen.

GaiaShield

Security-Subsystem für Anomalien, Audit-Events, Schutzmiddleware, Redaction und Operator-Sichtbarkeit.

Teil I – Benutzerhandbuch

7. Erster Start, Registrierung und Onboarding

7.1 Startbildschirm

Der Auth-Bereich stellt mehrere Einstiegspfade bereit:

- Login
- Registrierung
- Recovery
- Geräte-Kopplung
- GaiaDrop für externe Texteingaben ohne normalen Account-Login

7.2 Registrierung

Für einen neuen Account werden im UI mindestens Benutzername, Passwort und die erforderlichen rechtlichen beziehungsweise Beta-Bestätigungen benötigt.

Der Backend-Standard erzwingt für Benutzernamen und Passwörter serverseitige Plausibilitätsgrenzen. Für Passwörter ist im aktuellen Standard eine Mindestlänge von **12 Zeichen** vorgesehen.

Während des Registrierungsflusses werden kryptografische Identitätsmaterialien **clientseitig** erzeugt. Dazu gehört auch das Recovery-/Mnemonic-Material, das der Server nicht als Klartext-Backup besitzen soll.

7.3 SetupWizard – die fünf wesentlichen Phasen

Phase 1 – Bedingungen und Beta-Hinweise

Der Benutzer bestätigt unter anderem:

- Bedingungen gelesen.
- Beta-Status verstanden.
- Keine Garantie auf permanente Verfügbarkeit.
- SMTP liegt außerhalb des vollen nativen Security-Modells.
- Verantwortung für Recovery-/Credential-Verwahrung.
- Rechtmäßige Nutzung.

Phase 2 – Recovery sichern

Die Wiederherstellungsphrase wird lokal angezeigt. Der Benutzer muss bestätigen, dass er sie gesichert hat.

Empfehlung: Recovery-Material niemals in Chat, E-Mail, Cloud-Notizen oder Screenshots speichern, wenn der betreffende Speicher nicht bewusst als sicherer Recovery-Ort ausgewählt wurde.

Phase 3 – Identität und GaiaID

Der Benutzer legt beziehungsweise registriert seine Identität auf einem Node. Daraus entsteht die föderierte GaiaID.

Beispiel:

```
@rene:node.example.org
```

Je nach UI-Stand können verfügbare Nodes, ein eigener Node oder Fallback-Nodes angeboten werden.

Phase 4 – Security-Einführung

Das Onboarding erklärt die zentralen Ebenen:

- Identität
- Nachrichtenverschlüsselung
- Netzwerk/Föderation
- Transparenz und Trust

Phase 5 – Einstieg

Typische Schnellziele sind:

- Inbox
- Contacts
- Channels
- Security Center

7.4 First-Run-Onboarding

Zusätzlich zum SetupWizard existiert ein First-Run-Onboarding für Profil, Avatar, Security-Hinweise, Backup-Bestätigung und den ersten Sprung in die Anwendung.

8. Login, lokale Sperre und Wiederherstellung

8.1 Login

Der normale Login authentifiziert den Benutzer am Backend und initialisiert den lokalen Clientzustand. Access-/Refresh-Token werden getrennt vom eigentlichen Recovery-Material behandelt.

8.2 Lokale Session-Sperre

GaiaCom kann eine bereits authentifizierte Oberfläche bei Inaktivität lokal sperren. Die Entsperrung kann – je nach aktivierter Konfiguration – erfolgen über:

- Passwort
- PIN
- WebAuthn PRF / hardwaregebundene Ableitung

Die lokale Sperre ist nicht identisch mit einem vollständigen Logout.

8.3 Logout

Logout beendet die aktuelle serverseitige Session beziehungsweise den dafür vorgesehenen Tokenpfad und räumt den lokalen Auth-Zustand auf.

8.4 Recovery-Datei importieren

Der Recovery-Bereich kann verschlüsselte GaiaCom-Recovery-Dateien importieren, beispielsweise im `.json` - beziehungsweise `.gaiacom-recovery.json` -Format.

Typischer Ablauf:

1. Recovery-Datei auswählen.
2. Passwort der Recovery-Datei eingeben.
3. Neues lokales Entsperrpasswort festlegen.
4. Recovery entschlüsseln.
5. Mnemonic/Identitätsmaterial lokal wiederherstellen.
6. Account-/Identitätszustand erneut initialisieren.

Wichtig: Eine Recovery-Datei ist nur so sicher wie ihr Passwort und ihr Aufbewahrungsort.

9. Ein neues Gerät koppeln

GaiaCom trennt das **Geräte-Pairing** von einer gewöhnlichen Browser-/Login-Session.

9.1 Auf dem neuen Gerät

Im Pairing-Bereich:

1. Accountdaten angeben.
2. Einen Gerätenamen beziehungsweise ein Label vergeben.
3. Pairing anfordern.
4. GaiaCom erzeugt einen einmaligen Pairing-Code beziehungsweise eine URI in der Art:

```
gaiacom://pairing/<pairing-id>/<one-time-secret>
```

Diese kann als Code/QR an das bereits vertrauenswürdige Gerät übertragen werden.

9.2 Auf dem bereits vertrauenswürdigen Gerät

Im Profil unter **Devices / Geräte**:

1. Pairing-Code einfügen oder scannen.
2. Pairing prüfen.
3. Aktive Identität auswählen.
4. Recovery-/Sovereign-Material wird für das neue Gerät kryptografisch verpackt.
5. Freigabe signieren.
6. Pairing bestätigen.

9.3 Auf dem neuen Gerät abschließen

Das neue Gerät prüft, ob die Freigabe vorliegt, konsumiert das Pairing und speichert die freigegebenen lokalen Schlüsselmaterialien geschützt ab.

9.4 Geräte später verwalten

Im Profil können je nach Zustand:

- aktive Sessions widerrufen,
- registrierte Crypto-Geräte widerrufen,
- aktuelle Geräteschlüssel rotiert,
- neue Pairings freigegeben werden.

Bei Verlust eines Gerätes sollten sowohl **Sessions als auch Geräteschlüssel** geprüft und gegebenenfalls widerrufen werden.

10. Navigation und Dashboard

Die Hauptnavigation ist in drei große Bereiche gegliedert.

10.1 Workspace

- Dashboard
- GaiaMail
- SMTP Mail
- Quanten-Chat
- Gruppen-Chats

GaiaMail und SMTP besitzen Mailboxordner wie:

- Inbox
- Drafts
- Sent
- Starred
- Important
- Snoozed
- Archive
- Spam
- Trash

10.2 Netzwerk

- Public Channels
- GSN Social
- Network Health
- Security Center
- Abuse Center

10.3 Persönlich

- Adressbuch / Kontakte
- Mein Profil
- GaiaDrive
- GaiaDrop

Am unteren Navigationsbereich befinden sich – abhängig vom Client – Theme, Sprache, Sperren, Logout, Quantum-Shield-Status und Versionsanzeige.

10.4 Dashboard

Das Dashboard ist die Kommandozentrale. Es zeigt unter anderem:

- Session-/Security-Status
 - Netzwerk-/Uplink-Status
 - aktive Identität
 - kryptografische Statusindikatoren
 - verbundene Räume
 - aktuelle beziehungsweise neue Mail-/Chat-Aktivität
 - Schnellzugriffe
 - PWA-Installationshinweis, wenn anwendbar
-

11. Identität, GaiaID, Kontakte und Trust Passport

11.1 Kontakte hinzufügen

Kontakte können über eine GaiaID gesucht beziehungsweise hinzugefügt werden.

Beispiel:

```
@alice:node.example.org
```

Die GaiaID wird formal geprüft, bevor sie als föderierte Adresse benutzt wird.

11.2 Kontaktprofil

Das Kontaktprofil bündelt:

- GaiaID
- Profilinformationen
- Trust-/Passport-Daten
- Human-Proof-Status
- Schlüssel-/Fingerprint-Informationen
- Schlüsselhistorie

11.3 Fingerprints verifizieren

Für besonders sensible Kontakte sollte der angezeigte Fingerprint **über einen zweiten, bereits vertrauenswürdigen Kanal** verglichen werden.

11.4 Key Change Warning

GaiaCom führt eine lokale beziehungsweise append-only orientierte Schlüsselhistorie. Ändert sich ein erwarteter Empfängerschlüssel, kann der Client warnen.

Eine Warnung bedeutet nicht automatisch einen Angriff. Mögliche legitime Ursachen sind:

- neues Gerät,
- Schlüsselrotation,
- Account-Recovery,
- Neuinstallation.

Bei wichtigen Kontakten sollte die Änderung dennoch verifiziert werden.

11.5 Human Proof

Der Human-Proof-Workflow erzeugt beziehungsweise erneuert einen kryptografischen Nachweis menschlicher Interaktion. Er ist Teil des Passport-/Trust-Systems und keine universelle Identitätsfeststellung im rechtlichen Sinn.

12. Quanten-Chat / Direct Messaging

12.1 Chat öffnen

Quanten-Chat öffnen und einen Kontakt auswählen. Der Header kann Identität, Präsenz, Trust-/Rolleninformationen und den E2EE-/Quantum-Status anzeigen.

12.2 Nachricht senden

Native Chatnachrichten werden vor dem Upload clientseitig in ein verschlüsseltes Envelope überführt. Das Backend routet beziehungsweise speichert die Envelope-Daten, ohne den Nachrichteninhalt für den normalen Zustellpfad entschlüsseln zu müssen.

12.3 Accelerated und Top Secret

Accelerated

Geeignet als regulärer Sovereign-Modus.

Top Secret

Erhöhte Kaskade und strengere Capability-Anforderungen.

Der Client muss **fail closed** handeln: Wenn erforderliche Primitive, Empfängerschlüssel oder Capability-Nachweise fehlen, darf er nicht still auf eine schwächere Variante zurückfallen.

12.4 Präsenz und Tippstatus

Der Chat unterstützt Heartbeats/Presence und Tippindikatoren. Das sind Metadatenfunktionen; sie sind nicht mit Inhaltsverschlüsselung gleichzusetzen.

12.5 Nachrichtenaktionen

Je nach Nachricht und Berechtigung:

- Antworten
- Bearbeiten
- Löschen
- Reaktionen
- lokal speichern
- lokal anheften
- Lesezustand

12.6 Chat durchsuchen

Die Oberfläche enthält Suchfunktionen für die aktuelle Unterhaltung. Wo Suche lokal über entschlüsselte Daten erfolgt, sollte sie nicht als serverseitige Klartextsuche missverstanden werden.

12.7 Kontakt blockieren oder melden

Im Chat können Kontakte blockiert beziehungsweise über den Abuse-Workflow gemeldet werden.

12.8 GaiaProof / Export

Für relevante Kommunikationsverläufe können GaiaProof-/Exportfunktionen verfügbar sein. Verschlüsselte Exporte sollten mit einem eigenständigen, starken Exportpasswort geschützt werden.

12.9 Anhänge und Voice Notes

Im Quellcode sind UI- und Entwicklungsbestandteile für Dateien, GaiaDrive-Sharing und Voice Notes vorhanden.

Release-Grenze: Im Sovereign-Beta-v2-Vertrag sind Chat-Datei-/Medienanhänge noch nicht als vollständig freigegeben zu behandeln. Für Produktionsdokumentation gelten sie als experimentell, bis das vorgesehene Audit abgeschlossen und die Release-Matrix aktualisiert wurde.

13. GaiaMail

GaiaMail bildet ein mailbox-orientiertes Kommunikationsmodell innerhalb der GaiaCom-Anwendung ab.

13.1 Neue Mail verfassen

Composer öffnen und den nativen GaiaCom-Modus verwenden.

Typische Felder:

- Empfänger
- Betreff
- Nachrichtentext
- Formatierungsaktionen wie Fett, Kursiv, Unterstreichen, Überschrift und Code

Für native Empfänger nutzt die Oberfläche GaiaID-/Kontaktinformationen.

13.2 Entwürfe

Der Composer unterstützt Entwurfsverwaltung und automatisches beziehungsweise regelmäßiges Speichern des Draft-Zustands.

13.3 Zeitgeplanter Versand

Backend und Composer enthalten einen Scheduler-Pfad für zeitversetzten Versand.

Bei geplanten Nachrichten muss der Node zum relevanten Zeitpunkt lauffähig sein und die geplante Zustellung verarbeiten können.

13.4 Nachricht lesen

Im Reader stehen abhängig vom Nachrichtentyp unter anderem zur Verfügung:

- Antworten
- Allen antworten
- Weiterleiten
- Archivieren
- Löschen
- Spam
- Snooze
- Labels
- GaiaProof-/Disclosure-Funktionen
- Absender-/Empfängerprofil

13.5 Warnbanner

Der Reader kann Warnungen anzeigen für:

- Spam/Quarantäne
- SMTP Legacy/Downgrade
- geänderte Kontaktschlüssel

13.6 Filterregeln

Mailbox-Regeln können anhand von Kriterien wie Sender oder Betreff Aktionen anwenden, beispielsweise:

- Important
- Star
- Read
- Label

13.7 Signatur und Mailbox-Einstellungen

Im Profil stehen Mailbox-Einstellungen wie Signatur, Teil-Locale und keyboard-orientierte Bedienoptionen bereit.

14. SMTP Legacy Bridge

Die SMTP Bridge ermöglicht Interoperabilität mit klassischer E-Mail.

14.1 Sicherheitsbedeutung

SMTP ist **kein nativer GaiaCom-E2EE-Pfad**. Sobald eine Nachricht an das Legacy-Gateway übergeben wird, kann GaiaCom die Sicherheitsgarantien außerhalb der eigenen Infrastruktur nicht erzwingen.

Die UI muss diesen Wechsel sichtbar als Downgrade kennzeichnen.

14.2 SMTP-Nachricht verfassen

Im Composer SMTP/Legacy auswählen. Externe E-Mail-Adressen sind dann zulässig.

Vor dem Versand sollte der Benutzer die Downgrade-Warnung bewusst zur Kenntnis nehmen.

14.3 Betreiberpflichten

Für einen produktiven SMTP-Betrieb sind zusätzlich erforderlich:

- authentifizierter SMTP-Relay
- verpflichtendes TLS
- saubere Absenderdomain
- SPF
- DKIM
- DMARC mit sinnvoller Enforcement-Policy
- Schutz vor Header-/CRLF-Injection
- Monitoring von Abuse/Rate-Limits

15. Gruppen, Räume und Unterkanäle

15.1 Raum erstellen

Über **Gruppen-Chats** kann ein neuer Raum erstellt werden.

Mögliche Eigenschaften:

- Name
- Beschreibung
- Avatar
- Private Room
- Crisis Room
- Top Secret

15.2 Privater Raum

Ein privater Raum soll nicht wie ein normal öffentlicher Raum auffindbar sein. Beitritt kann über Einladung oder Join-Request erfolgen.

15.3 Unterkanäle

Innerhalb eines Raums können Kanäle zur thematischen Trennung angelegt werden.

15.4 Slow Mode

Der Gruppenbereich kennt Slow-Mode-Intervalle, unter anderem:

0 / 5 / 10 / 30 / 60 / 300 Sekunden

15.5 Mitglieder und Rollen

Je nach Berechtigung:

- Mitglied
- Admin
- Owner

Operatoren eines Raums können Mitglieder suchen, Rollen ändern, Mitglieder entfernen und Eigentümerschaft übertragen.

15.6 Join Requests

Bei privaten Räumen können Beitrittsanfragen akzeptiert oder abgelehnt werden.

15.7 Einladungslinks

Einladungen können zeitlich beziehungsweise nach Nutzungen begrenzt werden, zum Beispiel:

- 1 Stunde
- 1 Tag
- 7 Tage
- unbegrenzt

und:

- 1 Nutzung
- 5 Nutzungen
- 10 Nutzungen
- unbegrenzt

15.8 Moderationslogs

Raumrelevante Admin-/Moderationsaktionen werden für die dafür vorgesehene Auditierbarkeit erfasst.

16. Public Channels

Public Channels sind öffentliche Broadcast-/Diskussionsbereiche.

16.1 Kanäle finden und abonnieren

Benutzer können Kanäle entdecken, durchsuchen und abonnieren beziehungsweise deabonnieren.

16.2 Posts

Je nach Berechtigung:

- Post erstellen
- Post bearbeiten
- Post löschen
- Post anheften
- Reaktion hinzufügen

16.3 Kommentare

Kanäle können Kommentare erlauben oder deaktivieren. Moderatoren können Kommentare verwalten.

16.4 Moderation und Reports

Kanäle beziehungsweise Inhalte können gemeldet werden. Kategorien im System umfassen beispielsweise:

- Spam
- Phishing
- Malware
- Belästigung
- illegale Inhalte
- Bedrohung
- Sonstiges

Ein Report ist ein Governance-Signal, kein automatischer Beweis.

17. GSN – GaiaSocialNetwork

GSN ist die dezentrale Social-Layer-Komponente von GaiaCom.

17.1 Feeds

Je nach Ansicht:

- Node-/allgemeiner Feed
- Following-Feed

17.2 Beiträge

Benutzer können:

- Posts erstellen
- eigene Posts löschen
- reagieren
- kommentieren

17.3 Profile und Follows

GSN-Profile enthalten abhängig von den Einstellungen beispielsweise:

- Anzeigename
- Bio
- Website
- Avatar

Andere Profile können gefolgt beziehungsweise entfolgt werden.

17.4 Melden

Profile und Beiträge können in den Abuse-/Governance-Pfad gemeldet werden.

17.5 Verbindung zu anderen GaiaCom-Bereichen

Die UI verknüpft Social-Profile mit Chat, Räumen und öffentlichen Kanälen, damit zwischen öffentlicher und privater Kommunikation gewechselt werden kann.

18. GaiaDrive

GaiaDrive ist der aktuelle verschlüsselte persönliche Speicherbereich. Der ältere `VaultPane / useVault`-Pfad ist im Code als Legacy zu betrachten.

18.1 Drive entsperren

Der Benutzer entsperrt GaiaDrive lokal mit seinem Drive-Passwort. Dieses Passwort dient der lokalen Schlüsselableitung und sollte nicht mit einem schwachen Convenience-Passwort gleichgesetzt werden.

18.2 Notizen

GaiaDrive kann verschlüsselte lokale Notizen verwalten.

Mögliche Aktionen:

- Notiz erstellen
- Kategorie vergeben
- entschlüsselt anzeigen
- kopieren
- löschen

18.3 Lokale Dateien

Dateien werden lokal über Browser-/Client-Speichermechanismen wie OPFS verwaltet und verschlüsselt abgelegt.

18.4 Cloud-Synchronisation

Für unterstützte Dateien kann ein clientseitig verschlüsselter Chunk-Upload zum Node erfolgen. Der Server sieht dabei die verschlüsselten Chunks und Metadaten, nicht den entschlüsselten Dateiinhalt.

Die aktuelle UI setzt zusätzliche Größen-/TTL-Grenzen für bestimmte Sync-Funktionen. Diese Grenzen sollten vor Release mit Serverkonfiguration und README synchron gehalten werden.

18.5 Cross-Device-Sync

Die vollständige Schlüsselverteilung und Recovery-Zeremonie für nahtlosen Cross-Device-GaiaDrive-Sync ist im Beta-Stand noch eine bekannte Grenze.

19. GaiaDrop

GaiaDrop ist für externe Texteingaben an eine GaiaID ausgelegt.

19.1 Einsendung ohne normalen Account-Login

Auf dem Auth-/öffentlichen Einstieg:

1. GaiaDrop auswählen.
2. Empfänger-GaiaID eingeben.
3. Text verfassen.
4. Einsendung absenden.

Der aktuelle Release-Pfad ist **textorientiert**. Attachment-GaiaDrop ist noch nicht als freigegebenes Feature zu behandeln.

19.2 GaiaDrop-Postfach

Der Empfänger öffnet den GaiaDrop-Bereich und lädt seine Einreichungen.

Im Detail können unter anderem angezeigt werden:

- Zeitstempel
- Drop-ID
- Payload-Hash
- Senderlabel oder anonymisierter Status, soweit vorhanden
- nach Client-Entschlüsselung der Text

19.3 Sicherheitsgrenze „anonym“

GaiaDrop soll eine niederschwellige beziehungsweise nicht zwingend accountgebundene Einsendung ermöglichen. Das darf nicht als mathematische Netzwerk-Anonymitätsgarantie vermarktet werden. Netzwerk- und Betriebsmetadaten liegen außerhalb einer solchen Garantie.

20. Network Health

Network Health stellt aggregierte Netzwerk-/Node-Zustände dar.

Typische Daten:

- Accounts
- Identitäten
- Nodes
- Räume
- Nachrichten innerhalb eines Zeitfensters
- GaiaDrop-Aktivität
- Föderationsaktivität
- Uptime
- Protokollversion
- Krypto-/Capability-Status
- Federation-Status
- SMTP-Bridge-Status

Im Profil existiert eine Privacy-Einstellung für anonyme beziehungsweise aggregierte Telemetrie. Persönliche Kommunikationsgraphen oder GaiaIDs sollten nicht als normale öffentliche Health-Metrik behandelt werden.

21. Security Center / GaiaShield

Das Security Center ist die Benutzer- und Operatoroberfläche für GaiaShield und verwandte Security-Funktionen.

21.1 Protection Status

Hier erscheinen je nach Berechtigung:

- Accountschutz
- offene Warnungen
- fehlgeschlagene Loginversuche
- Message-/Replay-Anomalien
- Security Checks

Warnungen können nach Prüfung quittiert werden.

21.2 Security Log

Security-Ereignisse lassen sich nach Severity/Kategorie filtern und für Analysezwecke exportieren, beispielsweise als JSON oder CSV.

21.3 Gaia Passport

Der Passport-Bereich zeigt unter anderem Human-Proof- und Trust-Informationen. Ein Human Proof kann erneuert werden.

21.4 Node Security

Operatoren erhalten zusätzliche Node-bezogene Ereignisse und Statistiken.

GaiaShield erfasst im Design unter anderem:

- Auth-Angriffe
- malformed requests
- BOLA/BFLA-Versuche
- Storage-ACL-Verstöße
- Federation-Anomalien
- SMTP-Abuse
- Governance-Abuse
- GSN-Spam
- Upload-Abuse

Sensible Felder werden vor Persistenz beziehungsweise Anzeige redigiert oder gehasht, soweit vom Security-Subsystem vorgesehen.

21.5 Nodesystem

Der Nodesystem-Bereich zeigt Registry-/Nodeinformationen und Diagnoseaktionen.

Produktionsregel: Node-Secrets nicht primär über Browser-UI provisionieren. Für einen Produktionsnode `gaiacon setup` und `gaiacon doctor` verwenden.

22. Abuse Center, Governance und Gaia's Eyes

22.1 Rollen

Der Governance-Layer unterscheidet abhängig vom Bootstrap-/Credential-Modell Rollen wie:

- User
- Reviewer
- Senior Reviewer
- Node Operator

Rollenrechte müssen serverseitig geprüft werden.

22.2 Eigene Reports

Benutzer sehen eingereichte Fälle und deren Status. Je nach Workflow kann eine Berufung/Appeal eingereicht werden.

22.3 Reviewer Queue

Reviewer erhalten policy-basierte Fälle und können – nach serverseitiger Berechtigungsprüfung – Bewertungen beziehungsweise vorgeschlagene Aktionen signieren.

Mögliche Maßnahmen können zum Beispiel sein:

- reject
- quarantine
- suspend

Die konkrete Schwellenwertlogik ist Policy-/Governance-abhängig.

22.4 Gaia's Eyes

Gaia's Eyes ist kein „Godmode“. Der Workflow soll kontrollierte, begrenzte Offenlegung ermöglichen, wenn der betroffene Client beziehungsweise die dafür vorgesehene kryptografische Disclosure-Logik Material bereitstellt.

22.5 Node-Operator-Aktionen

Operatoren können für ihren Governance-Bereich Node-/Abuse-Aktionen anwenden und Transparenzinformationen veröffentlichen.

23. Profil und persönliche Einstellungen

23.1 Profil bearbeiten

Mögliche Felder:

- Anzeigename
- optionaler Realname
- Website
- Bio/Status
- Avatar

Uploads und Profilinhalte müssen client- und serverseitig sanitisiert beziehungsweise validiert bleiben.

23.2 Passwort ändern

Benötigt:

- aktuelles Passwort
- neues Passwort
- Bestätigung

Das Serverpasswort ist nicht dasselbe wie Recovery-Mnemonic oder ein lokales GaiaDrive-/Export-Passwort.

23.3 Geräte

Siehe Kapitel 9. Im Profil können Sessions und Crypto-Geräte verwaltet werden.

23.4 Schlüssel

Nach erneuter Authentisierung kann der Client – je nach UI – lokale private Schlüssel und die Mnemonic anzeigen.

Dieser Bereich ist besonders sensibel. Bildschirmfreigaben, Browserextensions, Malware oder physischer Zugriff auf ein entsperartes Gerät können den Schutz unterlaufen.

23.5 Recovery exportieren

Recovery-Dateien werden verschlüsselt exportiert. Ein eigenständiges starkes Passwort verwenden und die Datei offline beziehungsweise getrennt vom Passwort lagern.

23.6 Crypto Lock

Konfigurierbare Elemente können umfassen:

- Dauer der Crypto-Session
- Inaktivitäts-Autosperre
- PIN-Unlock
- WebAuthn PRF

23.7 Privacy

Der Benutzer kann die Teilnahme an anonymisierten/aggregierten Netzwerkstatistiken konfigurieren.

23.8 Accessibility

Im Client sind unter anderem vorgesehen:

- Font Scale: 90 %, 100 %, 112 %, 125 %
- High Contrast
- Reduce Motion

23.9 Notifications

Je nach Client:

- Benachrichtigungen an/aus
- Inhaltsvorschau
- Quiet Hours
- Start-/Endzeit

23.10 Sprache

Der globale i18n-Layer des bereitgestellten Frontends enthält 18 Sprachoptionen:

Deutsch, English, Русский, Español, Français, فارسی, 日本語, Português, العربية, 简体中文, हिन्दी, Türkçe, Italiano, Polski, Українська, 한국어, Indonesia und Shqip.

Einzelne Unterdialoge besitzen teilweise noch kleinere Locale-Auswahllisten als der globale Client.

23.11 Theme

Light-/Dark-Mode kann über die Navigation gewechselt werden.

23.12 Konto löschen

Der Danger-Bereich verlangt eine explizite Bestätigung und das aktuelle Passwort.

Die Löschung ist als irreversible Aktion zu behandeln. Vorher erforderliche Recovery-/Exportdaten sichern.

24. Tastaturkürzel

Der Client enthält Gmail-artige Navigations-/Arbeitskürzel. Sie werden nicht ausgelöst, während in geeigneten Eingabefeldern geschrieben wird.

Kürzel	Aktion
g danach i	Inbox
g danach s	Sent
g danach c	Chat
g danach g	Groups
g danach p	Profil
c	Compose
r	Reply
f	Forward
e	Archive
Delete, Backspace oder #	Trash
s	Star umschalten

Teil II – Node-Operator- und Deployment-Handbuch

25. Betriebsmodell

Ein produktiver GaiaCom-Node besteht im empfohlenen Linux-Pfad aus:



Port `8080` soll im Standard-Produktionsmodell **nicht direkt ins Internet exponiert** werden.

26. Voraussetzungen

26.1 Build-System

Je nach zu bauender Komponente:

- Go Toolchain passend zum Repository (`go 1.25.0` , Toolchain-Hinweis `go1.26.5`)
- Node.js passend zu `.node-version`
- npm
- Rust/Cargo
- `wasm-pack` für WASM
- Tauri-Toolchain für Desktop
- Python für Hygiene-/Security-Scripts
- Android Studio/Gradle/JDK für Android

26.2 Produktionsserver

Empfohlen im bereitgestellten Deploymentpfad:

- Linux AMD64
- NGINX
- systemd
- TLS-Zertifikat
- dedizierter Systembenutzer `gaiacom`
- lokales SQLite auf persistentem Storage
- separater persistenter GaiaCom-Storage-Pfad
- DNS/FQDN

27. Lokale Entwicklungsumgebung

27.1 Repository

Repository entpacken beziehungsweise klonen und im Root arbeiten.

27.2 Backend-Umgebung

Das Backend lädt nicht automatisch eine `.env`-Datei wie eine typische `godotenv`-Anwendung. Variablen müssen in die Prozessumgebung exportiert beziehungsweise durch Shell/Service geladen werden.

Ein minimales **lokales** Beispiel:

```
cd Backend

export GAIACOM_DEV_MODE=true
export SERVER_BIND_ADDRESS=127.0.0.1
export SERVER_PORT=8080
export DB_DRIVER=sqlite
export DB_PATH="$PWD/gaiacom-dev.db"
export GAIACOM_COOKIE_SECURE=false

go mod verify
go test ./...
go run ./cmd/gaiacom
```

Im Dev-Modus dürfen Produktions-Secrets nicht als echte Deploy-Secrets wiederverwendet werden.

27.3 Frontend

Für die im Backend vorhandene Dev-Origin-Konfiguration ist Port `3000` der robustere Pfad:

```
cd Frontend/frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund

VITE_API_URL=http://localhost:8080 \
npx vite --host localhost --port 3000
```

Unter PowerShell sinngemäß:

```
cd Frontend\frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
$env:VITE_API_URL='http://localhost:8080'
npx vite --host localhost --port 3000
```

27.4 Warum nicht einfach `npm start` ?

Das vorhandene Skript lautet:

```
vite --host 127.0.0.1
```

Ohne expliziten Port kann es auf Vites Standardport landen, während der Backend-Dev-CORS-Stand andere Origins freigibt. Deshalb in einer reproduzierbaren Entwickleranleitung Host und Port bewusst angeben.

28. Produktionsbuild

28.1 Frontend

Bash:

```
cd Frontend/frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
npm test
npm run test:hqc-browser
VITE_API_URL=https://node.example.org npm run build
```

PowerShell:

```
cd Frontend\frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
$env:VITE_API_URL='https://node.example.org'
npm test
npm run build
npm run test:hqc-browser
```

Ausgabe:

```
Frontend/frontend/dist/
```

28.2 Backend Linux AMD64

```
cd Backend
go mod verify
go test ./...

VERSION=2.0.0
COMMIT="$(git rev-parse HEAD)"
BUILT_AT="$(date -u +%Y-%m-%dT%H:%M:%SZ)"

CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \
go build -trimpath -buildvcs=false \
-ldflags="-s -w" \
-X gaiacom/backend/buildinfo.Version=${VERSION} \
-X gaiacom/backend/buildinfo.Commit=${COMMIT} \
-X gaiacom/backend/buildinfo.BuiltAt=${BUILT_AT}" \
-o gaiacom-backend-linux-amd64 ./cmd/gaiacom
```

Die Release-Metadaten sind im Produktionspfad sicherheits-/betriebsrelevant. Ein Produktionsserver soll nicht mit einem ungültigen Development-Release-Marker gestartet werden.

29. Linux-Server vorbereiten

Beispiel Debian/Ubuntu:

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y nginx certbot python3-certbot-nginx

sudo useradd -r -s /bin/false -d /opt/gaiacom gaiacom

sudo install -d -o root -g root -m 0755 /opt/gaiacom
sudo install -d -o root -g root -m 0755 /var/www/gaiacom
sudo install -d -o root -g root -m 0700 /etc/gaiacom
sudo install -d -o gaiacom -g gaiacom -m 0700 /var/lib/gaiacom
sudo install -d -o gaiacom -g gaiacom -m 0700 /opt/gaiacom/storage
```

Backend-Binary und Frontend anschließend kontrolliert übertragen. Binary und Frontend sollten nicht dem unprivilegierten Runtime-User schreibbar gehören, wenn das nicht zwingend erforderlich ist.

30. Produktionskonfiguration mit **gaiacom** setup

30.1 Canonical Setup

```
sudo /opt/gaiacom/gaiacom-backend setup \
--config /etc/gaiacom/gaiacom.env \
--server-name node.example.org \
--db-path /var/lib/gaiacom/gaiacom.db \
--port 8080
```

Danach:

```
sudo chmod 600 /etc/gaiacom/gaiacom.env
sudo chown root:root /etc/gaiacom/gaiacom.env
```

30.2 Was **setup** setzt beziehungsweise erzeugt

Der aktuelle Provisioning-Pfad setzt unter anderem:

```
GAIACOM_DEV_MODE=false
GAIACOM_SERVER_NAME=<FQDN>
SERVER_BIND_ADDRESS=127.0.0.1
GAIACOM_ALLOW_PUBLIC_BIND=false
DB_DRIVER=sqlite
GAIACOM_COOKIE_SECURE=true
```

und erzeugt beziehungsweise bewahrt gültige Secrets für:

- JWT
- GaiaShield
- Metrics
- TrustMesh Epoch
- Ed25519 Server Identity

Die Konfigurationsdatei wird atomar und mit restriktiven Rechten angelegt.

30.3 Idempotenz

setup ist so angelegt, dass gültige bestehende Secrets nicht bei jedem Lauf ersetzt werden. Ein Server-Name-Mismatch oder fehlerhafte bestehende Secrets sollen nicht still überschrieben werden.

Deshalb: Nicht „zur Sicherheit“ vor jedem Start neue Secrets generieren.

31. gaiacom doctor und gaiacom version

31.1 Doctor

```
sudo /opt/gaiacom/gaiacom-backend doctor \
--config /etc/gaiacom/gaiacom.env
```

Der Doctor prüft unter anderem:

- Konfigurationssyntax
- Secret-Stärke/Formate
- Serveridentität
- DB-Verzeichnis
- Dateirechte
- produktionskritische Einstellungen

Unter Unix darf die Produktions-Environment-Datei keine unnötigen Group-/World-Rechte besitzen.

31.2 Version

```
/opt/gaiacom/gaiacom-backend version
```

Ausgabe enthält Release-/Protokoll-/Buildinformationen wie Version, Commit und Buildzeit.

32. Governance-Bootstrap

Für den initialen Node-Operator wird eine Governance-Konfiguration vorgesehen.

Beispiel:

```
{  
  "bootstrap_gaia_id": "@operator:node.example.org"  
}
```

Speichern als:

```
/etc/gaiacom/governance.json
```

Rechte restriktiv setzen:

```
sudo chown root:root /etc/gaiacom/governance.json  
sudo chmod 600 /etc/gaiacom/governance.json
```

Konfigurationspfad in die Environment aufnehmen:

```
echo 'GAIACOM_GOVERNANCE_CONFIG=/etc/gaiacom/governance.json' \  
| sudo tee -a /etc/gaiacom/gaiacom.env >/dev/null
```

Die Bootstrap-GaiaID muss nach dem Start tatsächlich als passende Identität existieren beziehungsweise in den vorgesehenen Bootstrap-Ablauf gebracht werden.

33. Systemd-Härtung

Das Repository liefert eine gehärtete `deploy/systemd/gaiacom.service` .

Enthaltene beziehungsweise vorgesehene Schutzmaßnahmen umfassen:

- unprivilegierter User `gaiacom`
- `NoNewPrivileges=true`
- `PrivateTmp=true`
- `PrivateDevices=true`
- `ProtectSystem=strict`
- `ProtectHome=true`
- restriktive Capability-Grenzen
- `LockPersonality=true`
- `RestrictSUIDSGID=true`
- `UMask=0077`

Nach Installation:

```
sudo systemctl daemon-reload
sudo systemctl enable --now gaiacom
sudo systemctl status gaiacom
```

Logs:

```
sudo journalctl -u gaiacom -f
```

34. NGINX, TLS und Reverse Proxy

34.1 Routing

NGINX soll:

- `/` aus dem gebauten Frontend bedienen,
- `/api/` an `127.0.0.1:8080` leiten,
- `/livez` und `/readyz` intern weiterreichen,
- `/.well-known/gaiacom/` an das Backend weiterreichen,
- `/metrics` nicht öffentlich exponieren.

34.2 Security Header

Der mitgelieferte VHost sieht strikte Header vor, darunter sinngemäß:

- Content-Security-Policy
- `X-Frame-Options: DENY`
- `X-Content-Type-Options: nosniff`

Die CSP darf nicht leichtfertig durch breite `*`, `unsafe-inline` oder zusätzliche fremde Origins aufgeweicht werden, weil insbesondere der Webclient Teil des kryptografischen Vertrauenspfads ist.

34.3 TLS

TLS-Zertifikat mit Certbot oder gleichwertiger Infrastruktur einrichten und automatische Erneuerung überwachen.

35. Firewall und Netzwerkexposition

Minimal öffentlich:

- SSH, sofern benötigt
- TCP 80 für Redirect/ACME
- TCP 443 für GaiaCom HTTPS

Beispiel UFW:

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

Nicht öffentlich freigeben: Backend-Port `8080`, sofern nicht absichtlich ein abweichendes, separat gehärtetes Betriebsmodell verwendet wird.

36. Node Registry und kontrollierte Föderation

36.1 Voraussetzungen

Ein Node benötigt:

- gültigen FQDN
- DNS
- HTTPS
- Server-Signaturschlüssel
- korrekte Well-Known-Endpunkte
- Registry-/Authority-Konfiguration
- gültige Release-/Core-Informationen

36.2 Well-Known-Endpunkte

Zentrale Federation Discovery-/Transportpfade umfassen:

```
GET /.well-known/gaiacom/server
GET /.well-known/gaiacom/nodeinfo
GET /.well-known/gaiacom/nodes
POST /.well-known/gaiacom/s2s/v1/forward
```

36.3 Beitritt

Typischer Ablauf:

1. Node deployen.
2. `setup` und `doctor` erfolgreich ausführen.
3. Service und TLS aktivieren.
4. Bootstrap-Operator-GaiaID bereitstellen.
5. Im Nodesystem Main Node / Registry anpingen.
6. Zulassung/Quarantäne/Blockstatus prüfen.
7. Föderierte Testnachricht durchführen.

36.4 Kontrollierte Föderation

Sovereign Beta v2 ist **keine frei offene Federation**. Nodes werden kontrolliert zugelassen.

36.5 S2S-Security

Der Föderationspfad enthält Schutzmechanismen gegen:

- ungültige Signaturen
 - Replay
 - falsche Zeitfenster
 - Body-Hash-Manipulation
 - falsches Ziel
 - SSRF zu privaten/unerlaubten Netzen
-

37. Speicher: lokales Object Store und S3/MinIO

37.1 Lokales Object Store

Standardmäßig kann der Backend-Storage verschlüsselte Chunks im lokalen Dateisystem halten.

Konfigurationsrelevante Variablen:

```
GAIACOM_STORAGE_ROOT  
GAIACOM_STORAGE_USER_QUOTA_BYTES  
GAIACOM_STORAGE_PENDING_TTL_HOURS
```

Der Object-Store-Code enthält Path-Jail-/Pfadvalidierungslogik. Diese Schutzschicht darf bei eigenen Adaptionen nicht umgangen werden.

37.2 S3/MinIO

Optional:

```
GAIACOM_OBJECT_STORE=s3  
GAIACOM_S3_ENDPOINT  
GAIACOM_S3_BUCKET  
GAIACOM_S3_REGION  
GAIACOM_S3_ACCESS_KEY  
GAIACOM_S3_SECRET_KEY  
GAIACOM_S3_PREFIX  
GAIACOM_S3_PATH_STYLE
```

Auch bei S3 soll der Object Store nur die bereits clientseitig verschlüsselten Chunk-Daten aufnehmen.

37.3 Zugriffsmodelle

Storage-Metadaten unterstützen Besitzer-/Grant-/Public-artige ACL-Zustände sowie Expiration/Revocation-Pfade.

„Public“ bedeutet nicht, dass ein clientseitig verschlüsseltes Objekt automatisch entschlüsselt öffentlich wird; Zugriffs- und Kryptografiemodell müssen gemeinsam betrachtet werden.

38. SMTP produktiv konfigurieren

Relevante Variablen im Repository/Dokumentationsstand:

```
GAIACOM_SMTP_HOST  
GAIACOM_SMTP_PORT  
GAIACOM_SMTP_FROM  
GAIACOM_SMTP_USERNAME  
GAIACOM_SMTP_PASSWORD  
GAIACOM_SMTP_INGEST_TOKEN  
GAIACOM_SMTP_TLS_MODE
```

Typischer Port ist 587 für STARTTLS, abhängig vom Relay.

Produktionscheck:

1. Relay authentifiziert?
2. TLS erzwungen?
3. `From`-Domain korrekt?
4. SPF gesetzt?
5. DKIM signiert?
6. DMARC aktiv?
7. Bounce-/Abuse-Verhalten beobachtet?
8. Ingest-Token stark und geheim?
9. SMTP-Endpoint rate-limited?
10. Downgrade in UI sichtbar?

39. Healthchecks, Metrics und Logs

39.1 Liveness

```
curl --fail --silent https://node.example.org/livez
```

39.2 Readiness

```
curl --fail --silent https://node.example.org/readyz
```

39.3 Nodeinfo

```
curl --fail --silent \
https://node.example.org/.well-known/gaiacom/nodeinfo
```

39.4 Nodes

```
curl -s https://node.example.org/.well-known/gaiacom/nodes
```

39.5 Public Version

`/api/v1/public/version` sollte die tatsächlich gebaute Version, Commit-ID und Buildzeit widerspiegeln.

39.6 Metrics

`/metrics` ist im vorgesehenen Setup **nicht öffentlich**. Lokal beziehungsweise über autorisierten Monitoring-Zugriff benötigt der Endpoint das Metrics-Token. Der Schutzpfad soll bei fehlendem/falschem Token keine unnötigen Details preisgeben.

39.7 Journald

```
sudo journalctl -u gaiacom --since today
sudo journalctl -u gaiacom -f
```

Secrets oder Klartext-Keymaterial dürfen nicht in Logs landen.

40. Backups und Wiederherstellung

40.1 SQLite

Vor Updates eine konsistente SQLite-Sicherung erstellen, beispielsweise über die SQLite-Backup-Funktion beziehungsweise `.backup`.

Beispiel:

```
sudo systemctl stop gaiacom
sqlite3 /var/lib/gaiacom/gaiacom.db \
    ".backup '/var/backups/gaiacom/gaiacom-$(date +%F-%H%M%S).db'"
sudo systemctl start gaiacom
```

40.2 Migration-Snapshots

Der interne Migrationsmanager erstellt Sicherungen vor anstehenden Schema-Migrationen. Diese sind **kein Ersatz für eine vollständige Backupstrategie**.

40.3 Object Store

DB und Object Store müssen koordiniert gesichert werden. Eine Datenbankkopie ohne die zugehörigen Chunk-Objekte kann unvollständig sein.

40.4 Secrets

Produktions-Secrets getrennt und verschlüsselt sichern:

- Environment
- Governance-Konfiguration
- erforderliche serverseitige Identitätsschlüssel

Der Verlust von Server-Identity- oder Trust-bezogenen Secrets kann Föderations-/Verifikationsfolgen haben.

40.5 Client Recovery

Serverbackup ersetzt keine Benutzer-Recovery. Private Client-Schlüssel/Mnemonics sind bewusst nicht als Klartext-Godmode-Backup auf dem Server vorhanden.

41. Updates und Rollback

41.1 Vor dem Update

- Release Notes prüfen.
- Tests/Gates im Build ausführen.
- DB sichern.
- Object Store sichern.
- Secrets/Config sichern.
- aktuellen Binary-/Frontend-Build archivieren.

41.2 Update

1. Service stoppen.
2. neues Backend-Binary installieren.
3. neues Frontend atomar beziehungsweise kontrolliert deployen.
4. Service starten.
5. `livez`, `readyz`, `nodeinfo`, Version und Federation prüfen.
6. Logs auf Migration-/Runtime-Fehler prüfen.

41.3 Rollback

Ein Binary-Rollback ist nur sicher, wenn das ältere Binary mit dem inzwischen migrierten DB-Schema kompatibel ist.

Bei inkompatibler Migration muss das **vorherige DB-Backup zusammen mit dem dazu passenden Object-Store-Zustand** zurückgespielt werden.

42. Produktions-Checkliste

Vor öffentlichem Betrieb:

- ☐ FQDN und DNS korrekt.
- ☐ TLS gültig und Auto-Renewal geprüft.
- ☐ Backend nur auf Loopback gebunden.
- ☐ `GAIACOM_ALLOW_PUBLIC_BIND=false`, sofern kein bewusst anderes Modell.
- ☐ `gaiacom setup` erfolgreich.
- ☐ `gaiacom doctor` ohne kritische Fehler.
- ☐ Konfigurationsdatei `0600`.
- ☐ Secrets stark und nicht in Git.
- ☐ Governance-Bootstrap korrekt.
- ☐ systemd-Härtung aktiv.
- ☐ NGINX-Härtung aktiv.
- ☐ `/metrics` extern nicht erreichbar.
- ☐ `livez` / `readyz` erfolgreich.
- ☐ Version/Commit/Buildzeit korrekt.
- ☐ SQLite-/Object-Store-Backup getestet.
- ☐ SMTP TLS/SPF/DKIM/DMARC geprüft, falls aktiviert.
- ☐ Federation kontrolliert getestet.
- ☐ Browser-CSP nicht aufgeweicht.
- ☐ Security-/Hygiene-Gates erfolgreich.
- ☐ AGPLv3-Lizenzierung in LICENSE, README, Paketmetadaten und Quellcode-Hinweisen konsistent.
- ☐ Nicht freigegebene Beta-Features nicht als produktionsreif beworben.

43. Repository-Struktur

Vereinfacht:

```
GaiaCom-main/
├── Frontend/frontend/      React 18 / Vite Client
├── Backend/               Go Backend
├── SovereignCryptoCore/   Rust Crypto Core
├── SovereignCryptoWasm/   WASM Bindings
├── DesktopClient/        Tauri 2 Desktop Shell
├── Core/rust/gaiacore/    Zero-/Low-Dependency Core-Primitiven
├── Android/              Kotlin/Compose modulare Mobile-Plattform
├── deploy/               NGINX + systemd
├── security/             PoCs / adversariale Tests
├── scripts/              Hygiene / Architecture Gates
├── docs/                 Betrieb, Security, Federation, Runbooks
├── README.md
├── SECURITY.md
├── SECURITY_INVARIANTS.md
└── ARCHITECTURE.md       kurze Repository-Architekturdatei
```

Die separat bereitgestellte große Architekturkarte ist deutlich umfassender und wird im Anhang dieses Handbuchs vollständig eingebettet.

44. Frontend-Architektur

44.1 Stack

- React 18
- Vite
- Lazy Loading / Suspense
- Web Worker
- WebAssembly
- OPFS
- PWA Service Worker
- lokale i18n-Schicht

44.2 Zentrale Einstiege

```
Frontend/frontend/src/index.jsx
Frontend/frontend/src/App.jsx
Frontend/frontend/src/api.js
Frontend/frontend/src/crypto.js
Frontend/frontend/src/sovereignCrypto.js
```

44.3 UI-Orchestrierung

`App.jsx` verwaltet den globalen UI-/Sessionzustand. `AppMainContent` lädt die jeweils aktive Pane.
`AppModalLayer` bündelt Modal-/Dialogflächen.

44.4 Wichtige Panes

- DashboardPane
- ChatPane
- GroupChatPane
- PublicChannelsPane
- ReaderPane
- ComposerPane
- ProfilePane
- DrivePane
- DropPane
- GsnPane
- SecurityCenter
- AbuseCenter
- NetworkHealthDashboard

44.5 Hooks

Domänenlogik ist unter anderem gekapselt in:

- `useGaiaAuth`
- `useChat`
- `useEmails`
- `useDrive`
- `useGaiaDrop`
- `usePublicChannels`
- `useGsn`
- `usePresence`
- `useKeyChangeDetection`
- `useInactivityLock`

44.6 Crypto Provider

`sovereignCrypto.js` abstrahiert zwischen:

- Web-Provider mit isoliertem Crypto Worker/WASM
- Native Provider über Tauri IPC

Der Providerpfad soll bei fehlenden erforderlichen Kryptoprimitiven nicht still degradiert werden.

45. Backend-Architektur

45.1 Stack

- Go
- `net/http` über eigenes `Backend/httpx`
- pure-Go SQLite
- domänenorientierte Packages
- kein Gin/GORM im aktuellen Architekturpfad

45.2 Entrypoint

```
Backend/cmd/gaiacom/main.go
```

delegiert in den Backend-Lifecycle.

45.3 Hauptdomänen

```
auth/  
devices/  
identity/  
messaging/  
mailbox/  
room/  
publicchannels/  
gsn/  
storage/  
gaiadrop/  
smtpbridge/  
federation/  
noderegistry/  
trustmesh/  
governance/  
internal/security/  
networkhealth/  
operations/  
provision/  
mobileapi/
```

45.4 Security Middleware

`httpx`, GaiaShield und domänenspezifische Authorization-Prüfungen bilden mehrere Schichten. BOLA/BFLA-Schutz darf nicht nur im Frontend stattfinden.

45.5 Scheduler

`scheduler.go` verarbeitet zeitgesteuerte Draft-/Send-Aktionen.

46. Persistenz und SQLite

46.1 Datenbankmodus

Der DB-Layer verwendet pure-Go SQLite und konfiguriert unter anderem:

- WAL
- Foreign Keys
- Busy Timeout
- `synchronous=FULL`

46.2 Repository Layer

`Backend/repository` stellt Store-Interfaces und SQL-Implementierungen bereit.

Domänenadapter decken unter anderem ab:

- Auth
- Devices
- Identity
- Messaging
- Mailbox
- Rooms
- Public Channels
- GSN
- Storage/GaiaDrop
- Federation
- Governance
- Security Events
- Rate Limits
- Presence
- Offline Transport Queue

46.3 Migrationen

Migrationen besitzen ein Ledger/Checksummenmodell und erstellen vor anstehenden Migrationen DB-Snapshots.

46.4 Skalierungsgrenze

Der Beta-Betrieb ist auf **einen Backendprozess pro SQLite-Datenbank** auszulegen. SQLite in diesem Design ist kein aktives Multi-Primary-/Horizontal-Scale-Datenbankcluster.

47. Sovereign Crypto Core, WASM und Desktop

47.1 Hybrid KEM

Das Design kombiniert:

- ephemeral X25519
- ML-KEM-1024
- HQC-256

über eine gemeinsame Schlüsselableitung mit SHA3/HKDF-Bindung.

47.2 Accelerated Cascade

Design:

- Twofish-256-EAX
- AES-256-GCM
- Ed25519-Signatur

47.3 Top Secret Cascade

Design:

- Serpent-256-CTR + HMAC-SHA3-512
- Twofish-256-EAX
- XChaCha20-Poly1305
- AES-256-GCM-SIV
- Ed25519
- ML-DSA-87

47.4 SovereignCryptoCore

Rust-Implementierung der Cipher-Kaskaden und kontrollierter Speicherbereinigung.

47.5 WASM

`SovereignCryptoWasm` exportiert die Rust-Primitiven für den Webclient. Browser-HQC läuft zusätzlich über den lokalen isolierten Provider/Worker-Pfad.

47.6 Desktop Tauri

Der Tauri-Host stellt bewusst eine schmale IPC-Fläche bereit:

```
sovereign_hqc256_keypair  
sovereign_hqc256_encapsulate  
sovereign_hqc256_decapsulate  
sovereign_seal  
sovereign_open
```

Die schmale Bridge reduziert die Menge nativer Funktionen, die aus der Webview erreichbar sein müssen.

47.7 Desktop Custom Node

Im aktuellen Snapshot ist die Desktop-API-Origin auf <https://beta.gaiacom.de> gepinnt. Für einen eigenen Node müssen sowohl die Originlogik als auch die Tauri-CSP geprüft und neu gebaut werden.

48. Android-Plattform

Die Android-Struktur ist modular:

```
:app
:platform:contracts
:platform:kernel
:platform:node
:platform:security
:platform:security-android
:platform:security-auth
:platform:runtime
:platform:remote
:platform:api
:platform:identity
```

Geplante/angelegte Fähigkeiten:

- Jetpack Compose UI
- Embedded Go Node
- Android Keystore
- StrongBox
- Biometrie/PIN
- typisierte API-Fassade
- Wi-Fi Direct/BLE Transportadapter-Grundlagen

Status: technische Beta. Der vollständige Mobile-Sovereign-v1-Message-Pfad ist laut Release-Grenzen noch nicht als komplett produktionsfertig zu betrachten.

49. API-Übersicht

Die vollständige Endpunktreferenz befindet sich in der Architekturkarte in Anhang A. Für den täglichen Betrieb reicht folgende Domänenübersicht.

49.1 Auth

```
/api/v1/auth/register  
/api/v1/auth/login  
/api/v1/auth/refresh  
/api/v1/auth/logout  
/api/v1/auth/status  
/api/v1/auth/change-password  
/api/v1/auth/delete-account
```

49.2 Devices

Gerätesessions, Device Keys und Pairings unter `/api/v1/devices/...`.

49.3 Identity

Identitätserstellung, eigene Identität, öffentliche Identität, Human Proof und Trust Passport.

49.4 Messaging

Senden, Inbox, Read, Edit, Delete, Reactions, Proofs und Presence.

49.5 Mailbox

Nachrichtenstatus, Drafts, Labels, Filter, Mailbox Settings und globale Suche.

49.6 Rooms

Räume, Mitglieder, Kanäle, Join Requests, Einladungen und Moderation.

49.7 Public Channels

Kanäle, Posts, Kommentare, Reaktionen, Subscriptions und Moderation.

49.8 GSN

Feed, Posts, Comments, Reactions, Profiles und Follows.

49.9 Storage

Upload-Init, Chunks, Complete, Download, Grants und Metadaten.

49.10 GaiaDrop

Öffentlicher Submit-Pfad plus authentifizierte Inbox-/Read-/Delete-Funktionen.

49.11 SMTP

Outbound Send und geschützter Ingest.

49.12 Governance / Reports

Reports, Reviewer, Node Abuse, Disclosure und Gaia's Eyes.

49.13 Security / Node

Security Events, Node Security, Registry und Operatoraktionen.

49.14 Public / Operations

Network Health, Version, Transparency, `livez`, `readyz`, geschützte Metrics.

49.15 Federation

Well-Known Discovery und signiertes S2S Forwarding.

50. Security- und Quality-Gates

Der README-Releasepfad dokumentiert einen Gate-Run über Backend, Frontend, Rust, WASM, Tauri und Repository Hygiene.

Beispiel:

```
cd Backend
go mod verify
go test ./...

cd ../Frontend/frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
npm test
npm run build
npm run test:hqc-browser

cd ../../SovereignCryptoCore
cargo test --locked

cd ../SovereignCryptoWasm
cargo test --locked
wasm-pack build --target web --release

cd ../DesktopClient/src-tauri
cargo test --locked

cd ../../
python scripts/repository_hygiene.py
```

Zusätzlich existieren:

```
security/master_security_suite.py
security/total_system_poc_runner.py
security/poc/
security/extreme/
scripts/android_architecture_gate.py
```

PoC-/Adversarial-Bereiche decken unter anderem ab:

- Federation SSRF
- Replay
- Signaturprüfung
- GaiaDrop Größen-/XSS-/Rate-Limit-Verhalten
- GaiaProof
- Room Authorization
- Governance
- Key History
- Secure Disclosure
- SMTP Injection/Downgrade
- Trust Passport
- BOLA/BFLA

- DB-/Concurrency-Stress

Hinweis: Diese Befehle sind der im Repository dokumentierte Release-Gate. Dieses Handbuch behauptet nicht, dass jede Toolchain-Prüfung in der Dokumentationserstellung erneut vollständig ausgeführt wurde.

51. Sicher entwickeln und erweitern

51.1 Neue Funktion zuerst klassifizieren

Vor Implementierung fragen:

- Berührt sie Klartext?
- Berührt sie private Schlüssel?
- Berührt sie Envelopes?
- Berührt sie Federation?
- Berührt sie Authorization?
- Berührt sie Disclosure/Governance?
- Berührt sie Object Storage?
- Fügt sie eine externe Origin oder Dependency hinzu?

Je mehr Fragen mit Ja beantwortet werden, desto stärker muss die Security-Review sein.

51.2 Keine Security nur im UI

Ein ausgeblendeter Button ist keine Zugriffskontrolle.

Authorization muss serverseitig auf dem konkreten Objekt und der konkreten Aktion geprüft werden.

51.3 Kein stiller Downgrade

Wenn Top Secret, PQC-Fähigkeit oder ein Empfängerschlüssel fehlt:

- Fehler sichtbar machen.
- Nicht transparent auf schwächere Sicherheit wechseln.

51.4 CSP als Security Boundary behandeln

Gerade im Webclient ist die ausgelieferte JavaScript-Anwendung Teil des Vertrauenspfads. Neue CDN-Abhängigkeiten, Inline-Skripte oder fremde Origins verändern das Bedrohungsmodell.

51.5 Logs minimieren

Keine:

- Private Keys
- Mnemonics
- Recovery-Secrets
- JWT-Secrets
- SMTP-Passwörter

- Klartext-Nachrichten

in Debug-/Auditlogs.

51.6 Migrationen

Schemaänderungen:

- migration-basiert
- prüfbar
- rückwärts-/rollbackbewusst
- mit Backup
- unter Parallelzugriff testen

51.7 Neue Federation-Pfade

Zusätzlich testen:

- SSRF
- private IP ranges
- DNS-Rebinding
- Replay
- Clock Skew
- Body Hash
- Signature
- Target Confusion
- Queue Poisoning
- Resource Exhaustion

Teil IV – Betriebshilfe

52. Troubleshooting

52.1 Frontend kann Backend lokal nicht erreichen

Symptome: CORS-Fehler, Login schlägt sofort fehl, API „network error“.

Prüfen:

```
VITE_API_URL=http://localhost:8080
Frontend auf http://localhost:3000
Backend auf 127.0.0.1:8080
GAIACOM_DEV_MODE=true
```

Nicht `127.0.0.1:5173` und `localhost:3000` unbemerkt mischen, wenn die CORS-Allowlist nur einen Teil davon enthält.

52.2 `.env` liegt da, wird aber scheinbar ignoriert

Das Backend behandelt eine Datei nicht automatisch wie eine `.env`-Loader-Anwendung.

Lösung:

```
set -a
source /pfad/zu/gaiacom.env
set +a
./gaiacom-backend
```

Im Produktionsbetrieb übernimmt systemd `EnvironmentFile=` aus der bereitgestellten Unit.

52.3 Produktionsserver startet wegen Buildinfo nicht

Produktionsbuild mit Version, Commit und Buildzeit erzeugen. Nicht einfach ein nacktes Development-Binary kopieren.

52.4 `doctor` bemängelt Dateirechte

```
sudo chown root:root /etc/gaiacom/gaiacom.env
sudo chmod 600 /etc/gaiacom/gaiacom.env
```

52.5 `readyz` schlägt fehl

Prüfen:

- DB erreichbar?
- Migrationen erfolgreich?

- Storage-Verzeichnis schreibbar?
- Runtime-Config gültig?
- Logs: `journalctl -u gaiacom`

52.6 `/metrics` gibt 404

Das kann beabsichtigt sein. Metrics sind geschützt; fehlender/falscher Bearer Token soll den Endpoint nicht öffentlich offenlegen. NGINX soll ihn extern ohnehin blockieren.

52.7 Federation funktioniert lokal, aber nicht zwischen Nodes

Prüfen:

- DNS
- TLS
- Server Name
- Registry-Status
- Node-Zulassung
- Well-Known-Endpunkte
- Systemzeit
- Signaturschlüssel
- SSRF/Address Policy
- Zielnode unterstützt erforderliche Security Suite
- Quarantäne/Blockstatus

52.8 Top Secret lässt sich nicht aktivieren

Das ist bei fehlender Capability korrekt.

Prüfen:

- Empfänger-Keyset
- HQC-Provider
- ML-KEM-Key
- erforderliche Signaturkeys
- Clientprovider
- Key-Change-Warnung
- Node-/Protocol-Capability

Nicht durch Entfernen der Prüfung „reparieren“.

52.9 Desktop-Build spricht immer mit `beta.gaiacom.de`

Aktueller Snapshot pinnt die Desktop-Origin. `Frontend/frontend/src/api.js` und die Tauri-CSP prüfen/anpassen und anschließend neu bauen.

52.10 Registrierung ist trotz `OPEN_REGISTRATION=false` möglich

Im geprüften Snapshot ist diese Variable nicht als eindeutiger Gatekeeper im User-Registration-Service verdrahtet.

Wenn geschlossene Benutzerregistrierung benötigt wird, muss der Auth-Endpoint explizit serverseitig gated und getestet werden.

52.11 Chat zeigt Attachment-Funktion, Release sagt „disabled“

Die UI enthält experimentelle Pfade. Für Sovereign Beta v2 gilt die Release-Matrix. Nicht freigegebene Attachment-/Media-Pfade deaktivieren oder sichtbar als experimentell kennzeichnen.

52.12 Migration war erfolgreich, Rollback-Binary startet aber nicht

Schema kann inzwischen inkompatibel sein. Passendes DB-Backup plus Object-Store-Zustand wiederherstellen.

52.13 SMTP funktioniert, aber Nachrichten landen im Spam

Prüfen:

- PTR/rDNS
- SPF
- DKIM
- DMARC
- From Alignment
- Relay Reputation
- TLS
- Rate
- Content
- Bounce Handling

53. Bekannte Grenzen der Sovereign Beta v2

1. **SQLite Deployment:** Ein Backendprozess pro DB; keine aktive Multi-Primary-HA-Datenbank.
2. **Web-Origin-Vertrauen:** Ein kompromittierter Node kann beim Webclient theoretisch manipuliertes JavaScript ausliefern, bevor eine Nachricht verschlüsselt wird.
3. **Endpoint-Kompromittierung:** Entsperrte Geräte und kompromittierte Betriebssysteme bleiben außerhalb der kryptografischen Kernannahme.
4. **Keine Anonymitätsgarantie:** Inhaltsverschlüsselung beseitigt nicht alle Metadaten.
5. **SMTP-Downgrade:** Klassische E-Mail liegt außerhalb des vollständigen GaiaCom-Sicherheitsmodells.
6. **Attachments:** Chat-/Media-Attachments noch nicht vollständig freigegeben.
7. **GaiaDrop:** Releasepfad text-only; Attachments noch nicht freigegeben.
8. **Mobile:** Android-Architektur vorhanden, vollständiger Sovereign-v1-Mobile-Message-Pfad noch technische Beta.
9. **Federation:** kontrolliert, nicht offen.
10. **Governance:** Cross-Node-Policies sind nicht als globaler Konsens zu verstehen.
11. **GaiaDrive Cross Device:** Key-Share-/Recovery-Zeremonie noch Beta-Grenze.
12. **Automatic Updates:** keine vollständig freigegebene automatische Update-Pipeline.
13. **Recovery Operations:** Recovery-Prozesse müssen organisatorisch geprobt werden.
14. **Barrierefreiheit/Low-End-Performance:** Bestandteil der Release-Gates, aber weiterhin als Beta-Qualitätsbereich behandeln.

GaiaCom sollte daher nicht als „unhackbar“, „vollständig anonym“, „High Availability ohne weitere Architektur“ oder „immun gegen kompromittierte Endgeräte/Server“ beworben werden.

54. Glossar

- AAD** – Additional Authenticated Data; Metadaten, die kryptografisch an Ciphertext gebunden werden können.
- BFLA** – Broken Function Level Authorization.
- BOLA** – Broken Object Level Authorization.
- CSP** – Content Security Policy.
- E2EE** – Ende-zu-Ende-Verschlüsselung.
- Envelope** – strukturierter verschlüsselter Nachrichtenträger.
- Föderation** – Kommunikation zwischen unabhängigen GaiaCom-Nodes.
- GaiaID** – föderierte GaiaCom-Adresse.
- GaiaProof** – kryptografisches Nachweisformat.
- GaiaShield** – Security-/Audit-Subsystem.
- Gaia's Eyes** – kontrollierter Disclosure-/Reviewer-Workflow.
- GSN** – GaiaSocialNetwork.
- HQC-256** – post-quanten-orientierter KEM im Sovereign-Design.
- ML-DSA-87** – post-quanten-orientiertes Signaturverfahren.
- ML-KEM-1024** – post-quanten-orientierter Key Encapsulation Mechanism.
- OPFS** – Origin Private File System im Browser.
- PDU** – Protocol Data Unit im Federation-Transport.
- PQC** – Post-Quantum Cryptography.
- S2S** – Server-to-Server.
- TrustMesh** – lokales/policy-basiertes Reputationssystem.
- Trust Passport** – aggregierte Identitäts-/Trust-Ansicht.
- WASM** – WebAssembly.
- Zero Server Trust** – Designprinzip, bei dem vertrauliche Inhalte nicht vom Server entschlüsselt werden müssen.
-

55. Dokumentationspflege

Bei jeder relevanten Release-Änderung sollten mindestens folgende Dokumente gemeinsam geprüft werden:

```
README.md
SECURITY.md
SECURITY_INVARIANTS.md
DEPLOYMENT.md
docs/*
Frontend/frontend/package.json
Backend/go.mod
Backend/provision/*
deploy/nginx/*
deploy/systemd/*
```

Zusätzlich sollte die große Architekturkarte aktualisiert werden, wenn sich:

- Krypto-Suites,
- API-Endpunkte,
- Tabellen,
- Rollen,
- Module,
- Frontend-Panes,
- Federation-Flows,
- Deploymentanforderungen

ändern.

Für künftige Releases empfiehlt sich eine eindeutige Dokumentationshierarchie:

1. **Quellcode + Tests** – tatsächliches Verhalten.
2. **Release Contract / README** – freigegebene Funktionen und Grenzen.
3. **Deployment Guide** – kanonischer Produktionsbetrieb.
4. **Architecture** – technische Tiefenbeschreibung.
5. **Handbuch** – bedienungs- und betriebsorientierte Zusammenführung.

Anhang A – Vollständige Architekturkarte (unverändert)

Der folgende Inhalt ist die separat bereitgestellte `ARCHITECTURE(2).md` **wortgleich und ohne inhaltliche Korrekturen.**

Bei Konflikten mit dem tatsächlich bereitgestellten Quellcode gelten die Hinweise in Kapitel 4 dieses Handbuchs als Snapshot-spezifische Korrektur-/Drift-Notizen.

GaiaCom Technische System- und Architekturkarte

// STATUS: PLATIN VGT SUPREME

// REVISION: 2.0.0 Sovereign Beta v2

// ARCHITEKTUR-DOKUMENTATION & SYSTEMATISCHER DATENBAUM

Inhaltsverzeichnis (Architekturindex)

1. Systemübersicht

- Architekturphilosophie & Zero-Server-Trust
- Laufzeitumgebungen & Technologie-Stack
- Globaler Architekturbaum

2. Dateien einer Architektur zuordnen

- Frontend Client (React / Vite)
- Desktop Shell (Tauri 2 Native Client)
- Sovereign Crypto Core & WebAssembly
- Core Rust Engine (Zero-Dependency)
- Backend Application Layer (Go net/http & httpx)
- Persistenz & SQLite-Datenbank
- Android Embedded Node & Modular Platform
- Deployment, DevOps & Hardening
- Security PoCs, Audits & Quality Gates

3. Modul-Dokumentation

- Modul 1: Sovereign Cryptographic Engine (Dual-PQC & Multi-Cipher Cascade)
- Modul 2: Authentifizierung, Sessions & Key Vault (GaiaVault / Auth)
- Modul 3: Identität, GaiaID & Trust Passport
- Modul 4: Quanten-Chat & E2EE Direct Messaging
- Modul 5: Gruppen-Chats, Räume & Kanäle (Rooms & Channels)
- Modul 6: GaiaMail & Mailbox State Management
- Modul 7: SMTP Legacy Bridge & Downgrade Gateway
- Modul 8: Public Channels (Öffentliche Broadcast-Kanäle)
- Modul 9: GSN – GaiaSocialNetwork (Dezentrales Social Layer)
- Modul 10: GaiaDrive & Encrypted Chunk Storage
- Modul 11: GaiaDrop (Anonyme Text-Einreichungen)
- Modul 12: Federation & S2S PDU-Transport
- Modul 13: Node Registry & Core-Hash-Verifikation
- Modul 14: TrustMesh & Lokales Reputations-Scoring
- Modul 15: Governance, Gaia's Eyes & Reviewer Workflows
- Modul 16: GaiaShield Security Operations & Intrusion Defense
- Modul 17: Desktop Client Integration (Tauri 2 Bridge)
- Modul 18: Android Mobile Platform (Embedded Node)

4. Dashboard & Workspace Detailliert Dokumentieren

- Dashboard → Kommandozentrale (`/dashboard`)
- Dashboard → Quanten-Chat (`/chat`)
- Dashboard → GaiaMail Posteingang & Ordner (`/inbox` , etc.)

- Dashboard → SMTP Mailbox (`/smtp_inbox` , etc.)
- Dashboard → Gruppen-Chats & Räume (`/groups`)
- Dashboard → Public Channels (`/public_channels`)
- Dashboard → GSN Social Network (`/gsn`)
- Dashboard → Network Health (`/network_health`)
- Dashboard → Security Center (`/security_center`)
- Dashboard → Abuse Center (`/abuse_center`)
- Dashboard → Adressbuch / Kontakte (`/contacts`)
- Dashboard → Mein Profil & Sicherheit (`/profile`)
- Dashboard → GaiaDrive / Vault (`/vault`)
- Dashboard → GaiaDrop (`/gaiadrop`)
- Dashboard → Mail Composer Overlay (`isComposing`)

5. CSS & UI-Architektur

- Globale Styles & Resets
- Theme & Design Tokens (Dark / Light Mode)
- Layout, Navigation & Panels
- Modul- & Seitenspezifische Stylesheets
- Responsive Design & Mobile Dock
- Visuelle Effekte: Nebula & Quantum Interface
- Style-Dateien & Komponenten-Matrix

6. API-Architektur & Endpunkt-Referenz

7. Systemweite Datenflüsse

- Datenfluss 1: Identitätserstellung & Dual-PQC-Schlüsselgenerierung
- Datenfluss 2: E2EE Quanten-Chat Nachrichtensendung & -Empfang
- Datenfluss 3: GaiaDrive Lokale Ablage & Cloud-Synchronisation
- Datenfluss 4: S2S Föderation mit Signierten PDUs & SSRF-Schutz
- Datenfluss 5: Missbrauchsmeldung, Gaia's Eyes Grant & Reviewer-Queue
- Datenfluss 6: GaiaShield Sicherheitsereignis & Audit-Kette

8. Shared & Core Komponenten

9. Architektur-Relationen & Mermaid-Diagramme

- Diagramm 1: Gesamtsystem-Architektur
- Diagramm 2: Dashboard- & Navigation-Architektur
- Diagramm 3: Modul-Abhängigkeitsgraph
- Diagramm 4: Backend / API / Datenbank-Pipeline

10. Dateireferenzen

11. Architektur-Auffälligkeiten & Ungenutzte Dateien

1. Systemübersicht

1.1 Architekturphilosophie & Zero-Server-Trust

GaiaCom ist eine **post-quanten-sichere, föderierte Kommunikationsinfrastruktur**, die nach strengen Prinzipien des **Zero-Server-Trust** und **Client-Side-Computing** arbeitet:

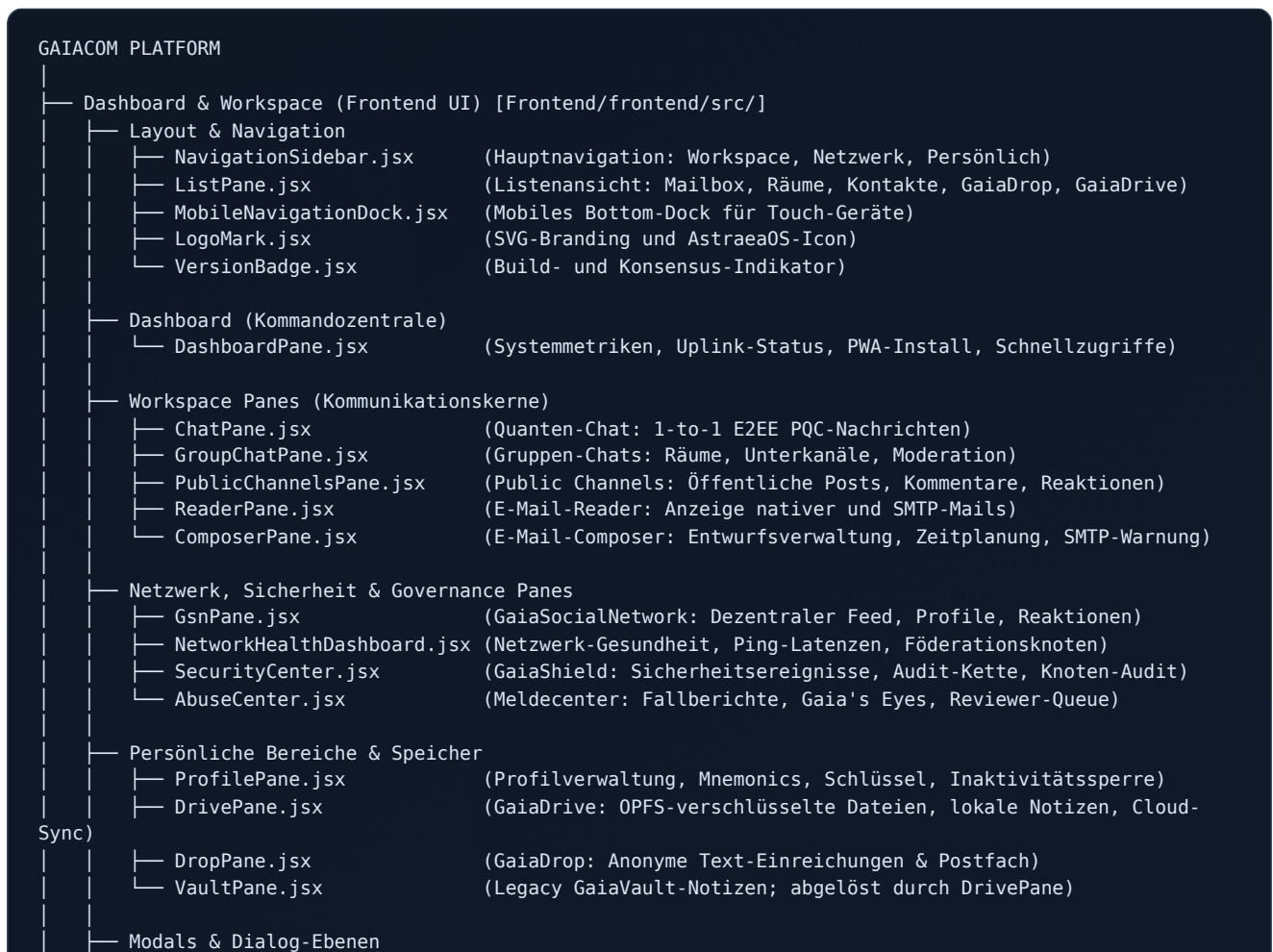
- 1. Zero Server Trust:** Der Server (Go-Backend) fungiert ausschließlich als Transporteur, Routing-Instanz, Föderationsknoten und Speicher für verschlüsselte Envelopes (`message_envelopes`), verschlüsselte Chunks (`file_chunks`) sowie minimale Identitäts- und Audit-Metadaten. Der Server besitzt zu keinem Zeitpunkt Zugriff auf:
 - Klartext-Nachrichten oder Anhänge
 - BIP-39 Mnemonics oder private Schlüssel (X25519, ML-KEM-1024, HQC-256, Ed25519, ML-DSA-87)
 - Symmetrische Nachrichtenschlüssel oder Session-Root-Secrets
 - Entschlüsselte Inhalte des lokalen Speichers (GaiaDrive / GaiaVault)
- 2. Dual-PQC Hybrid KEM:** Die Verschlüsselung basiert auf einem Drei-Geheimnis-Kombinator (`HKDF-SHA3-512`), der klassisches Ephemeral X25519, Post-Quantum ML-KEM-1024 und Post-Quantum HQC-256 vereint.
- 3. Multi-Cipher Authenticated Cascade:**
 - *Accelerated Mode:* Twofish-256-EAX + AES-256-GCM (Ed25519 Signaturen)
 - *Top Secret Mode:* Vierfache Kaskade: Serpent-256-CTR-HMAC-SHA3-512 + Twofish-256-EAX + XChaCha20-Poly1305 + AES-256-GCM-SIV (Ed25519 + ML-DSA-87 Signaturen)
- 4. Kein administrativer Generalschlüssel (No-Godmode):** Weder Node-Betreiber noch Auditoren können verschlüsselte Daten rückwirkend entschlüsseln. Moderation erfolgt nachweisbasiert über kryptografische Proofs (`GaiaProof`), Offenlegungs-Token (`Gaia's Eyes`) und lokale Föderationsreputation (`TrustMesh`).
- 5. Zero-Dependency Edge:** Gin und GORM wurden vollständig aus dem Backend entfernt. Das HTTP-Routing erfolgt über `Backend/httpx` (reines Go `net/http`), die Persistenz über pure-Go SQLite (`modernc.org/sqlite`).

1.2 Laufzeitumgebungen & Technologie-Stack

Schicht	Technologie	Aufgabe	Quellpfad
Web Frontend	React 18, Vite, Web Worker	Single-Page Application, UI, kryptografischer Browser-Worker, OPFS	<code>Frontend/frontend/</code>
Desktop Shell	Tauri 2.11, Rust 1.85	Native Desktop-App, isolated IPC, native HQC- und Kaskaden-Befehle	<code>DesktopClient/src-tauri/</code>
Sovereign Crypto Core	Rust 1.85, zeroize	Geteilte Multi-Cipher-Kaskade, native & WASM-Kompilierung	<code>SovereignCryptoCore/</code>
Sovereign Crypto WASM	Rust, wasmpack	WebAssembly-Brücke für den Browser-Crypto-Worker	<code>SovereignCryptoWasm/</code>

Schicht	Technologie	Aufgabe	Quellpfad
Core Primitives	Rust (Zero Dependency)	Deterministische Validierung (GaiaID, Envelope, TrustMesh)	Core/rust/gaiacore/
Backend Server	Go 1.26.5 (Linux AMD64)	Föderation, Routing, SQLite-Persistenz, ACLs, GaiaShield, SMTP	Backend/
In-Memory Mobile Node	Go (Backend/mobileapi)	Eingebetteter Knoten für Android/iOS via Gomobile-Bridge	Backend/mobileapi/
Android Client	Kotlin, Jetpack Compose	Modularer nativer mobiler Knoten mit Keystore/StrongBox	Android/
Datenbank	SQLite (pure-Go WAL-Mode)	Lokale und serverseitige relationale Persistenz mit Triggern	Backend/database/
Edge Proxy	NGINX	TLS-Terminierung, CSP-Header, Reverse Proxy (:8080)	deploy/nginx/
Service Supervisor	systemd	Sandboxed Linux-Service mit Minimalrechten	deploy/systemd/

1.3 Globaler Architekturbaum



├─ AppModalLayer.jsx	(Zentraler Modal-Manager)
├─ AppFeedbackModals.jsx	(Benutzer-Bestätigungen, Alerts, 3-Wege-Dialoge)
├─ AuthScreen.jsx	(Login, Registrierung, Kontowiederherstellung)
├─ UnlockScreen.jsx	(Sitzungsentsperrung via Passwort / PIN / WebAuthn)
├─ SetupWizard.jsx	(Ersteinrichtungs-Assistent für neue Identitäten)
├─ FirstRunOnboarding.jsx	(Einführungstour nach Erstinstallation)
├─ ContactProfileModal.jsx	(Kontakt-Detailkarte, Trust Passport, Schlüsselhistorie)
├─ AddContactModal.jsx	(Manueller Kontakt-Import via GaiaID)
├─ CreateGroupModal.jsx	(Raumerstellung mit Top-Secret-Option)
├─ JoinGroupModal.jsx	(Raumbeitritt via Hash oder Einladungslink)
├─ CreateChannelModal.jsx	(Unterkanal in bestehendem Raum erstellen)
├─ GroupSettingsModal.jsx	(Raumeinstellungen, Slow-Mode, Mitgliederverwaltung)
├─ KeyChangeWarningModal.jsx	(Sicherheitswarnung bei geändertem Empfängerschlüssel)
├─ QuantumShieldModal.jsx	(Erklärung & Statusanzeige des Quanten-Schutzes)
├─ HumanProofDialog.jsx	(Kryptografischer Proof-of-Human-Nachweis)
├─ Styling-Architektur [Frontend/frontend/src/styles/]	
├─ 27 kaskadierende CSS-Dateien (base, theme, layout, chat, nebula, ui, etc.)	
├─ startup.css	(Kritische Pre-Hydration-Styles zur Vermeidung von FOUC)
├─ Sovereign Cryptographic Engine (Dual-PQC & Multi-Cipher)	
├─ SovereignCryptoCore/	(Rust-Kaskade: Twofish, AES-GCM, Serpent, XChaCha20, AES-GCM-SIV)
├─ SovereignCryptoWasm/	(WASM-Export für Web-Browser)
├─ DesktopClient/src-tauri/	(Tauri 2 Native Host mit pqcrypto-hqc & IPC-Bridge)
├─ Core/rust/gaiacore/	(Zero-Dependency Rust-Primitiven: GaiaID, Envelopes, TrustMesh)
├─ Frontend/frontend/src/sovereign/	
├─ └─ nativeProvider.js	(Desktop Tauri IPC Provider)
├─ └─ webProvider.js	(Browser Worker & WASM Provider)
├─ └─ webCrypto.worker.js	(Isolierter Web Worker für HQC-256 & Kaskade)
├─ └─ vendor/liboqs-hqc256/	(Lokale, herstellerunabhängige liboqs HQC-256 Runtime)
├─ Frontend/frontend/src/crypto.js	(Klassische & PQC-Primitiven: noble-curves, scure-bip39, AES-GCM)
├─ Backend/crypto/	(Go CIRCL ML-KEM-1024, ML-DSA-87, Ed25519)
├─ Backend Application Services (Go 1.26.5) [Backend/]	
├─ └─ cmd/gaiacom/main.go	(CLI Einstiegspunkt: run, setup, doctor, version)
├─ └─ main.go	(Server Lifecycle, Graceful Shutdown, Signal Handling)
├─ └─ routes.go	(HTTP Router Setup, Middleware-Kaskade, Route-Deklaration)
├─ └─ route_runtime.go	(Validierung der Laufzeitkonfiguration)
├─ └─ scheduler.go	(DraftScheduler: Zeitgesteuerter Versand von E-Mails/Nachrichten)
├─ └─ httpx/	(Framework-freies net/http Routing, CORS & Security-Header)
├─ └─ auth/	(Benutzerkonten, Argon2/PBKDF2, JWT-Signierung & -Rotation)
├─ └─ devices/	(Geräte-Sessions, Schlüssel-Revokation, QR-Pairing-Protokoll)
├─ └─ identity/	(GaiaID-Verwaltung, öffentliche Schlüsselkataloge, HumanProof)
├─ └─ messaging/	(E2EE Nachrichtentransport, Proofs, Lesebestätigungen, Reaktionen)
├─ └─ mailbox/	(Mailbox-Zustände, Ordner, Labels, Kontakte, Filterregeln, Suche)
├─ └─ room/	(Raumverwaltung, Mitgliederrollen, Beitrittsanfragen, Pins, Audit-
├─ Logs)	
├─ └─ publicchannels/	(Öffentliche Broadcasts, Abonnements, Moderation, Kommentare)
├─ └─ gsn/	(Soziales Netzwerk: Posts, Kommentare, Reaktionen, Profile)
├─ └─ storage/	(GaiaDrive Chunked Object Store: Local FS & S3 Adapter, Quotas)
├─ └─ gaiadrop/	(Anonyme Text-Einreichungen mit Rate-Limiting & Hash-Prüfung)
├─ └─ smtpbridge/	(Legacy SMTP-Inbound & Outbound mit Downgrade-Kennzeichnung)
├─ └─ federation/	(S2S Signed PDUs, SSRF-Firewall, Replay-Schutz, Warteschlangen-
├─ Worker)	
├─ └─ noderegistry/	(Knotenregistrierung, Operator-Zulassung, Core-Hash-Integrität)
├─ └─ trustmesh/	(Lokale Reputationsbewertung, Epochen-Hashing, Missbrauchs-
├─ Friction)	
├─ └─ governance/	(Bootstrap-Rollen, Meldecenter-Fälle, Gaia's Eyes Offenlegung,
├─ Berufung)	
├─ └─ internal/security/	(GaiaShield: Kryptografische Audit-Kette, Redaktionsfilter, IDS-
├─ Regeln)	
├─ └─ networkhealth/	(Knoten-Metriken, Latenz-Pings, Dashboard-Daten)
├─ └─ operations/	(Healthchecks: /livez, /readyz, /metrics Prometheus-Monitoring)
├─ └─ provision/	(Idempotente Produktions-Initialisierung & Umgebungs-Diagnostik)
├─ └─ mobileapi/	(In-Process Gomobile Bindings für native Android/iOS-Knoten)
├─ Persistenz & Datenbank Layer [Backend/database/ & Backend/repository/]	
├─ └─ database/database.go	(SQLite Verbindungs-Manager, WAL-Pragmas, DDL-Migrationen)

- └─ database/migrations.go (Schema-Migrationsledger mit SHA-256 Prüfsummen & Backups)
- └─ database/migration_transport_queue.go (Offline Transport-Outbox und Deduplizierungs-Tabellen)
- └─ repository/repository.go (Zentrale Store-Schnittstellendefinitionen)
- └─ repository/sql_store.go (SQLStore Konstruktor und Transaktionsverwaltung)
- └─ repository/sql_store_*.go (25 domänenspezifische Persistenz-Adapter)
- └─ Android Mobile Platform [Android/]
 - └─ platform/contracts/ (Schnittstellenverträge zwischen Modulen)
 - └─ platform/kernel/ (App-Lifecycle, Command-Bus, Event-Dispatcher)
 - └─ platform/node/ (Bridge zum Go-Embedded-Node via Backend/mobileapi)
 - └─ platform/security/ (Kryptografische Schnittstellen & GaiaCore Rust-JNI)
 - └─ platform/security-android/ (Android Keystore & StrongBox Hardware-Bindung)
 - └─ platform/security-auth/ (Biometrische Authentifizierung & PIN-Sperre)
 - └─ platform/runtime/ (Lokale Runtime-Verwaltung)
 - └─ platform/remote/ (Transport-Adapter für HTTP, Wi-Fi Direct, BLE)
 - └─ platform/api/ (Typisierte Kotlin-APIs für Präsentationsschicht)
 - └─ platform/identity/ (Identitäts- und Schlüsselverwaltung auf dem Gerät)
 - └─ app/ (Android Jetpack Compose UI-Einstiegspunkt)
- └─ Deployment, DevOps & Runtime Hardening [deploy/]
 - └─ nginx/gaiacom.conf (Gehärtetes NGINX-VHost: TLS 1.3, CSP, Frame Deny, No Sniff)
 - └─ systemd/gaiacom.service (Gehärtete Systemd-Unit: Strict Sandbox, PrivateTmp, NoNewPrivileges)
- └─ Test- & Sicherheits-Gates [security/ & scripts/]
 - └─ scripts/repository_hygiene.py (Hygiene-Scanner: Prüft Binär-Hashes, Dateigrößen & Secrets)
 - └─ scripts/android_architecture_gate.py (Validiert Android-Modulabhängigkeiten)
 - └─ security/master_security_suite.py (Automatisierte End-to-End Adversarial Suite)
 - └─ security/poc/ (11 isolierte Proof-of-Concept Exploit- & Integritätstests)
 - └─ security/extreme/ (Extrem-Last-, BOLA/BFLA- und Replay-Stresstests)

2. Dateien einer Architektur zuordnen

Alle Pfadangaben verstehen sich relativ zum Repository-Root (`GaiaCom-main/`).

2.1 Frontend Client (`Frontend/frontend/`)

Einstiegspunkte & Konfiguration

- `Frontend/frontend/index.html` : HTML-Root-Dokument mit Content Security Policy (CSP), PWA-Manifest-Links und Startup-CSS.
- `Frontend/frontend/vite.config.js` : Vite-Bundler-Konfiguration, WASM-Support, Rollup-Bundle-Splitting.
- `Frontend/frontend/package.json` : Node-Paketdefinition (React 18, noble-curves, scure-bip39, vitest).
- `Frontend/frontend/src/index.jsx` : React 18 Root-Initialisierung, `RuntimeBoundary` (Fehlerabfang), PWA Service-Worker-Registrierung.
- `Frontend/frontend/src/index.css` : Haupt-CSS-Manifest; importiert alle 27 Einzelschnittstellen-Stylesheets.
- `Frontend/frontend/public/startup.css` : Kritisches Fallback-Styling vor der React-Hydration (verhindert FOUC).
- `Frontend/frontend/public/service-worker.js` : Offline-Caching und Asset-Verwaltung für Progressive Web Apps.

Globale Komponenten & Orchestrierung

- `Frontend/frontend/src/App.jsx` : Haupt-Container; verwaltet Theme, Identitäten, Menü-Status, Benachrichtigungen und Socket/Polling.
- `Frontend/frontend/src/components/app/AppMainContent.jsx` : Dynamischer View-Port; lädt via `React.lazy` und `Suspense` das jeweils aktive Pane.
- `Frontend/frontend/src/components/app/AppModalLayer.jsx` : Zentraler Layer für Popups, Modals und Dialoge.
- `Frontend/frontend/src/components/app/AppFeedbackModals.jsx` : Alert-, Bestätigungs- und 3-Wege-Auswahldialoge.
- `Frontend/frontend/src/api.js` : Zentraler HTTP-API-Client mit automatischer JWT-Rotation und Desktop-Erkennung.
- `Frontend/frontend/src/crypto.js` : Web-Krypto-Engine (noble/curves, scure/bip39, PBKDF2, AES-GCM, ECDSA).
- `Frontend/frontend/src/sovereignCrypto.js` : Abstraktionsschicht für Sovereign Dual-PQC (wechselt zwischen Native und Web Worker).

Navigation & Layout

- `Frontend/frontend/src/components/layout/NavigationSidebar.jsx` : 3-Bereiche-Seitenleiste (Workspace, Netzwerk, Persönlich), Identitätskarte, Quantum-Shield-Anzeige.

- `Frontend/frontend/src/components/layout/ListPane.jsx` : Mittlere Spalte für Nachrichtenlisten, Chat-Kontakte, Ordner und Filter.
- `Frontend/frontend/src/components/layout/MobileNavigationDock.jsx` : Touch-optimierte Navigation für Smartphones und Tablets.
- `Frontend/frontend/src/components/layout/LogoMark.jsx` : Vektorbasiertes GaiaCom-Logo.
- `Frontend/frontend/src/components/layout/VersionBadge.jsx` : Anzeige von Softwareversion und Föderationskonsens.

Dashboard & Feature-Panes

- `Frontend/frontend/src/components/chat/DashboardPane.jsx` : System-Kommandozentrale, Uplink-Status, Schnellübersicht.
- `Frontend/frontend/src/components/chat/ChatPane.jsx` : E2EE 1-to-1 Quanten-Chat, Timeline, Dateianhänge, Reaktionen.
- `Frontend/frontend/src/components/chat/GroupChatPane.jsx` : Gruppenraum-Ansicht, Unterkanäle, Teilnehmerliste, Rollenanzeige.
- `Frontend/frontend/src/components/chat/PublicChannelsPane.jsx` : Öffentliche Broadcast-Kanäle, Posts, Reaktionen, Kommentare.
- `Frontend/frontend/src/components/chat/ReaderPane.jsx` : E-Mail-Reader, Dekodierung nativer Envelopes, GaiaProof-Verifikation.
- `Frontend/frontend/src/components/chat/ComposerPane.jsx` : E-Mail-Editor mit Betreff, Empfängern, Entwurfsspeicherung und SMTP-Schalter.
- `Frontend/frontend/src/components/chat/ProfilePane.jsx` : Schlüsselanzeige, Mnemonics, Inaktivitätstimer, Kontolöschung, Privacy.
- `Frontend/frontend/src/components/chat/DrivePane.jsx` : GaiaDrive Speicher-Manager (OPFS-Dateien, Notizen, Cloud-Sync).
- `Frontend/frontend/src/components/chat/DropPane.jsx` : GaiaDrop Einreichungspostfach und Überprüfung anonymer Einsendungen.
- `Frontend/frontend/src/components/chat/GsnPane.jsx` : GaiaSocialNetwork Feed, Beitragserstellung, Folgen/Entfolgen.
- `Frontend/frontend/src/components/chat/SecurityCenter.jsx` : GaiaShield Sicherheitszentrale, Ereignisprotokolle, Knoten-Audit.
- `Frontend/frontend/src/components/chat/AbuseCenter.jsx` : Meldecenter, Vertrauensbewertung, Reviewer-Queue, Operator-Aktionen.
- `Frontend/frontend/src/components/public/NetworkHealthDashboard.jsx` : Öffentlicher Netzwerkstatus, Ping-Zeiten, Föderationsgesundheit.
- `Frontend/frontend/src/components/chat/VaultPane.jsx` : *Legacy*: Alter Notiz-Tresor (ersetzt durch `DrivePane.jsx`).

Authentifizierung & Onboarding

- `Frontend/frontend/src/components/auth/AuthScreen.jsx` : Authentifizierungs-Bildschirm (Login, Registrierung, Wiederherstellung).
- `Frontend/frontend/src/components/auth/UnlockScreen.jsx` : Lokale Sitzungsentsperrung (Passwort, PIN, WebAuthn PRF).

- `Frontend/frontend/src/components/auth/SetupWizard.jsx` : Assistent zur Erzeugung neuer Identitäten und Mnemonics.
- `Frontend/frontend/src/components/onboarding/FirstRunOnboarding.jsx` : Interaktive Einführung nach der Ersteinrichtung.

Modals & Dialoge

- `Frontend/frontend/src/components/modals/AddContactModal.jsx` : Kontakt hinzufügen per GaiaID.
- `Frontend/frontend/src/components/modals/ContactProfileModal.jsx` : Kontakt-Profil, Verifikationsnachweis, Schlüsselhistorie.
- `Frontend/frontend/src/components/modals/CreateGroupModal.jsx` : Gruppenraum anlegen (mit Top-Secret-Flag).
- `Frontend/frontend/src/components/modals/JoinGroupModal.jsx` : Gruppe beitreten per Einladungscode.
- `Frontend/frontend/src/components/modals/CreateChannelModal.jsx` : Kanal in einem Raum anlegen.
- `Frontend/frontend/src/components/chat/GroupSettingsModal.jsx` : Raumverwaltung, Slow-Mode, Rollenrechte, Audit-Logs.
- `Frontend/frontend/src/components/modals/KeyChangeWarningModal.jsx` : Sicherheitswarnung bei geändertem Empfängerschlüssel.
- `Frontend/frontend/src/components/modals/QuantumShieldModal.jsx` : Details zum Dual-PQC-Schutzstatus.
- `Frontend/frontend/src/components/common/HumanProofDialog.jsx` : Erstellung eines kryptografischen Human-Proofs.
- `Frontend/frontend/src/components/common/AppModal.jsx` : Generischer modaler Container.
- `Frontend/frontend/src/components/common/GaiaPassportCard.jsx` : Visuelle Pass-Karte mit Vertrauenswerten.
- `Frontend/frontend/src/components/common/AvatarPicker.jsx` : Avatar-Auswahl und Upload-Vorbereitung.

Frontend Custom Hooks (`Frontend/frontend/src/hooks/`)

- `useGaiaAuth.js` : Authentifizierungszustand, Tokenverwaltung, Identitätsabruf.
- `useChat.js` : Chat-Nachrichten, Polling, Senden, Entschlüsseln, Löschen, Reaktionen.
- `useDrive.jsx` : OPFS-Dateiverwaltung, lokale AES-GCM Notizen, Chunked Cloud-Uploads.
- `useEmails.js` : GaiaMail- & SMTP-Postfach-Logik, Filter, Entwürfe, Labels, Zustandsupdates.
- `usePublicChannels.js` : Öffentliche Kanäle abonnieren, Beiträge erstellen, Kommentare verwalten.
- `useGsn.js` : GSN Feed-Abruf, Beiträge veröffentlichen, Reaktionen, Profiländerungen.
- `useGaiaDrop.js` : Abruf und Bereinigung von GaiaDrop-Einreichungen.
- `usePresence.js` : Heartbeat-Senden und Online-Status-Abfrage von Kontakten.
- `useMessageMeta.js` : Pins, gespeicherte Nachrichten, Reaktionen im lokalen Speicher.
- `useUnreadMarkers.js` : Ungelesen-Markierungen und Positions-Tracking für Chats und Gruppen.
- `useChatNotifications.js` : Desktop- und Audio-Benachrichtigungen bei neuen Nachrichten.
- `useInactivityLock.js` : Automatisches Sperren der UI bei Inaktivität.
- `useKeyChangeDetection.js` : Erkennung von Manipulationsversuchen an Empfängerschlüsseln.
- `useVault.js` : *Legacy*: Alter Tresor-Hook (abgelöst durch `useDrive.jsx`).

Frontend Hilfsprogramme (`Frontend/frontend/src/utils/`)

- `avatar.js` : Generierung und Skalierung von Avataren.
- `channelsTranslations.js` : Mehrsprachige Systemtexte für Public Channels.
- `contacts.js` : Zusammenführung von Mail- und Chat-Kontakten.
- `deviceFanout.js` : Fanout-Berechnung für Multi-Device-Verschlüsselung.
- `gaiaAddress.js` : Parsing und Validierung von `@benutzer:knoten.tld` Adressen.
- `humanProof.js` : Proof-of-Humanity Berechnungen.
- `i18n.jsx` : Mehrsprachigkeits-Engine (DE, EN, ES, FR, IT, etc.).
- `keyHistory.js` : Append-Only Tracking von verifizierten Kontaktschlüsseln.
- `markdown.jsx` : Sicheres React-Rendering von Markdown ohne `dangerouslySetInnerHTML`.
- `notificationPreferences.js` : Normalisierung von Benachrichtigungseinstellungen.
- `payload.js` : Validierung und Serialisierung von Nachrichten-Nutzlasten.
- `recovery.js` : Export und Import von verschlüsselten Wiederherstellungsdateien.
- `safeJson.js` : Abgesicherte JSON-Parsing-Funktionen gegen Prototype-Pollution.
- `secureExport.js` : Verschlüsselter Export von Systemdaten.
- `terms.js` : Nutzungsbedingungen und Datenschutzerklärungen.
- `useContactActions.js` : Hilfsfunktionen zur Kontaktpflege.
- `useProfileActions.js` : Hilfsfunktionen zur Profil- und Schlüsselverwaltung.
- `uuid.js` : RFC-4122 v4 UUID-Generierung und Prüfung.
- `webauthnPrf.js` : WebAuthn PRF-Extension für hardwaregebundene Schlüsselableitung.

2.2 Desktop Shell (`DesktopClient/`)

- `DesktopClient/src-tauri/Cargo.toml` : Tauri-Projektkonfiguration und Abhängigkeiten (`pqcrypto-hqc` , `zeroize` , `gaiacom-sovereign-core`).
- `DesktopClient/src-tauri/tauri.conf.json` : Sicherheitsgrenzen: Deaktivierter lokaler Webserver, strikte CSP, Fenstereigenschaften.
- `DesktopClient/src-tauri/capabilities/default.json` : Feingranulare Berechtigungsdeklaration für Tauri 2.
- `DesktopClient/src-tauri/src/main.rs` : Desktop-Ausführungs-Einstiegspunkt.
- `DesktopClient/src-tauri/src/lib.rs` : Registrierung der nativen IPC-Befehle im Tauri-Builders.
- `DesktopClient/src-tauri/src/sovereign_crypto.rs` : Implementierung der 5 isolierten IPC-Crypto-Befehle:
 - `sovereign_hqc256_keypair` : Generiert HQC-256 Schlüsselpaar nativ in Rust.
 - `sovereign_hqc256_encapsulate` : Erzeugt KEM-Ciphertext und Shared Secret.
 - `sovereign_hqc256_decapsulate` : Entkapselt KEM-Ciphertext mit dem privaten Schlüssel.
 - `sovereign_seal` : Führt Twofish/AES-GCM oder Vierfach-Kaskade nativ aus.
 - `sovereign_open` : Entschlüsselt Kaskaden-Ciphertext nativ in Rust.

2.3 Sovereign Crypto Core & WebAssembly

- `SovereignCryptoCore/Cargo.toml` : Rust-Crate `gaiacom-sovereign-core` v2.0.0.
- `SovereignCryptoCore/src/lib.rs` : Mathematische Implementierung der Kaskaden-Chiffren:
 - `seal()` & `open()` : Hauptfunktionen für Accelerated und Top-Secret Profile.
 - Twofish-256-EAX, AES-256-GCM, Serpent-256-CTR, HMAC-SHA3-512, XChaCha20-Poly1305, AES-256-GCM-SIV.
 - Sichere Speicherbereinigung (`zeroize`) aller temporären Schlüsselpuffer.
- `SovereignCryptoWasm/Cargo.toml` : Crate für WASM-Kompilierung via `wasm-pack` .
- `SovereignCryptoWasm/src/lib.rs` : WebAssembly-Bindings (`#[wasm_bindgen]`) zur Bereitstellung der Rust-Kaskade für Browser.
- `Frontend/frontend/src/sovereign/wasm/gaiacom-sovereign-wasm_bg.wasm` : Kompilierte WASM-Binärdatei.
- `Frontend/frontend/src/sovereign/vendor/liboqs-hqc256/hqc-256.runtime.js` : Lokale liboqs HQC-256 JavaScript/WASM-Runtime.
- `Frontend/frontend/src/sovereign/webCrypto.worker.js` : Isolierter Web Worker; führt HQC-256 Operationen und Kaskaden im Hintergrund-Thread aus.

2.4 Core Rust Engine (`Core/rust/gaiacore/`)

- `Core/rust/gaiacore/Cargo.toml` : Zero-Dependency Crate `gaiacore` .
- `Core/rust/gaiacore/src/lib.rs` : Grundlegende Invarianten:
 - `validate_gaia_id()` : RFC-konforme Prüfung von `@user:domain.tld` .
 - `constant_time_eq()` : Timing-resistenter Byte-Vergleich.
 - `is_fixed_hex()` : Sichere Hexadezimal-Validierung.
- `Core/rust/gaiacore/src/envelope.rs` : Parsing und Integritätsprüfung von Nachrichten-Envelopes.
- `Core/rust/gaiacore/src/trust_mesh.rs` : Mathematische Berechnung von Reputationswerten und Verfallskurven.

2.5 Backend Application Layer (`Backend/`)

Hauptprogramm & Routing

- `Backend/cmd/gaiacom/main.go` : Binary Entrypoint; delegiert an `backend.Execute()` .
- `Backend/main.go` : Server-Lifecycle, Initialisierung von Config, DB und Routen, Graceful Shutdown.
- `Backend/routes.go` : Registriert alle Endpunkte, Middleware-Pipelines und Hintergrund-Worker.
- `Backend/route_runtime.go` : Lädt und validiert Umgebungsvariablen (`GAIACOM_JWT_SECRET` , etc.).
- `Backend/scheduler.go` : Hintergrunddienst für zeitverzögert geplante E-Mails und Nachrichten.
- `Backend/embedded_node.go` : Ermöglicht den In-Process-Betrieb als eingebettete Engine.
- `Backend/mobile_node_registry.go` : Verwaltet aktive Knoteninstanzen im mobilen Kontext.

- `Backend/mobile_transport_registry.go` : Registrierung verfügbarer mobiler Transportwege.

Domänen-Pakete

- `Backend/auth/` :
 - `auth_service.go` : Benutzer-Registrierung, Argon2-Passworthashing, JWT-Ausstellung.
 - `auth_handler.go` : HTTP-Handler für `/api/v1/auth/*`.
- `Backend/devices/` :
 - `device_service.go` : Verwaltung von Geräte-Schlüsseln, Revokation und QR-Code-Kopplung.
 - `device_handler.go` : HTTP-Handler für `/api/v1/devices/*`.
- `Backend/identity/` :
 - `identity_service.go` : CRUD für Identitäten, Veröffentlichung öffentlicher Schlüssel.
 - `identity_handler.go` : HTTP-Handler für `/api/v1/identity/*` und öffentliche Profile.
- `Backend/messaging/` :
 - `messaging_service.go` : Envelope-Validierung, E2EE Routing, Proof-Generierung.
 - `messaging_handler.go` : HTTP-Handler für `/api/v1/messaging/*`.
- `Backend/mailbox/` :
 - `service.go` : Mailbox-Ordner, Labels, Filterregeln, Adressbuch, globale Volltextsuche.
 - `handler.go` : HTTP-Handler für `/api/v1/mailbox/*` und `/api/v1/search/global`.
- `Backend/room/` :
 - `service.go` : Gruppenräume, Unterkanäle, Rollen, Einladungslinks, Beitrittsanfragen.
 - `handler.go` : HTTP-Handler für `/api/v1/rooms/*`.
- `Backend/publicchannels/` :
 - `service.go` : Öffentliche Broadcast-Kanäle, Posts, Kommentare, Reaktionen, Moderation.
 - `handler.go` : HTTP-Handler für `/api/v1/public-channels/*`.
- `Backend/gsn/` :
 - `service.go` : GaiaSocialNetwork Beiträge, Reaktionen, Kommentare, Profile, Follower.
 - `handler.go` : HTTP-Handler für `/api/v1/gsn/*`.
- `Backend/storage/` :
 - `storage_service.go` : Chunked Upload-Koordination, Quotas, Zugriffsberechtigungen (ACL).
 - `storage_handler.go` : HTTP-Handler für `/api/v1/storage/*`.
 - `object_store.go` : Dateisystem-basierter lokaler Objektspeicher mit Path-Jail-Schutz.
 - `s3_object_store.go` : S3/MinIO-kompatibler Objektspeicher-Adapter.
- `Backend/gaiadrop/` :
 - `service.go` : Anonyme Text-Einreichungen, Hash-Verifikation, Größenbegrenzung.
 - `handler.go` : HTTP-Handler für `/api/v1/gaiadrop/*` und `/api/v1/public/gaiadrop/*`.
- `Backend/smtplibbridge/` :

- `service.go` : Legacy SMTP-Gateway mit STARTTLS, Header-Sanitizing und Downgrade-Kennzeichnung.
- `handler.go` : HTTP-Handler für `/api/v1/smtp/send` und `/api/v1/public/smtp/ingest`.
- `Backend/federation/` :
 - `federation_service.go` : Server-zu-Server Kommunikation, Signierte PDUs, Replay-Schutz.
 - `federation_handler.go` : HTTP-Handler für `/well-known/gaiacom/*` und S2S-Forwarding.
 - `federation_types.go` : Datenstrukturen für Föderations-Transaktionen.
- `Backend/noderegistry/` :
 - `service.go` : Registrierung neuer Knoten, Operator-Zulassung, Quarantäne, Core-Hash-Abgleich.
 - `handler.go` : HTTP-Handler für `/api/v1/node/registry/*`.
- `Backend/trustmesh/` :
 - `service.go` : Proof-basierte Missbrauchsmeldungen, Reputationsabzug, Epochen-Hashing.
 - `handler.go` : HTTP-Handler für `/api/v1/reports/submit`.
- `Backend/governance/` :
 - `service.go` : Bootstrap-Operator-Rechte (`governance.json`), Gaia's Eyes Token, Reviewer-Queue.
 - `handler.go` : HTTP-Handler für `/api/v1/governance/*`, `/api/v1/reviewer/*`, `/api/v1/node/abuse/*`.
 - `bootstrap.go` : Lädt vertrauenswürdige Betreiber-Identitäten beim Systemstart.
- `Backend/internal/security/` (GaiaShield):
 - `shield.go` : Zentrales Sicherheits-Subsystem, EdgeShield-Middleware, Retention-Sweeper.
 - `audit.go` : Manipulationssichere kryptografische Audit-Kette (Hash-Chain).
 - `events.go` : Definition und Erfassung von Sicherheitsereignissen (Severity, Kategorien).
 - `privacy.go` : Automatische Maskierung und Anonymisierung sensibler Daten.
 - `security_routes.go` : HTTP-Handler für `/api/v1/security/*` und `/api/v1/node/security/*`.
- `Backend/networkhealth/` :
 - `service.go` : Uptime-Messung, Latenzprüfungen, Status der Nachbarknoten.
 - `handler.go` : HTTP-Handler für `/api/v1/public/network-health`.
- `Backend/operations/` :
 - `monitor.go` : Liveness- (`/livez`), Readiness- (`/readyz`) und Prometheus-Metriken (`/metrics`).
- `Backend/httpx/` :
 - `router.go` : Zero-Dependency HTTP-Router mit parametrisierten Pfaden.
 - `middleware.go` : Sicherheits-Header (HSTS, CSP, Framing-Schutz).
 - `cors.go` : Feingranulare CORS-Richtlinie.
 - `json.go` : Standardisierte JSON-Ausgabe und Fehler-Serialisierung.
- `Backend/crypto/` :
 - `crypto_service.go` : CIRCL ML-KEM-1024, Ed25519 und ML-DSA-87 Hilfsfunktionen.
- `Backend/models/` :
 - `models.go` : Sämtliche Go-Structs für Entitäten, API-Requests, DTOs und Datenbankzeilen.

- `Backend/buildinfo/` :
 - `buildinfo.go` : Unveränderliche Release-Metadaten (Version, Git-Commit, Build-Zeitpunkt).
- `Backend/provision/` :
 - `setup.go` : Idempotente Generierung der Produktionsumgebung (`gaiacom setup`).
 - `doctor.go` : Automatische Sicherheitsprüfung der Konfiguration (`gaiacom doctor`).

2.6 Persistenz & SQLite-Datenbank

- `Backend/database/database.go` :
 - SQLite-Initialisierung (`modernc.org/sqlite` , pure-Go, CGO-frei).
 - Erzwingt WAL-Modus (`PRAGMA journal_mode=WAL`), Fremdschlüssel (`PRAGMA foreign_keys=ON`), Busy-Timeout (10.000 ms) und Synchronous-Stufe FULL.
 - Enthält die DDL-Migrationstabelle und über 70 DDL-Statements.
- `Backend/database/migrations.go` :
 - Deterministischer Schema-Migrations-Manager.
 - Führt vor jeder anstehenden Migration ein atomares SQLite-Backup durch (`backupDatabaseBeforeMigration`).
- `Backend/database/migration_transport_queue.go` :
 - Schema für die Offline-Transport-Outbox (`transport_outbox`) und Deduplizierung (`transport_inbox_dedup`).
- `Backend/repository/repository.go` :
 - Interfaces: `Store` , `AuthStore` , `MessagingStore` , `StorageStore` , `GovernanceStore` , etc.
- `Backend/repository/sql_store.go` :
 - Hauptadapter `SQLStore` , kapselt `*sql.DB` und Transaktions-Helfer.
- `Backend/repository/sql_store_*.go` :
 - 25 domänenspezifische Implementierungsdateien:
 - `sql_store_auth.go` : Benutzer, Sitzungen, Passwörter.
 - `sql_store_devices.go` / `sql_store_device_keys.go` : Geräte und Pairings.
 - `sql_store_identity.go` : Identitäten, Human-Proofs.
 - `sql_store_messaging.go` : Envelopes, Inboxes, Proofs, Lesebestätigungen, Reaktionen.
 - `sql_store_mailbox.go` : Mailbox-Zustände, Entwürfe, Labels, Filter, Suche.
 - `sql_store_rooms_public_channels.go` : Räume, Kanäle, Mitgliedschaften, Moderation.
 - `sql_store_public_channel_interactions.go` : Public Channel Posts, Reaktionen, Kommentare.
 - `sql_store_gsn.go` : Posts, Kommentare, Reaktionen, Profile, Follower.
 - `sql_store_storage_gaiadrop.go` : Datei-Metadaten, Chunks, Zugriffsrechte, Drop-Submissions.
 - `sql_store_federation.go` : Server-Liste, Queue, S2S-Routing.

- `sql_store_federation_replay.go` : Replay-Prüfung für eingehende PDUs.
- `sql_store_governance_abuse.go` : Richtlinien, Rollenbeglaubigungen, Fälle, Offenlegungen.
- `sql_store_security_events.go` : GaiaShield Sicherheitsereignisse & Audit-Chain.
- `sql_store_rate_limit.go` : Rate-Limiting-Zähler im SQLite-Speicher.
- `sql_store_presence.go` : Heartbeat- und Online-Zustände.
- `sql_store_transport_queue.go` : Multi-Hop Offline-Warteschlange.

2.7 Android Embedded Node & Modular Platform (`Android/`)

- `Android/settings.gradle.kts` : Gradle-Multi-Modul-Struktur:
 - `:app` : Nativer Jetpack-Compose Client.
 - `:platform:contracts` : Schnittstellen für Entkopplung aller Feature-Module.
 - `:platform:kernel` : Anwendungs-Kernel, Command-/Event-Bus, Zustandsverwaltung.
 - `:platform:node` : In-Process JNI-Bridge zum Go-Backend (`Backend/mobileapi`).
 - `:platform:security` : Abstrakte Sicherheits- und Krypto-Schnittstellen.
 - `:platform:security-android` : Hardwaregestützte Schlüsselspeicherung in Android Keystore / StrongBox.
 - `:platform:security-auth` : Biometrische Authentifizierung (Fingerabdruck / Face Unlock).
 - `:platform:runtime` : Lifecycle- und Hintergrund-Dienststeuerung.
 - `:platform:remote` : Transportverbindungen (Internet, Wi-Fi Direct, Bluetooth Mesh).
 - `:platform:api` : Typisierte Kotlin-Fassade für Präsentationsschichten.
 - `:platform:identity` : Lokale Identitäts- und Adressbuchverwaltung.
- `Android/build.gradle.kts` : Zentrale Build-Konfiguration.

2.8 Deployment, DevOps & Hardening (`deploy/`)

- `deploy/nginx/gaiacom.conf` :
 - Gehärteter NGINX Reverse-Proxy.
 - Leitet `/api/v1/`, `/.well-known/gaiacom/`, `/livez` und `/readyz` an den lokalen Backend-Port `:8080` weiter.
 - Blockiert direkten Zugriff auf `/metrics` von extern.
 - Schützt statische Frontend-Dateien durch strikte Header: `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Content-Security-Policy`.
- `deploy/systemd/gaiacom.service` :
 - Gehärtete systemd-Service-Unit für Linux AMD64.
 - `ProtectSystem=strict`, `ProtectHome=true`, `PrivateTmp=true`, `PrivateDevices=true`.
 - `NoNewPrivileges=true`, `LockPersonality=true`, `RestrictSUIDSGID=true`.
 - Berechtigungslose Ausführung unter dem Systembenutzer `gaiacom:gaiacom` mit `UMask=0077`.

2.9 Security PoCs, Audits & Quality Gates (`security/` & `scripts/`)

- `scripts/repository_hygiene.py` : Verhindert das Einchecken privater Schlüssel, unerlaubter Binärdateien, Logs oder Passwörter (überprüft 653+ Dateien).
- `scripts/android_architecture_gate.py` : Überprüft strikte Einhaltung der Android-Modulgrenzen.
- `security/master_security_suite.py` : Führt die gesamte automatisierte Sicherheitsprüfung aus.
- `security/total_system_poc_runner.py` : Orchestriert systemweite PoC-Angriffsszenarien.
- `security/poc/` :
 - `poc/federation/` : Testet SSRF-Schutz, IP-Sperren, ungültige Signaturen und Replay-Angriffe.
 - `poc/gaiadrop/` : Testet Payload-Größenlimits, XSS-Filterung und Rate-Limits.
 - `poc/gaiaproof/` : Verifiziert Unverfälschbarkeit von GaiaProof-Zertifikaten.
 - `poc/gaiarooms-pro/` : Testet Raumzugriffsberechtigungen, Rolleneskalation und Top-Secret-Integrität.
 - `poc/gaiavault/` : Verifiziert die Unlesbarkeit clientseitig verschlüsselter Datensätze auf dem Server.
 - `poc/governance/` : Testet Rollen-Credentials, Signaturketten und Schwellenwert-Konsens.
 - `poc/key-history/` : Überprüft Erkennung manipulierter Empfängerschlüssel (Downgrade-Schutz).
 - `poc/secure-disclosure/` : Testet zeitlich befristete Offenlegung über Gaia's Eyes Token.
 - `poc/smtp-bridge/` : Testet Schutz vor CRLF-Injection und strikte Downgrade-Signalisierung.
 - `poc/total-system/` : Gesamtsystem-Härtungstest.
 - `poc/trust-passport/` : Validierung der Reputationsmetriken gegen Fälschungen.
- `security/extreme/` : Extrem-Stresstests für parallele Datenbank-Schreibzugriffe und BOLA/BFLA-Angriffe.

3. Modul-Dokumentation

Modul 1: Sovereign Cryptographic Engine (Dual-PQC & Multi-Cipher Cascade)

Zweck

Dieses Modul bildet das fundamentale kryptografische Fundament von GaiaCom. Es garantiert die Vertraulichkeit, Authentizität und Post-Quanten-Sicherheit aller Nachrichten, Dateien und Identitäten.

Fähigkeiten

- **Dual-PQC Hybrid KEM:** Kombiniert ephemeral X25519 (klassisch), ML-KEM-1024 (FIPS 203) und HQC-256 (NIST PQC Round 4) über HKDF-SHA3-512 zu einem gemeinsamen Root-Secret.
- **Sovereign Accelerated Cascade:** Authentifizierte Verschlüsselung mittels Twofish-256-EAX und AES-256-GCM; signiert mit Ed25519.
- **Sovereign Top Secret Cascade:** Vierfache authentifizierte Kaskade: Serpent-256-CTR-HMAC-SHA3-512 + Twofish-256-EAX + XChaCha20-Poly1305 + AES-256-GCM-SIV; dual signiert mit Ed25519 und ML-DSA-87 (FIPS 204).
- **Transcript Binding:** Kryptografische Bindung aller Metadaten (Version, Sender, Empfänger, Message-ID, Zeitstempel, Algorithmen-Suite) an den Ciphertext via AAD.
- **Fail-Closed Provider-Architektur:** Wechselt nahtlos zwischen Tauri-Native-Host und isoliertem Web Worker; blockiert sofort bei fehlenden Primitiven.

Zugehörige Dateien

Frontend:

- Frontend/frontend/src/sovereignCrypto.js
- Frontend/frontend/src/crypto.js
- Frontend/frontend/src/sovereign/webProvider.js
- Frontend/frontend/src/sovereign/nativeProvider.js
- Frontend/frontend/src/sovereign/webCrypto.worker.js
- Frontend/frontend/src/sovereign/wasm/gaiacom_sovereign_wasm.js
- Frontend/frontend/src/sovereign/vendor/liboqs-hqc256/hqc-256.runtime.js

Desktop Shell:

- DesktopClient/src-tauri/src/sovereign_crypto.rs
- DesktopClient/src-tauri/src/lib.rs

Sovereign Core (Rust):

- SovereignCryptoCore/src/lib.rs
- SovereignCryptoWasm/src/lib.rs

Core Primitives (Rust):

- Core/rust/gaiacore/src/lib.rs
- Core/rust/gaiacore/src/envelope.rs

Backend:

- Backend/crypto/crypto_service.go
- Backend/crypto/kyber.go
- Backend/crypto/types/crypto_types.go

Tests:

```
- Frontend/frontend/src/adversarial.test.js
- Frontend/frontend/src/hqc-smoke.js
- SovereignCryptoCore/tests / DesktopClient/src-tauri/tests
```

Dashboard-Integration

- **Verwendet in:** `ChatPane.jsx`, `GroupChatPane.jsx`, `ComposerPane.jsx`, `ReaderPane.jsx`, `DrivePane.jsx`.
- **Visuelle Indikatoren:** `QuantumShieldModal.jsx`, Badge `HQC-256 + ML-KEM-1024 VERIFIED` in `DashboardPane.jsx`, Sicherheitsstatus in `NavigationSidebar.jsx`.
- **Benutzeraktionen:** Umschalten zwischen *Accelerated* und *Top Secret* Modus im Chat; Anzeige des kryptografischen Prüfsertifikats.

Abhängigkeiten

```
Sovereign Cryptographic Engine
├─ benötigt WebAssembly & Web Worker Runtime im Browser
├─ benötigt pqcrypto-hqc & gaiacom-sovereign-core im Desktop-Client
├─ benötigt cloudflare/circl im Backend (ML-KEM / ML-DSA)
└─ exportiert Primitiven an Messaging, Storage, GSN und Drive
```

Wird verwendet von

- Modul 2 (Authentifizierung & Key Vault)
- Modul 4 (Quanten-Chat)
- Modul 5 (Gruppen-Chats)
- Modul 6 (GaiaMail)
- Modul 9 (GSN Social)
- Modul 10 (GaiaDrive)

Modul 2: Authentifizierung, Sessions & Key Vault (GaiaVault / Auth)

Zweck

Verwaltet Benutzerkonten, die sichere clientseitige Generierung und Verwahrung von Schlüsseln (Key Vault), Sitzungstoken (JWT-Access und Refresh-Token) sowie die Kopplung neuer Geräte via QR-Code.

Fähigkeiten

- Kontoregistrierung mit Argon2id-Passworthashing auf dem Server und PBKDF2/BIP-39 Schlüsselableitung im Client.
- Sichere Sitzungsverwaltung über kurzlebige JWT-Access-Tokens und langlebige Refresh-Tokens mit automatischer Rotation.
- Geräte-Kopplung (Device Pairing) über ephemere Einmalgeheimnisse ohne Übertragung privater Schlüssel.

- Inaktivitätssperre mit lokaler Wiederentsperrung via Passwort, PIN oder WebAuthn PRF (Hardware-Token).

Zugehörige Dateien

Frontend:

- Frontend/frontend/src/components/auth/AuthScreen.jsx
- Frontend/frontend/src/components/auth/UnlockScreen.jsx
- Frontend/frontend/src/components/auth/SetupWizard.jsx
- Frontend/frontend/src/hooks/useGaiaAuth.js
- Frontend/frontend/src/hooks/useInactivityLock.js
- Frontend/frontend/src/utils/webauthnPrf.js
- Frontend/frontend/src/utils/recovery.js

Backend:

- Backend/auth/auth_service.go
- Backend/auth/auth_handler.go
- Backend/devices/device_service.go
- Backend/devices/device_handler.go

API:

- POST /api/v1/auth/register, /login, /refresh, /logout, /change-password, /delete-account
- GET /api/v1/auth/status, /devices
- POST /api/v1/devices/pairings, /devices/pairings/:id/approve, /consume

Datenbank:

- users, identities, device_sessions, device_pairings, device_keys

Dashboard-Integration

- Nach dem Login Einstieg in `DashboardPane.jsx`.
- Entsperrung via `UnlockScreen.jsx` bei Inaktivität.
- Sitzungsverwaltung in `ProfilePane.jsx` (aktive Geräte einsehen und widerrufen).

Abhängigkeiten

Authentifizierung & Vault

- └─ verwendet Modul 1 (Sovereign Crypto) für Schlüsselableitung
- └─ liest/schreibt Tabellen: users, identities, device_sessions, device_keys
- └─ verwendet Shared Component: api.js, safeJson.js

Wird verwendet von

- Allen Modulen des Gesamtsystems (Zugangsvoraussetzung).

Modul 3: Identität, GaiaID & Trust Passport

Zweck

Ermöglicht die dezentrale Adressierung von Kommunikationspartnern im Format `@username.node.domain.tld`, stellt kryptografische Identitätsnachweise bereit und verwaltet den Trust Passport.

Fähigkeiten

- Parsing und Validierung föderierter GaiaIDs (`gaiaAddress.js` & `Core/rust/gaiacore/src/lib.rs`).
- Verwaltung öffentlicher Schlüsselkataloge (X25519 Box-Key, ML-KEM-1024 Key, HQC-256 Key, Ed25519 Sign-Key, ML-DSA-87 Sign-Key).
- HumanProof-Verifikation: Kryptografischer Nachweis menschlicher Interaktion.
- Trust Passport: Aggregierte Vertrauens- und Reputationsmetriken eines Nutzers.
- Key Change Detection: Warnung bei Änderung des Empfängerschlüssels zur Abwehr von Man-in-the-Middle-Angriffen.

Zugehörige Dateien

Frontend:

- `Frontend/frontend/src/components/common/GaiaPassportCard.jsx`
- `Frontend/frontend/src/components/common/HumanProofDialog.jsx`
- `Frontend/frontend/src/components/modals/ContactProfileModal.jsx`
- `Frontend/frontend/src/components/modals/KeyChangeWarningModal.jsx`
- `Frontend/frontend/src/hooks/useKeyChangeDetection.js`
- `Frontend/frontend/src/utils/gaiaAddress.js`
- `Frontend/frontend/src/utils/humanProof.js`
- `Frontend/frontend/src/utils/keyHistory.js`

Backend:

- `Backend/identity/identity_service.go`
- `Backend/identity/identity_handler.go`

API:

- `POST /api/v1/identity/create, /human-proof`
- `GET /api/v1/identity/me, /public/identity/:gaiaID, /public/trust-passport/:gaiaID`

Datenbank:

- `identities, user_notification_preferences`

Dashboard-Integration

- Anzeige der aktiven Identität in `NavigationSidebar.jsx` und `DashboardPane.jsx` .
- Adressbuch-Detailansicht in `ContactProfileModal.jsx` .

Abhängigkeiten

Identität & Trust Passport

- └─ benötigt `Backend/identity` & `Core/rust/gaiacore`
- └─ schreibt `identities` Tabelle
- └─ stellt Identitätsinformationen für alle Kommunikationsmodule bereit

Wird verwendet von

- Modul 4 (Quanten-Chat), Modul 5 (Gruppen), Modul 6 (GaiaMail), Modul 9 (GSN), Modul 15 (Governance).

Modul 4: Quanten-Chat & E2EE Direct Messaging

Zweck

Ermöglicht direkte Ende-zu-Ende-verschlüsselte 1-zu-1-Kommunikation mit Dual-PQC-Schlüsselaustausch, Lesebestätigungen, Echtzeit-Präsenz und Reaktionen.

Fähigkeiten

- Post-quanten-sichere Verschlüsselung jeder Chat-Nachricht auf dem sendenden Gerät.
- Server speichert ausschließlich unlesbare Envelopes (`message_envelopes`) und Zustellnachweise.
- Unterstützung für *Accelerated* und *Top Secret* Sicherheitsstufen.
- Echtzeit-Präsenzerkennung (Online-Status und Tipp-Indikatoren).
- Nachträgliches Bearbeiten (`/messaging/edit`), Löschen (`/messaging/delete`) und Reaktionen (`/messaging/reaction`).
- Generierung von `GaiaProof` -Zertifikaten zur Nachweiserbringung bei Missbrauch.

Zugehörige Dateien

Frontend:

- Frontend/frontend/src/components/chat/ChatPane.jsx
- Frontend/frontend/src/components/chat/MessageActions.jsx
- Frontend/frontend/src/hooks/useChat.js
- Frontend/frontend/src/hooks/usePresence.js
- Frontend/frontend/src/hooks/useMessageMeta.js
- Frontend/frontend/src/hooks/useUnreadMarkers.js

Backend:

- Backend/messaging/messaging_service.go
- Backend/messaging/messaging_handler.go
- Backend/presence/service.go
- Backend/presence/handler.go

API:

- POST `/api/v1/messaging/send`, `/read`, `/edit`, `/reaction`, `/delete`, `/clear`
- GET `/api/v1/messaging/inbox`, `/proof`
- POST `/api/v1/presence/heartbeat`, `/typing`; GET `/presence/status`, `/typing`

Datenbank:

- `message_envelopes`, `inboxes`, `message_proofs`, `delivery_receipts`, `message_read_receipts`, `message_reactions`, `identity_presence`

Dashboard-Integration

- Menüpunkt `chat` öffnet `ChatPane.jsx`.
- Anzeige ungelesener Direktnachrichten in `NavigationSidebar.jsx`.
- Schnellzugriff auf kürzliche Konversationen in `DashboardPane.jsx`.

Abhängigkeiten

Quanten-Chat

- verwendet Modul 1 (Sovereign Crypto) für KEM & Kaskaden
- verwendet Modul 3 (Identität) für Empfängerschlüssel

```
└─ liest/schreibt message_envelopes, inboxes, message_proofs
└─ bindet GaiaDrive für Dateifreigaben ein
```

Wird verwendet von

- Endnutzern zur vertraulichen Direktkommunikation.

Modul 5: Gruppen-Chats, Räume & Kanäle (Rooms & Channels)

Zweck

Stellt Multi-User-Gruppenräume mit rollenbasierter Zugriffskontrolle, Unterkanälen, Beitrittsanfragen, Einladungslinks und Moderationsprotokollen bereit.

Fähigkeiten

- Räume mit mehreren thematischen Textkanälen (`channels`).
- Rollenhierarchie: `owner`, `admin`, `moderator`, `member`.
- Private und öffentliche Räume mit Beitrittsanfragen (`room_join_requests`).
- Zeitgesteuerte und nutzungsbeschränkte Einladungslinks (`room_invite_links`).
- Angeheftete Nachrichten (`room_pinned_messages`) und Slow-Mode-Drosselung.
- Unveränderliche Revisions- und Moderationsprotokolle (`room_moderation_logs`).

Zugehörige Dateien

Frontend:

- Frontend/frontend/src/components/chat/GroupChatPane.jsx
- Frontend/frontend/src/components/chat/GroupSettingsModal.jsx
- Frontend/frontend/src/components/modals/CreateGroupModal.jsx
- Frontend/frontend/src/components/modals/JoinGroupModal.jsx
- Frontend/frontend/src/components/modals/CreateChannelModal.jsx

Backend:

- Backend/room/service.go
- Backend/room/handler.go

API:

- POST `/api/v1/rooms/create`, `/update`, `/join`, `/leave`, `/delete`, `/channels`, `/members/role`, `/members/kick`, `/transfer-ownership`
- POST `/api/v1/rooms/pins/toggle`, `/invites/create`, `/invites/join`, `/join-requests/create`, `/join-requests/moderate`
- GET `/api/v1/rooms`, `/channels`, `/search`, `/pins`, `/join-requests`, `/moderation-logs`

Datenbank:

- `rooms`, `room_members`, `channels`, `room_pinned_messages`, `room_invite_links`, `room_join_requests`, `room_moderation_logs`

Dashboard-Integration

- Menüpunkt `groups` lädt `GroupChatPane.jsx`.

- Raumübersicht und Schnellbeitritt in `DashboardPane.jsx`.

Abhängigkeiten

Gruppen-Chats

- verwendet Modul 4 (Messaging) für Nachrichten-Envelopes
- liest/schreibt Tabellen: `rooms`, `room_members`, `channels`
- interagiert mit Modul 16 (GaiaShield) bei Moderationsaktionen

Modul 6: GaiaMail & Mailbox State Management

Zweck

Implementiert ein vollwertiges, clientseitig verschlüsseltes E-Mail-System (GaiaMail) mit Ordnern, Labels, serverseitigem Scheduling und globaler Suche.

Fähigkeiten

- Ordnerverwaltung: Posteingang, Entwürfe, Gesendet, Markiert, Wichtig, Snoozed, Archiv, Spam, Papierkorb.
- Benutzerdefinierte farbige Labels (`mail_labels`).
- Zeitgesteuerter Versand von E-Mails über den `DraftScheduler`.
- Automatisierte Filterregeln (`mail_filter_rules`) basierend auf Absender und Betreff.
- Globale datenschutzkonforme Suche (`/api/v1/search/global`).

Zugehörige Dateien

Frontend:

- `Frontend/frontend/src/components/layout/ListPane.jsx` (Mailbox-Modus)
- `Frontend/frontend/src/components/chat/ReaderPane.jsx`
- `Frontend/frontend/src/components/chat/ComposerPane.jsx`
- `Frontend/frontend/src/hooks/useEmails.js`

Backend:

- `Backend/mailbox/service.go`
- `Backend/mailbox/handler.go`
- `Backend/scheduler.go`

API:

- GET `/api/v1/mailbox/messages`, `/drafts`, `/labels`, `/contacts`, `/filters`, `/settings`, `/search/global`
- POST `/api/v1/mailbox/state`, `/drafts/save`, `/drafts/delete`, `/labels/save`, `/contacts/save`, `/filters/save`, `/settings`

Datenbank:

- `mailbox_states`, `mail_drafts`, `mail_labels`, `mail_contacts`, `mail_filter_rules`, `mail_settings`

Dashboard-Integration

- Aufruf über Dropdown `GaiaMail` in `NavigationSidebar.jsx`.

- Neueste ungelesene Nachrichten werden im Widget von `DashboardPane.jsx` angezeigt.

Modul 7: SMTP Legacy Bridge & Downgrade Gateway

Zweck

Ermöglicht den Austausch mit klassischen E-Mail-Systemen über SMTP unter explizitem Hinweis auf den Verlust der nativen GaiaCom-Sicherheitsgarantien.

Fähigkeiten

- Ausgehender E-Mail-Versand via STARTTLS mit striktem CRLF- und Header-Injection-Schutz.
- Eingehender Webhook `/api/v1/public/smtp/ingest` mit Token-Authentifizierung.
- Explizite Kennzeichnung aller SMTP-Nachrichten als `untrusted` im Frontend.
- Permanenter Warnhinweis im Composer bei Auswahl des SMTP-Modus.

Zugehörige Dateien

Frontend:

- `Frontend/frontend/src/components/layout/NavigationSidebar.jsx` (SMTP-Dropdown)
- `Frontend/frontend/src/components/chat/ComposerPane.jsx` (SMTP-Modus & Warnung)

Backend:

- `Backend/smtplib/service.go`
- `Backend/smtplib/handler.go`

API:

- `POST /api/v1/smtp/send`
- `POST /api/v1/public/smtp/ingest`

Dashboard-Integration

- Eigener Navigationsbereich `SMTP Mail` in `NavigationSidebar.jsx`.
- Farbliche Differenzierung durch Warnabzeichen (`smtp-badge`).

Modul 8: Public Channels (Öffentliche Broadcast-Kanäle)

Zweck

Bietet offene, abonnierbare Informationskanäle für Ankündigungen, Diskussionsstränge, Medien und Community-Updates.

Fähigkeiten

- Erstellung öffentlicher Kanäle mit Name, Beschreibung, Avatar und Kategorien.
- Abonnieren (`subscribe`) und Deabonnieren (`unsubscribe`).

- Veröffentlichung von Beiträgen mit Formatierung, Medienanhängen und Zeitplanung (`scheduled_for`).
- Emoji-Reaktionen und mehrstufige Kommentarstränge mit Moderationsoptionen.
- Kanal-Blockierung und thematische Discovery-Suche (`/discover`).

Zugehörige Dateien

Frontend:

- Frontend/frontend/src/components/chat/PublicChannelsPane.jsx
- Frontend/frontend/src/hooks/usePublicChannels.js

Backend:

- Backend/publicchannels/service.go
- Backend/publicchannels/handler.go

API:

- GET `/api/v1/public-channels, /posts, /discover`
- POST `/api/v1/public-channels/create, /update, /comments, /delete, /subscribe, /unsubscribe, /block, /unblock`
- POST `/api/v1/public-channels/posts/create, /reaction, /comment, /pin, /comments/delete, /comments/moderate`

Datenbank:

- `public_channels, public_channel_admins, public_channel_subscribers, public_channel_posts, public_channel_post_reactions, public_channel_post_comments, public_channel_blocks`

Modul 9: GSN – GaiaSocialNetwork (Dezentrales Social Layer)

Zweck

Ein dezentrales, föderationsfähiges soziales Netzwerk für Microblogging, Status-Updates und Community-Interaktionen.

Fähigkeiten

- Erstellung kryptografisch signierter Beiträge (`gsn_posts`) mit optionalen Bildanhängen.
- Node-Feed (lokaler Server) und Following-Feed (abonnierte Kontakte).
- Signierte Kommentare (`gsn_post_comments`) und Emoji-Reaktionen.
- Benutzerprofile mit verifizierten Statusabzeichen (Betreiber, Governance, Passport).
- Clientseitige Entschlüsselung von Avataren und Beitragsbildern (`DecryptedAvatar.jsx` , `DecryptedGsnImage.jsx`).

Zugehörige Dateien

Frontend:

- Frontend/frontend/src/components/chat/GsnPane.jsx
- Frontend/frontend/src/components/chat/gsn/GsnPostCard.jsx
- Frontend/frontend/src/components/chat/gsn/GsnPostComposer.jsx
- Frontend/frontend/src/components/chat/gsn/GsnProfileEditor.jsx
- Frontend/frontend/src/components/chat/gsn/DecryptedAvatar.jsx
- Frontend/frontend/src/components/chat/gsn/DecryptedGsnImage.jsx
- Frontend/frontend/src/hooks/useGsn.js

Backend:

- Backend/gsn/service.go
- Backend/gsn/handler.go

API:

- POST /api/v1/gsn/posts, /posts/:id/react, /posts/:id/comment, /follow, /unfollow, /profile
- GET /api/v1/gsn/feed/node, /feed/following, /posts/:id/comments, /profile:gaia_id
- DELETE /api/v1/gsn/posts/:id, /posts/:id/comments/:commentId

Datenbank:

- gsn_posts, gsn_post_comments, gsn_post_reactions, gsn_follows, gsn_profiles

Modul 10: GaiaDrive & Encrypted Chunk Storage

Zweck

Kombiniert lokale verschlüsselte Dateispeicherung im Browser (Origin Private File System - OPFS) mit optionaler Ende-zu-Ende-verschlüsselter Cloud-Synchronisation über das Backend.

Fähigkeiten

- Lokale Speicherung von Notizen und Dateien in OPFS; verschlüsselt mit AES-GCM (Schlüssel im geschützten Index).
- Cloud-Upload in 1-MB-Blöcken (`file_chunks`) mit SHA-256 Hash-Validierung.
- Serverseitige Speicherquoten pro Benutzer (`GAIACOM_STORAGE_USER_QUOTA_BYTES`).
- Automatische Ablaufbereinigung temporärer Uploads (14-Tage-Cloud-TTL via Retention-Sweeper).
- Pluggable Storage Backend: Lokales Dateisystem mit Path-Jail oder S3-kompatibler Objektspeicher.

Zugehörige Dateien

Frontend:

- Frontend/frontend/src/components/chat/DrivePane.jsx
- Frontend/frontend/src/hooks/useDrive.jsx

Backend:

- Backend/storage/storage_service.go
- Backend/storage/storage_handler.go
- Backend/storage/object_store.go
- Backend/storage/s3_object_store.go

API:

- POST /api/v1/storage/init, /chunk, /complete, /grant
- GET /api/v1/storage/download/:fileId

Datenbank:

- file_metadata, file_access_grants, file_chunks

Modul 11: GaiaDrop (Anonyme Text-Einreichungen)

Zweck

Ermöglicht externen oder nicht authentifizierten Personen, vertrauliche Textnachrichten sicher an eine GaiaID zu übermitteln (Whistleblower- / Drop-Kanal).

Fähigkeiten

- Öffentlicher Einreichungs-Endpunkt `/api/v1/public/gaiadrop/submit`.
- Strikte Größenbeschränkung (max. 64 KB), XSS-Sanitization und IP-basiertes Rate-Limiting.
- Empfänger liest und verwaltet Einreichungen im geschützten `DropPane.jsx`.

Zugehörige Dateien

```
Frontend:
- Frontend/frontend/src/components/chat/DropPane.jsx
- Frontend/frontend/src/hooks/useGaiaDrop.js

Backend:
- Backend/gaiadrop/service.go
- Backend/gaiadrop/handler.go

API:
- POST /api/v1/public/gaiadrop/submit
- GET /api/v1/gaiadrop/inbox
- POST /api/v1/gaiadrop/read, /delete

Datenbank:
- gaia_drop_submissions
```

Modul 12: Federation & S2S PDU-Transport

Zweck

Verbindet unabhängige GaiaCom-Server zu einem dezentralen Netzwerk durch kryptografisch signierte Protokolldateneinheiten (Protocol Data Units - PDUs).

Fähigkeiten

- Server-zu-Server-Autorisierung über Header:
`Authorization: X-Gaia-S2S-V1 Signature="...",KeyId="node.example.org",Timestamp="..."`
- SSRF- und DNS-Rebinding-Firewall: Blockiert `localhost`, RFC 1918 Privatbereiche und Loopbacks.
- Replay-Schutz durch Transaktions-Cache (`federation_replay_guard`).
- Asynchrone Föderations-Warteschlange (`federation_queues`) mit exponentiellem Backoff und Lease-Locking.

Zugehörige Dateien

```
Frontend:
- Frontend/frontend/src/components/public/NetworkHealthDashboard.jsx

Backend:
- Backend/federation/federation_service.go
- Backend/federation/federation_handler.go
- Backend/federation/federation_types.go

API:
- GET /.well-known/gaiacom/server, /nodeinfo, /nodes
- POST /.well-known/gaiacom/s2s/v1/forward, /_gaiacom/s2s/v1/forward
- GET /api/v1/public/nodes

Datenbank:
- federation_servers, federation_queues, federation_replay_guard
```

Modul 13: Node Registry & Core-Hash-Verifikation

Zweck

Kontrolliertes Onboarding neuer Server-Knoten, Erkennung von Software-Forks und Gewährleistung der Versionskompatibilität im Netzwerk.

Fähigkeiten

- Öffentlicher Ping-Endpunkt zur Anmeldung neuer Server.
- Status-Workflow für Betreiber: `pending` → `accepted`, `quarantined` oder `blocked`.
- Automatischer Abgleich des `GAIACOM_CORE_HASH` zur Erkennung veralteter oder modifizierter Server-Kerne.

Zugehörige Dateien

```
Frontend:
- Frontend/frontend/src/components/chat/SecurityCenter.jsx (Reiter Nodesystem)

Backend:
- Backend/noderegistry/service.go
- Backend/noderegistry/handler.go

API:
- POST /api/v1/public/node-registry/ping
- GET /api/v1/public/node-registry/nodes
- GET /api/v1/node/registry/summary
- POST /api/v1/node/registry/secrets, /ping-main, /:domain/status

Datenbank:
- node_registry_entries
```

Modul 14: TrustMesh & Lokales Reputations-Scoring

Zweck

Lokale, nachweisbasierte Erkennung und Drosselung von Spam und böartigem Verhalten ohne Verletzung der Ende-zu-Ende-Verschlüsselung.

Fähigkeiten

- Erstellung kryptografischer Missbrauchsmeldungen (`reports`) gebunden an Chiffirat-Hashes und Epochen-Geheimnisse.
- Lokale Reputationsabzüge (`abuse_scores`) mit Verfallskurve (Score Decay).
- Progressive Reibung (Friction): Dynamische Drosselung der Durchsatzrate verdächtiger Absender bis hin zur Quarantäne.

Zugehörige Dateien

```
Frontend:  
- Frontend/frontend/src/components/chat/ReaderPane.jsx (Melden-Funktion)  
  
Backend:  
- Backend/trustmesh/service.go  
- Backend/trustmesh/handler.go  
- Core/rust/gaiacore/src/trust_mesh.rs  
  
API:  
- POST /api/v1/reports/submit  
  
Datenbank:  
- reports, abuse_scores
```

Modul 15: Governance, Gaia's Eyes & Reviewer Workflows

Zweck

Rollenbasierte Verwaltung von Vorfällen und Missbrauchsmeldungen über Reviewer-Gremien, begrenzte Fall-Offenlegung und Transparenzberichte.

Fähigkeiten

- Schwellenwert-Konsens für Reviewer (`Senior Reviewer` , `Trusted Reviewer` , `Node Operator`).
- Bootstrap-Verankerung über `governance.json` und kryptografische `role_credentials` .
- Gaia's Eyes: Einmalige, zeitlich und im Umfang streng begrenzte Offenlegungs-Token für Beweismaterial in Missbrauchsfällen.
- Berufungsverfahren (`abuse_appeals`) gegen Sanktionen.
- Veröffentlichung manipulationsfester Transparenz-Snapshots (`transparency_snapshots`).

Zugehörige Dateien

Frontend:

- Frontend/frontend/src/components/chat/AbuseCenter.jsx

Backend:

- Backend/governance/service.go
- Backend/governance/handler.go
- Backend/governance/bootstrap.go

API:

- GET /api/v1/governance/roles, /reports/mine, /reports/:caseID, /reviewer/cases, /node/abuse/queue, /public/transparency
- POST /api/v1/reports, /reports/:caseID/disclosures, /reports/:caseID/appeal, /gais-eyes/grants, /reviewer/cases/:caseID/review, /node/abuse/actions, /node/transparency/snapshot, /public/gais-eyes/consume

Datenbank:

- governance_policies, role_credentials, role_credential_revocations, abuse_cases, abuse_case_events, abuse_disclosure_packages, abuse_disclosure_requests, gais_eyes_grants, abuse_reviews, abuse_actions, abuse_appeals, federation_abuse_signals, transparency_snapshots

Modul 16: GaiaShield Security Operations & Intrusion Defense

Zweck

Internes Sicherheits- und Überwachungssystem zum Schutz der Knoten-Integrität, Abwehr von Angriffen und manipulationssicheren Auditierung.

Fähigkeiten

- EdgeShield: Vor-Router-Inspektion zur Abwehr von Path-Traversal, DoS und Header-Manipulationen.
- Kryptografische Audit-Kette (`security_audit_chain`): Hash-verkettetes, unveränderliches Audit-Log geschützt durch SQLite-Trigger.
- Datenschutzkonforme Schwärzung (`privacy.go`): Entfernt IP-Adressen und sensible Payload-Daten vor der Protokollierung.
- Intrusion-Detection-Regeln (`security_rules`, `security_rule_hits`) mit automatischer Quarantäne.

Zugehörige Dateien

Frontend:

- Frontend/frontend/src/components/chat/SecurityCenter.jsx

Backend:

- Backend/internal/security/shield.go
- Backend/internal/security/audit.go
- Backend/internal/security/events.go
- Backend/internal/security/privacy.go
- Backend/internal/security/security_routes.go
- Backend/internal/security/*_guard.go

API:

- GET /api/v1/public/security/health, /security/me/summary, /security/me/events, /security/me/report, /node/security/summary, /node/security/events

```
- POST /api/v1/security/me/events/:event_id/acknowledge
```

Datenbank:

```
- security_events, security_event_private_context, security_rules, security_rule_hits,  
security_user_acknowledgements, security_quarantines, security_audit_chain, security_rate_limits  
- Trigger: trg_security_events_immutable_update, trg_security_events_no_delete,  
trg_security_audit_chain_no_update, trg_security_audit_chain_no_delete
```

Modul 17: Desktop Client Integration (Tauri 2 Bridge)

Zweck

Stellt die native Desktop-Laufzeitumgebung für Windows, macOS und Linux bereit und kapselt kryptografische Operationen in einem isolierten Rust-Prozess.

Fähigkeiten

- Kein lokaler HTTP-Port: Kommunikation erfolgt ausschließlich über den nativen Tauri-IPC-Kanal.
- Ausführung von HQC-256 und Multi-Cipher-Kaskaden in kompilierter Maschinensprache ohne WebAssembly-Overhead.
- Strikte Content Security Policy (CSP) und Verbot von `unsafe-eval`.

Zugehörige Dateien

Frontend:

```
- Frontend/frontend/src/sovereign/nativeProvider.js
```

Desktop Shell:

```
- DesktopClient/src-tauri/Cargo.toml  
- DesktopClient/src-tauri/tauri.conf.json  
- DesktopClient/src-tauri/src/lib.rs  
- DesktopClient/src-tauri/src/sovereign_crypto.rs
```

Modul 18: Android Mobile Platform (Embedded Node)

Zweck

Ermöglicht den Betrieb als vollwertiger mobiler Kommunikationsknoten mit hardwaregestützter Sicherheit und Offline-Fähigkeit.

Fähigkeiten

- In-Process-Betrieb des Go-Backends ohne offenen Netzwerk-Port (`Backend/mobileapi/node.go`).
- Modulare Android-Architektur: Trennung von Präsentation (Compose), Kernel, Sicherheit und Transport.
- Hardware-Bindung an Android Keystore und StrongBox Keymaster.
- Offline-Multi-Hop-Transport über Internet, Wi-Fi Direct und Bluetooth Low Energy.

Zugehörige Dateien

Android:

- Android/settings.gradle.kts
- Android/platform/contracts/
- Android/platform/kernel/
- Android/platform/node/
- Android/platform/security-android/
- Android/platform/security-auth/
- Android/app/

Backend In-Memory Bridge:

- Backend/mobileapi/node.go
- Backend/mobileapi/bootstrap.go
- Backend/mobileapi/identity.go
- Backend/mobileapi/transport.go

4. Dashboard & Workspace Detailliert Dokumentieren

Alle Dashboard- und Workspace-Ansichten werden im Frontend zentral über den Zustandswechsel von `currentMenu` in `Frontend/frontend/src/App.jsx` und den Renderer `AppMainContent.jsx` geschaltet.

Dashboard → Kommandozentrale (`/dashboard`)

- **Interner Menü-Status:** `currentMenu === 'dashboard'`
- **Frontend-Datei:** `Frontend/frontend/src/components/chat/DashboardPane.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/dashboard.css`, `nebula.css`, `quantum-interface.css`
- **Verwendete Komponenten:** `LogoMark`, `VersionBadge`, Metrik-Karten, Schnellzugriffs-Grid, PWA-Install-Banner.
- **Zuständiges Modul:** Modul 1 (Sovereign Crypto), Modul 2 (Auth), Modul 4 (Messaging), Modul 5 (Rooms).
- **API-Aufrufe:** Keine direkten HTTP-Calls beim Rendern; nutzt den im Speicher synchronisierten State von `useEmails`, `useChat`, `useGaiaAuth` und `/api/v1/auth/status`.
- **Backend Handler:** `auth.GetStatus`, `operations.Liveness`.
- **Services:** `auth.AuthService`, `operations.Monitor`.
- **Datenbankzugriffe:** Tabellen `users`, `identities`, `device_sessions`.
- **Authentifizierung / Berechtigungen:** Authentifizierter Benutzer mit gültigem JWT-Token.
- **Datenfluss:**

```
Benutzer öffnet Dashboard
↓
App.jsx initialisiert DashboardPane.jsx
↓
Aggregiert State (Identität, ungelesene Mails, aktive Räume, Chat-Metriken)
↓
Rendert Uplink-Status, PQC-Sicherheitslevel & Schnellzugriffe
↓
Benutzer klickt Aktion (z.B. Chat öffnen) → Schaltet currentMenu um
```

Dashboard → Quanten-Chat (`/chat`)

- **Interner Menü-Status:** `currentMenu === 'chat'`
- **Frontend-Datei:** `Frontend/frontend/src/components/chat/ChatPane.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/chat.css`, `responsive.css`, `nebula.css`
- **Verwendete Komponenten:** `MessageActions.jsx`, Emoji-Picker, Datei-Upload-Indikator, TopSecret-Toggle.
- **Zuständiges Modul:** Modul 4 (Quanten-Chat) in Kooperation mit Modul 1 (Sovereign Crypto).
- **API-Aufrufe:**
 - Senden: `POST /api/v1/messaging/send`

- Abrufen: `GET /api/v1/messaging/inbox`
- Gelesen: `POST /api/v1/messaging/read`
- Reaktion: `POST /api/v1/messaging/reaction`
- Löschen: `POST /api/v1/messaging/delete`
- Präsenz: `POST /api/v1/presence/heartbeat`, `GET /api/v1/presence/status`
- **Backend Handler:** `messaging.MessagingHandler.*`, `presence.PresenceHandler.*`.
- **Services:** `messaging.MessagingService`, `presence.PresenceService`, `security.SecuritySystem`.
- **Datenbankzugriffe:** Tabellen `message_envelopes`, `inboxes`, `message_proofs`, `message_reactions`, `identity_presence`.
- **Authentifizierung:** JWT Bearer Token + Prüfung der Identitäts-Inhaberschaft (`senderIdentityId`).
- **Datenfluss:**

```

Benutzer sendet Nachricht im Chat
    ↓
ChatPane.jsx / useChat.js
    ↓
sovereignCrypto.js (Dual-PQC KEM + Twofish/AES-GCM oder Vierfach-Kaskade)
    ↓
POST /api/v1/messaging/send (Nur Ciphertext-Envelope)
    ↓
Backend: MessagingHandler.SendMessage()
    ↓
MessagingService.SaveAndRouteEnvelope()
    ↓
SQLStore: Insert in message_envelopes & inboxes
    ↓
JSON Response (Status: queued / delivered)
    ↓
Empfänger-Client pollt GET /api/v1/messaging/inbox
    ↓
Empfänger-Worker entschlüsselt Envelope clientseitig via HQC/ML-KEM
    ↓
Anzeige im ChatPane des Empfängers

```

Dashboard → GaiaMail Posteingang & Ordner (`/inbox` , etc.)

- **Interner Menü-Status:** `currentMenu === 'inbox' | 'sent' | 'drafts' | 'starred' | 'important' | 'archive' | 'trash'`
- **Frontend-Datei:** `Frontend/frontend/src/components/layout/ListPane.jsx` & `Frontend/frontend/src/components/chat/ReaderPane.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/listpane.css`, `reader.css`, `product-refresh.css`
- **Verwendete Komponenten:** E-Mail-Listenzeilen, Ordernavigation, Sterne-Schalter, Label-Verwaltung.
- **Zuständiges Modul:** Modul 6 (GaiaMail) & Modul 1 (Crypto).
- **API-Aufrufe:**
 - `GET /api/v1/mailbox/messages?identityId=...&folder=...`
 - `POST /api/v1/mailbox/state`
 - `GET /api/v1/messaging/proof?messageId=...`

- **Backend Handler:** `mailbox.Handler.ListMessages`, `mailbox.Handler.UpdateStates`, `messaging.MessagingHandler.GetMessageProof`.
- **Services:** `mailbox.Service`, `messaging.MessagingService`.
- **Datenbankzugriffe:** Tabellen `mailbox_states`, `message_envelopes`, `message_proofs`.
- **Authentifizierung:** JWT Bearer Token.
- **Datenfluss:**

```

Benutzer wählt Ordner (z.B. Inbox)
↓
ListPane.jsx triggert getMailboxMessages()
↓
GET /api/v1/mailbox/messages
↓
Backend: mailbox.Handler -> mailbox.Service
↓
SQLStore: Query mailbox_states JOIN message_envelopes
↓
Client empfängt Envelopes und entschlüsselt Betreff/Body im Web Worker
↓
Auswahl einer Mail lädt ReaderPane.jsx
↓
Prüfung der kryptografischen Signatur & Anzeige des GaiaProof-Siegels

```

Dashboard → SMTP Mailbox (/smtp_inbox , etc.)

- **Interner Menü-Status:** `currentMenu === 'smtp_inbox' | 'smtp_sent' | 'smtp_trash' | ...`
- **Frontend-Datei:** `Frontend/frontend/src/components/layout/ListPane.jsx` & `Frontend/frontend/src/components/chat/ReaderPane.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/listpane.css`, `reader.css`, `gaiadrop.css`
- **Verwendete Komponenten:** Gelbe Warnbanner für Legacy-Transport, Absenderüberprüfung.
- **Zuständiges Modul:** Modul 7 (SMTP Bridge).
- **API-Aufrufe:** `GET /api/v1/mailbox/messages?untrusted=true`.
- **Backend Handler:** `mailbox.Handler.ListMessages`.
- **Services:** `mailbox.Service`, `smtpbridge.Service`.
- **Datenbankzugriffe:** Tabellen `mailbox_states`, `message_envelopes`.
- **Authentifizierung:** JWT Bearer Token.
- **Datenfluss:**

```

Externe Mail trifft am SMTP-Gateway ein
↓
POST /api/v1/public/smtp/ingest (Header-Prüfung, SPF/DKIM)
↓
SMTPBridgeService: Speichert Envelope mit untrusted = 1
↓
Client ruft SMTP-Liste über ListPane.jsx ab
↓
ReaderPane.jsx blendet strikten Downgrade-Warnhinweis ein

```

Dashboard → Gruppen-Chats & Räume (/groups)

- **Interner Menü-Status:** `currentMenu === 'groups'`
- **Frontend-Datei:** `Frontend/frontend/src/components/chat/GroupChatPane.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/groupchat.css`, `chat.css`
- **Verwendete Komponenten:** Kanalliste, Teilnehmerleiste, Rollen-Manager, Slow-Mode-Timer, Anheftungs-Manager.
- **Zuständiges Modul:** Modul 5 (Gruppen-Chats) & Modul 4 (Messaging).
- **API-Aufrufe:**
 - `GET /api/v1/rooms`, `/rooms/channels`, `/rooms/pins`
 - `POST /api/v1/rooms/channels`, `/rooms/members/role`, `/rooms/pins/toggle`
 - `POST /api/v1/messaging/send` (Channel Envelope)
- **Backend Handler:** `room.Handler.*`, `messaging.MessagingHandler.SendMessage`.
- **Services:** `room.Service`, `messaging.MessagingService`.
- **Datenbankzugriffe:** Tabellen `rooms`, `room_members`, `channels`, `room_pinned_messages`, `message_envelopes`.
- **Authentifizierung:** JWT Bearer Token + Prüfung der Raum-Mitgliedschaft und Rollenberechtigung.
- **Datenfluss:**

```
Benutzer wählt Raum und Kanal aus
↓
GroupChatPane.jsx ruft Kanal-Nachrichten ab
↓
Clientseitige Entschlüsselung der Gruppen-Envelopes
↓
Nachrichtenversand via POST /api/v1/messaging/send (mit channel_id)
↓
Backend prüft Schreibberechtigung & Slow-Mode
↓
Speicherung in message_envelopes mit Fanout an Raum-Inboxes
```

Dashboard → Public Channels (/public_channels)

- **Interner Menü-Status:** `currentMenu === 'public_channels'`
- **Frontend-Datei:** `Frontend/frontend/src/components/chat/PublicChannelsPane.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/publicChannels.css`
- **Verwendete Komponenten:** Feed-Karten, Post-Editor, Kommentarstränge, Discovery-Dialog.
- **Zuständiges Modul:** Modul 8 (Public Channels).
- **API-Aufrufe:**
 - `GET /api/v1/public-channels`, `/posts`, `/discover`
 - `POST /api/v1/public-channels/posts/create`, `/posts/reaction`, `/posts/comment`
- **Backend Handler:** `publicchannels.Handler.*`.
- **Services:** `publicchannels.Service`.

- **Datenbankzugriffe:** Tabellen `public_channels`, `public_channel_posts`, `public_channel_post_reactions`, `public_channel_post_comments`.
- **Authentifizierung:** JWT Bearer Token.

Dashboard → GSN Social Network (/gsn)

- **Interner Menü-Status:** `currentMenu === 'gsn'`
- **Frontend-Datei:** `Frontend/frontend/src/components/chat/GsnPane.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/gsn.css`
- **Verwendete Komponenten:** `GsnPostCard`, `GsnPostComposer`, `GsnProfileEditor`, `DecryptedAvatar`, `DecryptedGsnImage`.
- **Zuständiges Modul:** Modul 9 (GSN).
- **API-Aufrufe:**
 - `GET /api/v1/gsn/feed/node`, `/feed/following`, `/profile/:gaia_id`
 - `POST /api/v1/gsn/posts`, `/posts/:id/react`, `/posts/:id/comment`, `/follow`
- **Backend Handler:** `gsn.Handler.*`.
- **Services:** `gsn.Service`, `federation.Service`.
- **Datenbankzugriffe:** Tabellen `gsn_posts`, `gsn_post_comments`, `gsn_post_reactions`, `gsn_follows`, `gsn_profiles`.
- **Authentifizierung:** JWT Bearer Token + Ed25519 Beitragssignatur.

Dashboard → Network Health (/network_health)

- **Interner Menü-Status:** `currentMenu === 'network_health'`
- **Frontend-Datei:** `Frontend/frontend/src/components/public/NetworkHealthDashboard.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/networkHealth.css`
- **Verwendete Komponenten:** Status-Kacheln, Latenz-Graphen, Föderations-Tabelle.
- **Zuständiges Modul:** Modul 12 (Federation) & Modul 16 (Operations).
- **API-Aufrufe:** `GET /api/v1/public/network-health`, `GET /api/v1/public/version`.
- **Backend Handler:** `networkhealth.Handler.Dashboard`.
- **Services:** `networkhealth.Service`.
- **Datenbankzugriffe:** Tabellen `federation_servers`, `node_registry_entries`.
- **Authentifizierung:** Öffentlich zugänglich (kein Token erforderlich).

Dashboard → Security Center (/security_center)

- **Interner Menü-Status:** `currentMenu === 'security_center'`
- **Frontend-Datei:** `Frontend/frontend/src/components/chat/SecurityCenter.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/security.css`

- **Verwendete Komponenten:** Audit-Log-Tabelle, Ereignis-Anerkennungs-Button, Knoten-System-Monitor, Secret-Generator.
- **Zuständiges Modul:** Modul 16 (GaiaShield) & Modul 13 (Node Registry).
- **API-Aufrufe:**
 - GET /api/v1/security/me/summary , /events , /report
 - POST /api/v1/security/me/events/:event_id/acknowledge
 - Betreiber: GET /api/v1/node/security/summary , /events , /node/registry/summary
- **Backend Handler:** security.SecurityHandler.* , noderegistry.Handler.*.
- **Services:** security.SecuritySystem , noderegistry.Service .
- **Datenbankzugriffe:** Tabellen security_events , security_audit_chain , node_registry_entries .
- **Authentifizierung:** JWT Bearer Token; Betreiber-Rechte erfordern governance.json -Verifikation.

Dashboard → Abuse Center (/abuse_center)

- **Interner Menü-Status:** currentMenu === 'abuse_center'
- **Frontend-Datei:** Frontend/frontend/src/components/chat/AbuseCenter.jsx
- **CSS / Styling:** Frontend/frontend/src/styles/abuseCenter.css
- **Verwendete Komponenten:** Vorfalls-Warteschlange, Fall-Details, Beweis-Entschlüsseler, Schwellenwert-Abstimmung.
- **Zuständiges Modul:** Modul 15 (Governance) & Modul 14 (TrustMesh).
- **API-Aufrufe:**
 - GET /api/v1/governance/roles , /reports/mine , /reviewer/cases , /node/abuse/queue
 - POST /api/v1/reports , /reviewer/cases/:caseID/review , /node/abuse/actions
- **Backend Handler:** governance.Handler.*.
- **Services:** governance.Service .
- **Datenbankzugriffe:** Tabellen abuse_cases , abuse_reviews , abuse_actions , role_credentials .
- **Authentifizierung:** JWT Bearer Token + Prüfung der Reviewer-/Operator-Rolle.

Dashboard → Adressbuch / Kontakte (/contacts)

- **Interner Menü-Status:** currentMenu === 'contacts'
- **Frontend-Datei:** Frontend/frontend/src/components/layout/ListPane.jsx & ContactProfileModal.jsx
- **CSS / Styling:** Frontend/frontend/src/styles/listpane.css , modals.css
- **Verwendete Komponenten:** Kontaktkarten, Trust-Badge, Schlüssel-Fingerprint, Notizen.
- **Zuständiges Modul:** Modul 3 (Identität) & Modul 6 (Mailbox).
- **API-Aufrufe:** GET /api/v1/mailbox/contacts , POST /api/v1/mailbox/contacts/save , GET /api/v1/public/identity/:gaiaID .
- **Backend Handler:** mailbox.Handler.* , identity.IdentityHandler.GetPublicIdentity .
- **Services:** mailbox.Service , identity.IdentityService .

- **Datenbankzugriffe:** Tabellen `mail_contacts`, `identities`.

Dashboard → Mein Profil & Sicherheit (`/profile`)

- **Interner Menü-Status:** `currentMenu === 'profile'`
- **Frontend-Datei:** `Frontend/frontend/src/components/chat/ProfilePane.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/profile.css`, `auth.css`
- **Verwendete Komponenten:** Passwortwechsel-Formular, Mnemonic-Aufdeckungs-Modal, PIN-/WebAuthn-Schalter, Session-Liste.
- **Zuständiges Modul:** Modul 2 (Auth) & Modul 3 (Identität).
- **API-Aufrufe:** `POST /api/v1/auth/change-password`, `/privacy`, `/delete-account`, `GET /api/v1/auth/devices`.
- **Backend Handler:** `auth.AuthHandler.*`.
- **Services:** `auth.AuthService`.
- **Datenbankzugriffe:** Tabellen `users`, `identities`, `device_sessions`.

Dashboard → GaiaDrive / Vault (`/vault`)

- **Interner Menü-Status:** `currentMenu === 'vault'`
- **Frontend-Datei:** `Frontend/frontend/src/components/chat/DrivePane.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/drive.css`
- **Verwendete Komponenten:** OPFS-Dateimanager, Notiz-Editor, Cloud-Sync-Statusanzeige, Upload-Fortschrittsbalken.
- **Zuständiges Modul:** Modul 10 (GaiaDrive).
- **API-Aufrufe:**
 - `POST /api/v1/storage/init`
 - `POST /api/v1/storage/chunk`
 - `POST /api/v1/storage/complete`
 - `GET /api/v1/storage/download/:fileId`
- **Backend Handler:** `storage.StorageHandler.*`.
- **Services:** `storage.Service`, `object_store.*`.
- **Datenbankzugriffe:** Tabellen `file_metadata`, `file_chunks`, `file_access_grants`.

Dashboard → GaiaDrop (`/gaiadrop`)

- **Interner Menü-Status:** `currentMenu === 'gaiadrop'`
- **Frontend-Datei:** `Frontend/frontend/src/components/chat/DropPane.jsx`
- **CSS / Styling:** `Frontend/frontend/src/styles/gaiadrop.css`
- **Verwendete Komponenten:** Einreichungsliste, Integritäts-Hash-Prüfer, Detailbetrachter.
- **Zuständiges Modul:** Modul 11 (GaiaDrop).

- **API-Aufrufe:** GET /api/v1/gaiadrop/inbox , POST /api/v1/gaiadrop/read , POST /api/v1/gaiadrop/delete .
 - **Backend Handler:** gaiadrop.Handler.* .
 - **Services:** gaiadrop.Service .
 - **Datenbankzugriffe:** Tabelle gaia_drop_submissions .
-

Dashboard → Mail Composer Overlay (isComposing)

- **Interner Zustand:** isComposing === true
 - **Frontend-Datei:** Frontend/frontend/src/components/chat/ComposerPane.jsx
 - **CSS / Styling:** Frontend/frontend/src/styles/reader.css , ui.css
 - **Verwendete Komponenten:** Rich-Text-Editor, Empfänger-Autovervollständigung, SMTP-Toggle, Dateianhangs-Vorschau.
 - **Zuständiges Modul:** Modul 6 (GaiaMail) & Modul 7 (SMTP Bridge).
 - **API-Aufrufe:** POST /api/v1/messaging/send , POST /api/v1/smtp/send , POST /api/v1/mailbox/drafts/save .
 - **Backend Handler:** messaging.MessagingHandler.SendMessage , smtpbridge.Handler.Send , mailbox.Handler.SaveDraft .
-

5. CSS & UI-Architektur

Die Styling-Architektur von GaiaCom basiert auf einem rein modularen CSS-Kaskadensystem ohne externe CSS-Frameworks wie Tailwind oder Bootstrap. Alle Styles werden in `Frontend/frontend/src/index.css` über `@import`-Regeln in einer genau abgestimmten Reihenfolge zusammengeführt.

5.1 Globale Styles & Resets

- `base.css`: CSS-Reset, globale Box-Sizing-Regeln, Typografie-Basis (Inter / Roboto), Scrollbalken-Styling (`gaia-scrollbar`).
- `startup.css`: Vorab geladene Styles im HTML-Header; rendert den Fehlerbildschirm (`astraea-startup--fault`) und Ladezustände ohne FOUC.

5.2 Theme & Design Tokens (Dark / Light Mode)

- `theme.css`: Definiert CSS-Custom-Properties auf `:root`, `.dark-mode` und `.light-mode`:
 - Akzentfarben: `--accent-cyan: #00f2fe;`, `--accent-blue: #4facfe;`, `--accent-purple: #654ff0;`
 - Statusfarben: `--success: #2ed573;`, `--warning: #ffa502;`, `--danger: #ff4757;`
 - Oberflächen: `--bg-primary: #0a0e17;`, `--bg-secondary: #111827;`, `--card-bg: rgba(17, 24, 39, 0.7);`
 - Unschärfen & Schatten: `--glass-blur: blur(16px);`, `--panel-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);`
- `product-refresh.css`: Veredelte Rahmenradien, konsistente Polsterungen und Sub-Pixel-Glow-Effekte.

5.3 Layout, Navigation & Panels

- `layout.css`: 3-Spalten-Grid-Architektur:
 - Spalte 1: `.navigation-sidebar` (260px)
 - Spalte 2: `.mail-list-pane` (320px, einklappbar)
 - Spalte 3: `.mail-content-pane` (flex-grow: 1)
- `ui.css`: Wiederverwendbare UI-Primitives: Buttons (`.primary-btn`, `.secondary-btn`), Badges, Inputs, Form-Groups.
- `modals.css`: Modaler Hintergrund (`.modal-backdrop`), zentrierte Container (`.modal-container`), Animationen.

5.4 Modul- & Seitenspezifische Stylesheets

- `dashboard.css`: Kachel-Raster (`.dashboard-grid-cards`), Uplink-Status, Schnellzugriffskarten.
- `chat.css`: Chat-Blasen (`.chat-bubble.sent`, `.received`), Audio-/Dateianhänge, Reaktions-Picker, Zitat-Balken.

- `groupchat.css` : Spezifische Overrides für Gruppen-Chats, Raum-Kopfzeile, Moderationsleiste.
- `reader.css` : E-Mail-Leseansicht, Kopfzeilen-Metadaten, Dateianhang-Karten, Composer-Formular.
- `drive.css` : GaiaDrive Baumstruktur, OPFS-Dateiliste, Verschlüsselungsindikatoren, Cloud-Status.
- `profile.css` : Profilkarten, Mnemonic-Aufdeckungsraaster, Schalter für Inaktivität und Privatsphäre.
- `publicChannels.css` : Social-Broadcast-Feed, Post-Karten, Kommentar-Threads, Discovery-Kacheln.
- `gsn.css` : GSN Social Feed, Microblogging-Composer, verifizierte Pass-Badges, Bildgitter.
- `auth.css` : Anmelde-, Registrierungs- und Entsperrkarten mit animierten Hintergrund-Gradienten.
- `onboarding.css` : Schritt-für-Schritt-Assistent für Erstbenutzer.
- `security.css` : GaiaShield Ereignis-Logs, Ampel-Statusanzeigen, Betreiber-Audit-Tabellen.
- `abuseCenter.css` : Meldecenter-Fälle, Beweisprüfer, Schwellenwert-Abstimmungsbalken.
- `networkHealth.css` : Föderationsstatus-Tabellen, Ping-Latenz-Anzeigen.
- `gaiadrop.css` : Einreichungsformulare und Postfach-Listen für anonyme Einreichungen.
- `vault.css` : *Legacy*: Älteres Styling für gesperrte Notiz-Karten.

5.5 Responsive Design & Mobile Dock

- `responsive.css` : Media-Queries für Bildschirme unter 992px und 600px:
 - Einklappen der Seitenspalten zu Off-Canvas-Drawern (`.mobile-menu-open`).
 - Einblenden des unteren mobilen Docks (`.mobile-navigation-dock`).
 - Touch-optimierte Mindesthöhen (min. 34px / 44px) für Buttons.
- `stability-refresh.css` : Defensive Layout-Regeln, Scroll-Anchoring, Verhindern horizontaler Überläufe auf Mobilgeräten.

5.6 Visuelle Effekte: Nebula & Quantum Interface

- `nebula.css` : Futuristische visuelle Effekte: Glassmorphismus, animierte Chrome-Rahmen (`.nebula-content-chrome`), Leucht-Glows.
- `quantum-interface.css` : Quantenkryptografie-Visualisierungen: Pulsierende Schutzschilde, Algorithmen-Pills (`HQC-256` , `ML-KEM-1024`).

5.7 Style-Dateien & Komponenten-Matrix

CSS-Datei	Beeinflusste Komponenten & Views
<code>base.css</code>	Globales DOM, <code>App.jsx</code> , alle Panes
<code>theme.css</code>	Globales Farbschema, Dark/Light Mode Schalter
<code>ui.css</code>	<code>AppFeedbackModals.jsx</code> , <code>AppModal.jsx</code> , alle Buttons und Formulare
<code>layout.css</code>	<code>NavigationSidebar.jsx</code> , <code>ListPane.jsx</code> , <code>AppMainContent.jsx</code>
<code>auth.css</code>	<code>AuthScreen.jsx</code> , <code>UnlockScreen.jsx</code> , <code>SetupWizard.jsx</code>
<code>landing.css</code>	Landing-Page, Hero-Banner, Begrüßungsansichten

CSS-Datei	Beeinflusste Komponenten & Views
<code>listpane.css</code>	<code>ListPane.jsx</code> , Ordnerlisten, Kontaktlisten, Raumlisten
<code>reader.css</code>	<code>ReaderPane.jsx</code> , <code>ComposerPane.jsx</code>
<code>chat.css</code>	<code>ChatPane.jsx</code> , <code>MessageActions.jsx</code>
<code>groupchat.css</code>	<code>GroupChatPane.jsx</code> , <code>GroupSettingsModal.jsx</code>
<code>drive.css</code>	<code>DrivePane.jsx</code> (GaiaDrive)
<code>vault.css</code>	<code>VaultPane.jsx</code> (Legacy)
<code>profile.css</code>	<code>ProfilePane.jsx</code> , <code>ContactProfileModal.jsx</code>
<code>networkHealth.css</code>	<code>NetworkHealthDashboard.jsx</code>
<code>publicChannels.css</code>	<code>PublicChannelsPane.jsx</code> , <code>CreateChannelModal.jsx</code>
<code>gaiaDrop.css</code>	<code>DropPane.jsx</code>
<code>modals.css</code>	<code>AppModalLayer.jsx</code> , <code>AddContactModal.jsx</code> , <code>CreateGroupModal.jsx</code>
<code>abuseCenter.css</code>	<code>AbuseCenter.jsx</code>
<code>dashboard.css</code>	<code>DashboardPane.jsx</code>
<code>security.css</code>	<code>SecurityCenter.jsx</code>
<code>gsn.css</code>	<code>GsnPane.jsx</code> , alle Komponenten in <code>components/chat/gsn/</code>
<code>onboarding.css</code>	<code>FirstRunOnboarding.jsx</code>
<code>responsive.css</code>	<code>MobileNavigationDock.jsx</code> , mobile Drawer in allen Panes
<code>nebula.css</code>	Hintergrundrahmen und Panels in <code>AppMainContent.jsx</code> , <code>DashboardPane.jsx</code>
<code>product-refresh.css</code>	Veredelte Karten und Tooltips im gesamten Workspace
<code>quantum-interface.css</code>	<code>QuantumShieldModal.jsx</code> , Sicherheitsabzeichen in Chat und Dashboard
<code>stability-refresh.css</code>	Globale Scroll-Container und Überlauf-Prävention

6. API-Architektur & Endpunkt-Referenz

Endpunkt	Methode	Frontend-Aufrufer	Backend-Handler	Service	Datenquelle
/livez	GET	Browser / Monitor	operations.Monitor.Liveness	operations.Monitor	In-Memory Status
/readyz	GET	Browser / Monitor	operations.Monitor.Readiness	operations.Monitor	DB Ping (database.db)
/metrics	GET	Monitoring-Agent	operations.Monitor.Metrics	operations.Monitor	In-Memory Zähler
/api/v1/public/version	GET	VersionBadge , NetworkHealth	Anonyme Route	buildinfo	buildinfo.Current()
/api/v1/public/identity/:gaiaID	GET	api.getPublicIdentity	identity.Handler.GetPublicIdentity	identity.IdentityService	identities
/api/v1/public/trust-passport/:gaiaID	GET	api.getTrustPassport	identity.Handler.GetTrustPassport	identity.IdentityService	identities , abuse_scores
/api/v1/public/nodes	GET	api.getNodes	federation.Handler.GetNodes	federation.Service	federation_servers
/api/v1/public/node-registry/ping	POST	Externe Knoten	noderegistry.Handler.PublicPing	noderegistry.Service	node_registry_entries
/api/v1/public/node-registry/nodes	GET	NetworkHealthDashboard	noderegistry.Handler.PublicNodes	noderegistry.Service	node_registry_entries
/api/v1/public/network-health	GET	api.getNetworkHealth	networkhealth.Handler.Dashboard	networkhealth.Service	In-Memory & federation_servers
/api/v1/public/gaiadrop/submit	POST	Extern / Whistleblower	gaiadrop.Handler.Submit	gaiadrop.Service	gaia_drop_submissions
/api/v1/public/smtp/ingest	POST	Externer Mailserver	smtpbridge.Handler.Ingest	smtpbridge.Service	message_envelopes , inboxes
/api/v1/public/transparency	GET	api.getPublicTransparency	governance.Handler.GetPublicTransparency	governance.Service	transparency_snapshots
/api/v1/public/gaias-eyes/consume	POST	api.consumeGaiasEyesKey	governance.Handler.ConsumeGaiasEyesGrant	governance.Service	gaias_eyes_grants
/api/v1/public/security/health	GET	api.getPublicSecurityHealth	security.SecurityHandler.GetPublicHealth	security.SecuritySystem	security_events
/well-known/gaiacom/server	GET	Föderationspartner	federation.Handler.HandleServerDiscovery	federation.Service	Konfiguration
/well-known/gaiacom/nodeinfo	GET	Föderationspartner	federation.Handler.HandleNodeInfo	federation.Service	Konfiguration & Buildinfo

Endpunkt	Methode	Frontend-Aufrufer	Backend-Handler	Service	Datenquelle
<code>/well-known/gaiacom/nodes</code>	GET	Föderationspartner	<code>noderegistry.Handler.PublicNodes</code>	<code>noderegistry.Service</code>	<code>node_registry_entries</code>
<code>/well-known/gaiacom/s2s/v1/forward</code>	POST	Föderationspartner	<code>federation.Handler.HandleS2SForward</code>	<code>federation.Service</code>	<code>message_envelopes</code> , <code>inboxes</code>
<code>/gaiacom/s2s/v1/forward</code>	POST	Föderationspartner	<code>federation.Handler.HandleS2SForward</code>	<code>federation.Service</code>	<code>message_envelopes</code> , <code>inboxes</code>
<code>/api/v1/auth/register</code>	POST	<code>api.register</code>	<code>auth.AuthHandler.Register</code>	<code>auth.AuthService</code>	<code>users</code> , <code>identities</code>
<code>/api/v1/auth/login</code>	POST	<code>api.login</code>	<code>auth.AuthHandler.Login</code>	<code>auth.AuthService</code>	<code>users</code> , <code>device_sessions</code>
<code>/api/v1/auth/refresh</code>	POST	<code>api.rotateSession</code>	<code>auth.AuthHandler.Refresh</code>	<code>auth.AuthService</code>	<code>device_sessions</code>
<code>/api/v1/auth/status</code>	GET	<code>api.getStatus</code>	<code>auth.AuthHandler.GetStatus</code>	<code>auth.AuthService</code>	<code>users</code> , <code>identities</code>
<code>/api/v1/auth/change-password</code>	POST	<code>api.changePassword</code>	<code>auth.AuthHandler.ChangePassword</code>	<code>auth.AuthService</code>	<code>users</code>
<code>/api/v1/auth/logout</code>	POST	<code>api.logout</code>	<code>auth.AuthHandler.Logout</code>	<code>auth.AuthService</code>	<code>device_sessions</code>
<code>/api/v1/auth/delete-account</code>	POST	<code>api.deleteAccount</code>	<code>auth.AuthHandler.DeleteAccount</code>	<code>auth.AuthService</code>	<code>users</code> (CASCADE)
<code>/api/v1/auth/privacy</code>	POST	<code>api.updatePrivacySettings</code>	<code>auth.AuthHandler.UpdatePrivacy</code>	<code>auth.AuthService</code>	<code>users</code>
<code>/api/v1/auth/notification-preferences</code>	GET	<code>api.getNotificationPreferences</code>	<code>auth.AuthHandler.GetNotificationPreferences</code>	<code>auth.AuthService</code>	<code>user_notification_preferences</code>
<code>/api/v1/auth/notification-preferences</code>	POST	<code>api.saveNotificationPreferences</code>	<code>auth.AuthHandler.SaveNotificationPreferences</code>	<code>auth.AuthService</code>	<code>user_notification_preferences</code>
<code>/api/v1/auth/devices</code>	GET	<code>api.getDeviceSessions</code>	<code>auth.AuthHandler.ListDevices</code>	<code>auth.AuthService</code>	<code>device_sessions</code>
<code>/api/v1/auth/devices/revoke</code>	POST	<code>api.revokeDeviceSession</code>	<code>auth.AuthHandler.RevokeDevice</code>	<code>auth.AuthService</code>	<code>device_sessions</code>
<code>/api/v1/devices/pairings</code>	POST	<code>api.startDevicePairing</code>	<code>devices.Handler.Start</code>	<code>devices.Service</code>	<code>device_pairings</code>
<code>/api/v1/devices/pairings/:id</code>	GET	<code>api.getDevicePairing</code>	<code>devices.Handler.Get</code>	<code>devices.Service</code>	<code>device_pairings</code>
<code>/api/v1/devices/pairings/:id/approve</code>	POST	<code>api.approveDevicePairing</code>	<code>devices.Handler.Approve</code>	<code>devices.Service</code>	<code>device_pairings</code> , <code>device_keys</code>
<code>/api/v1/devices/pairings/:id/consume</code>	POST	<code>api.consumeDevicePairing</code>	<code>devices.Handler.Consume</code>	<code>devices.Service</code>	<code>device_pairings</code>

Endpoint	Methode	Frontend-Aufrufer	Backend-Handler	Service	Datenquelle
/api/v1/devices/keys	GET	api.getDeviceKeys	devices.Handler.ListKeys	devices.Service	device_keys
/api/v1/devices/recipient-keys	GET	api.getRecipientDeviceKeys	devices.Handler.RecipientKeys	devices.Service	device_keys
/api/v1/devices/keys/:id/revoke	POST	api.revokeDeviceKey	devices.Handler.RevokeKey	devices.Service	device_keys
/api/v1/identity/create	POST	api.createIdentity	identity.Handler.CreateIdentity	identity.IdentityService	identities
/api/v1/identity/me	GET	api.getMyIdentities	identity.Handler.GetMyIdentities	identity.IdentityService	identities
/api/v1/identity/human-proof	POST	api.saveIdentityHumanProof	identity.Handler.SaveHumanProof	identity.IdentityService	identities
/api/v1/messaging/send	POST	api.sendMessage	messaging.Handler.SendMessage	messaging.MessagingService	message_envelopes, inboxes
/api/v1/smtp/send	POST	api.sendSmtpMail	smtpbridge.Handler.Send	smtpbridge.Service	Outgoing SMTP / message_envelopes
/api/v1/messaging/inbox	GET	api.getInbox	messaging.Handler.GetInbox	messaging.MessagingService	inboxes, message_envelopes
/api/v1/messaging/read	POST	api.markMessagesRead	messaging.Handler.MarkRead	messaging.MessagingService	inboxes, message_read_receipts
/api/v1/messaging/edit	POST	api.editDirectMessage	messaging.Handler.EditMessage	messaging.MessagingService	message_envelopes
/api/v1/messaging/reaction	POST	api.toggleMessageReaction	messaging.Handler.ToggleReaction	messaging.MessagingService	message_reactions
/api/v1/messaging/proof	GET	api.getMessageProof	messaging.Handler.GetMessageProof	messaging.MessagingService	message_proofs
/api/v1/messaging/delete	POST	api.deleteInboxMessage	messaging.Handler.DeleteInboxMessage	messaging.MessagingService	inboxes, message_envelopes
/api/v1/messaging/clear	POST	api.clearInboxConversation	messaging.Handler.ClearInboxConversation	messaging.MessagingService	inboxes
/api/v1/presence/heartbeat	POST	api.sendPresenceHeartbeat	presence.Handler.Heartbeat	presence.Service	identity_presence
/api/v1/presence/status	GET	api.getPresenceStatus	presence.Handler.Status	presence.Service	identity_presence
/api/v1/presence/typing	POST	api.updateTypingStatus	presence.Handler.UpdateTyping	presence.Service	In-Memory Typing Map
/api/v1/presence/typing	GET	api.getTypingStatus	presence.Handler.TypingStatus	presence.Service	In-Memory Typing Map
/api/v1/mailbox/messages	GET	api.getMailboxMessages	mailbox.Handler.ListMessages	mailbox.Service	mailbox_states, message_envelopes

Endpunkt	Methode	Frontend-Aufrufer	Backend-Handler	Service	Datenquelle
/api/v1/mailbox/state	POST	api.updateMailboxStates	mailbox.Handler.UpdateStates	mailbox.Service	mailbox_states
/api/v1/mailbox/drafts	GET	api.getMailDrafts	mailbox.Handler.ListDrafts	mailbox.Service	mail_drafts
/api/v1/mailbox/drafts/save	POST	api.saveMailDraft	mailbox.Handler.SaveDraft	mailbox.Service	mail_drafts
/api/v1/mailbox/drafts/delete	POST	api.deleteMailDraft	mailbox.Handler.DeleteDraft	mailbox.Service	mail_drafts
/api/v1/mailbox/labels	GET	api.getMailLabels	mailbox.Handler.ListLabels	mailbox.Service	mail_labels
/api/v1/mailbox/labels/save	POST	api.saveMailLabel	mailbox.Handler.SaveLabel	mailbox.Service	mail_labels
/api/v1/mailbox/contacts	GET	api.getMailContacts	mailbox.Handler.ListContacts	mailbox.Service	mail_contacts
/api/v1/mailbox/contacts/save	POST	api.saveMailContact	mailbox.Handler.SaveContact	mailbox.Service	mail_contacts
/api/v1/mailbox/filters	GET	api.getMailFilters	mailbox.Handler.ListFilters	mailbox.Service	mail_filter_rules
/api/v1/mailbox/filters/save	POST	api.saveMailFilter	mailbox.Handler.SaveFilter	mailbox.Service	mail_filter_rules
/api/v1/mailbox/settings	GET	api.getMailSettings	mailbox.Handler.GetSettings	mailbox.Service	mail_settings
/api/v1/mailbox/settings	POST	api.saveMailSettings	mailbox.Handler.SaveSettings	mailbox.Service	mail_settings
/api/v1/search/global	GET	api.globalSearch	mailbox.Handler.GlobalSearch	mailbox.Service	message_envelopes, mail_contacts
/api/v1/storage/init	POST	api.initUpload	storage.Handler.InitUpload	storage.Service	file_metadata
/api/v1/storage/chunk	POST	api.uploadChunk	storage.Handler.UploadChunk	storage.Service	file_chunks, ObjectStore
/api/v1/storage/complete	POST	api.completeUpload	storage.Handler.CompleteUpload	storage.Service	file_metadata
/api/v1/storage/grant	POST	api.grantFileAccess	storage.Handler.GrantAccess	storage.Service	file_access_grants
/api/v1/storage/download/:fileId	GET	api.downloadFileAttachment	storage.Handler.DownloadFile	storage.Service	file_metadata, ObjectStore
/api/v1/reports/submit	POST	api.submitReport	trustmesh.Handler.SubmitReport	trustmesh.Service	reports, abuse_scores
/api/v1/gaiadrop/inbox	GET	api.getGaiaDropInbox	gaiadrop.Handler.ListInbox	gaiadrop.Service	gaia_drop_submissions
/api/v1/gaiadrop/read	POST	api.markGaiaDropRead	gaiadrop.Handler.MarkRead	gaiadrop.Service	gaia_drop_submissions

Endpunkt	Methode	Frontend-Aufrufer	Backend-Handler	Service	Datenquelle
/api/v1/gaiadrop/delete	POST	api.deleteGaiaDrop	gaiadrop.Handler.Delete	gaiadrop.Service	gaia_drop_submissions
/api/v1/gsn/posts	POST	api.createGsnPost	gsn.Handler.CreatePost	gsn.Service	gsn_posts
/api/v1/gsn/posts/:id	DELETE	api.deleteGsnPost	gsn.Handler.DeletePost	gsn.Service	gsn_posts
/api/v1/gsn/feed/node	GET	api.getGsnFeedNode	gsn.Handler.GetFeedNode	gsn.Service	gsn_posts
/api/v1/gsn/feed/following	GET	api.getGsnFeedFollowing	gsn.Handler.GetFeedFollowing	gsn.Service	gsn_posts, gsn_follows
/api/v1/gsn/posts/:id/react	POST	api.reactToGsnPost	gsn.Handler.ReactToPost	gsn.Service	gsn_post_reactions
/api/v1/gsn/posts/:id/comment	POST	api.addGsnComment	gsn.Handler.AddComment	gsn.Service	gsn_post_comments
/api/v1/gsn/posts/:id/comments	GET	api.getGsnComments	gsn.Handler.GetComments	gsn.Service	gsn_post_comments
/api/v1/gsn/posts/:id/comments/:id	DELETE	api.deleteGsnComment	gsn.Handler.DeleteComment	gsn.Service	gsn_post_comments
/api/v1/gsn/follow	POST	api.followGsnUser	gsn.Handler.FollowUser	gsn.Service	gsn_follows
/api/v1/gsn/unfollow	POST	api.unfollowGsnUser	gsn.Handler.UnfollowUser	gsn.Service	gsn_follows
/api/v1/gsn/profile/:gaia_id	GET	api.getGsnProfile	gsn.Handler.GetProfile	gsn.Service	gsn_profiles
/api/v1/gsn/profile	POST	api.updateGsnProfile	gsn.Handler.UpdateProfile	gsn.Service	gsn_profiles
/api/v1/rooms/create	POST	api.createRoom	room.Handler.CreateRoom	room.Service	rooms, room_members, channels
/api/v1/rooms/update	POST	api.updateRoom	room.Handler.UpdateRoom	room.Service	rooms
/api/v1/rooms	GET	api.getRooms	room.Handler.GetRooms	room.Service	rooms, room_members
/api/v1/rooms/join	POST	api.joinRoom	room.Handler.JoinRoomByHash	room.Service	room_members
/api/v1/rooms/leave	POST	api.leaveRoom	room.Handler.LeaveRoom	room.Service	room_members
/api/v1/rooms/channels	POST	api.createChannel	room.Handler.CreateChannel	room.Service	channels
/api/v1/rooms/channels	GET	api.getChannels	room.Handler.GetChannels	room.Service	channels
/api/v1/rooms/members/role	POST	api.updateMemberRole	room.Handler.UpdateMemberRole	room.Service	room_members

Endpunkt	Methode	Frontend-Aufrufer	Backend-Handler	Service	Datenquelle
/api/v1/rooms/delete	POST	api.deleteRoom	room.Handler.DeleteRoom	room.Service	rooms (CASCADE)
/api/v1/rooms/search	GET	api.searchPublicRooms	room.Handler.SearchPublicRooms	room.Service	rooms
/api/v1/rooms/members/kick	POST	api.kickRoomMember	room.Handler.KickMember	room.Service	room_members
/api/v1/rooms/transfer-ownership	POST	api.transferRoomOwnership	room.Handler.TransferOwnership	room.Service	rooms, room_members
/api/v1/rooms/pins	GET	api.getRoomPinnedMessages	room.Handler.GetRoomPinnedMessages	room.Service	room_pinned_messages
/api/v1/rooms/pins/toggle	POST	api.toggleRoomMessagePin	room.Handler.ToggleRoomMessagePin	room.Service	room_pinned_messages
/api/v1/rooms/invites/create	POST	api.createRoomInviteLink	room.Handler.CreateRoomInviteLink	room.Service	room_invite_links
/api/v1/rooms/invites/join	POST	api.joinRoomViaInviteLink	room.Handler.JoinRoomViaInviteLink	room.Service	room_invite_links, room_members
/api/v1/rooms/join-requests	GET	api.getRoomJoinRequests	room.Handler.GetRoomJoinRequests	room.Service	room_join_requests
/api/v1/rooms/join-requests/create	POST	api.createRoomJoinRequest	room.Handler.CreateRoomJoinRequest	room.Service	room_join_requests
/api/v1/rooms/join-requests/moderate	POST	api.moderateRoomJoinRequest	room.Handler.ModerateRoomJoinRequest	room.Service	room_join_requests, room_members
/api/v1/rooms/moderation-logs	GET	api.getRoomModerationLogs	room.Handler.GetRoomModerationLogs	room.Service	room_moderation_logs
/api/v1/public-channels	GET	api.getPublicChannels	publicchannels.Handler.List	publicchannels.Service	public_channels
/api/v1/public-channels/create	POST	api.createPublicChannel	publicchannels.Handler.Create	publicchannels.Service	public_channels, admins
/api/v1/public-channels/update	POST	api.updatePublicChannel	publicchannels.Handler.Update	publicchannels.Service	public_channels
/api/v1/public-channels/comments	POST	api.updatePublicChannelComments	publicchannels.Handler.UpdateComments	publicchannels.Service	public_channels
/api/v1/public-channels/delete	POST	api.deletePublicChannel	publicchannels.Handler.Delete	publicchannels.Service	public_channels
/api/v1/public-channels/subscribe	POST	api.subscribePublicChannel	publicchannels.Handler.Subscribe	publicchannels.Service	public_channel_subscribers
/api/v1/public-channels/unsubscribe	POST	api.unsubscribePublicChannel	publicchannels.Handler.Unsubscribe	publicchannels.Service	public_channel_subscribers
/api/v1/public-channels/posts	GET	api.getPublicChannelPosts	publicchannels.Handler.ListPosts	publicchannels.Service	public_channel_posts

Endpunkt	Methode	Frontend-Aufrufer	Backend-Handler	Service	Datenquelle
/api/v1/public-channels/posts/create	POST	api.createPublicChannelPost	publicchannels.Handler.CreatePost	publicchannels.Service	public_channel_posts
/api/v1/public-channels/posts/reaction	POST	api.togglePublicChannelPostReaction	publicchannels.Handler.TogglePostReaction	publicchannels.Service	public_channel_post_reactions
/api/v1/public-channels/posts/comment	POST	api.createPublicChannelPostComment	publicchannels.Handler.CreatePostComment	publicchannels.Service	public_channel_post_comments
/api/v1/public-channels/posts/pin	POST	api.updatePublicChannelPostPin	publicchannels.Handler.UpdatePostPin	publicchannels.Service	public_channel_posts
/api/v1/public-channels/block	POST	api.blockPublicChannel	publicchannels.Handler.Block	publicchannels.Service	public_channel_blocks
/api/v1/public-channels/unblock	POST	api.unblockPublicChannel	publicchannels.Handler.Unblock	publicchannels.Service	public_channel_blocks
/api/v1/public-channels/discover	GET	api.discoverPublicChannels	publicchannels.Handler.Discover	publicchannels.Service	public_channels
/api/v1/public-channels/posts/comments/delete	POST	api.deleteChannelComment	publicchannels.Handler.DeleteComment	publicchannels.Service	public_channel_post_comments
/api/v1/public-channels/posts/comments/moderate	POST	api.moderateChannelComment	publicchannels.Handler.ModerateComment	publicchannels.Service	public_channel_post_comments
/api/v1/governance/roles	GET	api.getGovernanceRoles	governance.Handler.CheckRoles	governance.Service	role_credentials
/api/v1/reports	POST	api.submitAbuseReport	governance.Handler.CreateReport	governance.Service	abuse_cases, abuse_case_events
/api/v1/reports/mine	GET	api.getMyReports	governance.Handler.GetMyReports	governance.Service	abuse_cases
/api/v1/reports/:caseID	GET	api.getReportDetail	governance.Handler.GetReportDetail	governance.Service	abuse_cases, events, packages
/api/v1/reports/:caseID/disclosures	POST	api.createCaseDisclosure	governance.Handler.CreateCaseDisclosure	governance.Service	abuse_disclosure_packages
/api/v1/reviewer/cases/:caseID/disclosure-requests	POST	api.requestCaseDisclosure	governance.Handler.RequestCaseDisclosure	governance.Service	abuse_disclosure_requests
/api/v1/reports/:caseID/appeal	POST	api.submitAppeal	governance.Handler.AppealReport	governance.Service	abuse_appeals
/api/v1/gaias-eyes/grants	POST	api.createGaiasEyesGrant	governance.Handler.CreateGaiasEyesGrant	governance.Service	gaias_eyes_grants
/api/v1/reviewer/cases	GET	api.getReviewerQueue	governance.Handler.GetReviewerQueue	governance.Service	abuse_cases

Endpunkt	Methode	Frontend-Aufrufer	Backend-Handler	Service	Datenquelle
/api/v1/reviewer/cases/:caseID/review	POST	api.submitReview	governance.Handler.SubmitReview	governance.Service	abuse_reviews
/api/v1/node/abuse/queue	GET	api.getNodeOperatorQueue	governance.Handler.GetNodeOperatorQueue	governance.Service	abuse_cases
/api/v1/node/abuse/actions	POST	api.applyNodeOperatorAction	governance.Handler.ApplyNodeOperatorAction	governance.Service	abuse_actions
/api/v1/node/transparency/snapshot	POST	api.createTransparencySnapshot	governance.Handler.CreateTransparencySnapshot	governance.Service	transparency_snapshots
/api/v1/security/me/summary	GET	api.getSecuritySummary	security.SecurityHandler.GetMySummary	security.SecuritySystem	security_events
/api/v1/security/me/events	GET	api.getSecurityEvents	security.SecurityHandler.GetMyEvents	security.SecuritySystem	security_events
/api/v1/security/me/events/:id/ack	POST	api.acknowledgeSecurityEvent	security.SecurityHandler.AcknowledgeEvent	security.SecuritySystem	security_user_acknowledgements
/api/v1/security/me/report	GET	api.exportSecurityReport	security.SecurityHandler.ExportReport	security.SecuritySystem	security_events, audit_chain
/api/v1/node/security/summary	GET	api.getNodeSecuritySummary	security.SecurityHandler.GetNodeSummary	security.SecuritySystem	security_events, Rules
/api/v1/node/security/events	GET	api.getNodeSecurityEvents	security.SecurityHandler.GetNodeEvents	security.SecuritySystem	security_events
/api/v1/node/registry/summary	GET	api.getNodeRegistrySummary	noderegistry.Handler.GetSummary	noderegistry.Service	node_registry_entries
/api/v1/node/registry/secrets	POST	api.generateNodeRegistrySecrets	noderegistry.Handler.GenerateSecrets	noderegistry.Service	In-Memory Secret Generator
/api/v1/node/registry/ping-main	POST	api.pingNodeRegistryMain	noderegistry.Handler.PingMain	noderegistry.Service	Föderations-Ping an Main-Node
/api/v1/node/registry/:domain/status	POST	api.updateNodeRegistryStatus	noderegistry.Handler.UpdateStatus	noderegistry.Service	node_registry_entries

7. Systemweite Datenflüsse

Datenfluss 1: Identitätserstellung & Dual-PQC-Schlüsselgenerierung

```
Benutzer öffnet SetupWizard.jsx / AuthScreen.jsx
↓
Eingabe von Master-Passwort & gewünschter GaiaID (@alice:node.domain)
↓
Frontend: crypto.js generiert 12 Wörter BIP-39 Mnemonic
↓
BIP-39 Mnemonic
├─> PBKDF2 -> X25519 Identitäts-Schlüsselpaar
├─> PBKDF2 -> Ed25519 Signaturschlüsselpaar
├─> Sovereign Worker -> ML-KEM-1024 KEM-Schlüsselpaar
└─> Sovereign Worker / Tauri Host -> HQC-256 KEM-Schlüsselpaar
↓
Private Schlüssel werden clientseitig mit Master-Passwort (PBKDF2/AES-GCM) verschlüsselt
und im lokalen Browser-Speicher / Desktop-Vault gesichert
↓
POST /api/v1/identity/create
  Payload: { gaiaId, displayName, publicRecord: { x25519, ed25519, ml_kem_1024, hqc_256 } }
↓
Backend: identity.IdentityHandler -> identity.IdentityService
↓
Integritätsprüfung & Validierung der GaiaID nach RFC-Spezifikation
↓
Repository: Speicherung in identities (is_active = 1)
↓
200 OK (Identität registriert, öffentliche Schlüssel publiziert)
```

Datenfluss 2: E2EE Quanten-Chat Nachrichtensendung & -Empfang

```
Alice tippt Nachricht an Bob (@bob:remote.domain)
↓
ChatPane.jsx ruft useChat.js -> sendMessage()
↓
Client ruft Bobs öffentliche Schlüssel ab: GET /api/v1/public/identity/@bob:remote.domain
↓
Prüfung gegen lokalen Key-History-Speicher (Key Change Detection)
↓
Sovereign Crypto Engine (Dual-PQC KEM):
  1. X25519 Diffie-Hellman Shared Secret (ss_classic)
  2. ML-KEM-1024 Encapsulation -> kem_ciphertext + ss_mlkem
  3. HQC-256 Encapsulation -> hqc_ciphertext + ss_hqc
↓
Kombinator: root_secret = HKDF-SHA3-512(ss_classic || ss_mlkem || ss_hqc, salt, transcript_aad)
↓
Kaskaden-Chiffrierung:
  Twofish-256-EAX gefolgt von AES-256-GCM (Accelerated)
  oder 4-Stufen-Kaskade inkl. Serpent & XChaCha20 (Top Secret)
↓
Signierung des Envelopes mit Alices privatem Ed25519 / ML-DSA-87 Schlüssel
↓
POST /api/v1/messaging/send (Server sieht NUR Ciphertext, KEM-Ciphertexte, Noncen, Hashes)
↓
Alices Heimatserver: Speichert in message_envelopes & inboxes
↓
Prüfung: Ist Bob lokal oder auf Remoteknoten?
```

```

└─> Lokal: Bereitstellung im Inbox-Index von Bob
└─> Remote: Föderations-Queue erzeugt signierte PDU -> S2S POST an Bobs Server
↓
Bobs Client ruft GET /api/v1/messaging/inbox ab
↓
Bobs Sovereign Worker:
  1. Decapsulate ML-KEM-1024 Ciphertext mit Bobs privatem ML-KEM-Schlüssel
  2. Decapsulate HQC-256 Ciphertext mit Bobs privatem HQC-Schlüssel
  3. X25519 DH mit ephemeren Key
  4. Root Secret Ableitung & Kaskaden-Entschlüsselung
↓
Anzeige der Klartext-Nachricht in Bobs ChatPane.jsx
↓
Automatischer Sendeauftrag: POST /api/v1/messaging/read (Lesebestätigung)

```

Datenfluss 3: GaiaDrive Lokale Ablage & Cloud-Synchronisation

```

Benutzer legt Datei in DrivePane.jsx ab
↓
useDrive.jsx fängt Datei ab
↓
Clientseitige Verschlüsselung:
- Generiert zufälligen 256-Bit AES-GCM Key & IV
- Verschlüsselt Datei-Payload im Browser-Speicher
- Speichert Roh-Chiffre im Origin Private File System (OPFS)
- Aktualisiert den verschlüsselten Metadaten-Index in localStorage
↓
Optionale Cloud-Synchronisation (≤ 20 MB, 14 Tage TTL):
↓
POST /api/v1/storage/init (fileName, fileSize, mimeType, fileHash)
↓
Backend generiert file_id und prüft Benutzer-Speicherquote
↓
Client teilt verschlüsselte Chunks (je 1 MB) und sendet:
POST /api/v1/storage/chunk (Multipart FormData mit chunkHash & chunkIndex)
↓
StorageService validiert SHA-256 Hash des Chunks
└─> Lokal: Schreibt Chunks in gesichertes Storage-Jail (uploads/<fileId>/chunk_<index>)
└─> S3: Schreibt Chunks direkt in S3/MinIO Bucket
↓
POST /api/v1/storage/complete
↓
StorageService assembliert Metadaten, setzt Status = 'completed'

```

Datenfluss 4: S2S Föderation mit Signierten PDUs & SSRF-Schutz

```

Knoten A möchte Nachricht an Knoten B übermitteln
↓
FederationService erzeugt PDU:
{ pdu_id, origin: "node-a.org", destination: "node-b.org", type: "message", payload: envelope }
↓
Signierung der PDU mit Ed25519-Server-Privatschlüssel von Knoten A
↓
HTTP POST an Knoten B: /.well-known/gaiacom/s2s/v1/forward
Header: Authorization: X-Gaia-S2S-V1 Signature="...",KeyId="node-a.org",Timestamp="..."
↓
Knoten B EdgeShield & FederationHandler:

```

```

1. Zeitfenster-Prüfung (|timestamp - now| < 300s)
2. SSRF-Firewall: Löst IP von node-a.org auf; blockiert private/Loopback-IPs
3. Replay-Guard: Prüft federation_replay_guard; lehnt doppelte pdu_id sofort ab
4. Signaturprüfung: Verifiziert Signatur gegen hinterlegten Ed25519 Public Key von node-a.org
↓
200 OK PDU Accepted
↓
Knoten B hinterlegt Nachricht im lokalen Posteingang des Empfängers

```

Datenfluss 5: Missbrauchsmeldung, Gaia's Eyes Grant & Reviewer-Queue

```

Empfänger bemerkt Belästigung oder Spam im ReaderPane.jsx / ChatPane.jsx
↓
Klick auf "Melden" öffnet AbuseCenter-Dialog
↓
Client generiert GaiaProof (Ciphertext-Hash, Sender-Signatur, Server-Eingangszeitstempel)
↓
POST /api/v1/reports
  Payload: { targetGaiaID, category, severity, messageId, gaiaProof }
↓
Backend: governance.Handler -> governance.Service
↓
Erstellt abuse_case (Status: 'pending_review')
↓
Reviewer öffnet AbuseCenter.jsx -> GET /api/v1/reviewer/cases
↓
Reviewer fordert Fall-Beweis an: POST /reviewer/cases/:id/disclosure-requests
↓
Gemeldeter Nutzer oder Melder erteilt temporäre Lese freigabe via Gaia's Eyes:
POST /api/v1/gaias-eyes/grants
  Generiert zeitlich beschränktes Einmal-Token (TTL: 15 Minuten)
↓
Reviewer konsumiert Token via POST /api/v1/public/gaias-eyes/consume
↓
Reviewer stimmt ab: POST /reviewer/cases/:id/review (categoryVote, severityVote, recommendation)
↓
Schwellenwert erreicht (z.B. 2 von 3 Reviewern):
Node Operator wendet Aktion an (Friction-Drosselung oder temporäre Quarantäne)

```

Datenfluss 6: GaiaShield Sicherheitsereignis & Audit-Kette

```

Unerlaubter API-Zugriff, BOLA-Versuch oder Rate-Limit-Überschreitung
↓
Backend: EdgeShieldMiddleware oder SecuritySystem registriert Verstoß
↓
SecuritySystem.RecordEvent():
1. Klassifizierung: severity (INFO, WARNING, CRITICAL), category (AUTH, API, FEDERATION)
2. Privacy-Redaction: IP-Adresse und User-Agent werden gehasht, Token entfernt
3. Audit-Verkettung:
  event_hash = SHA256(previous_hash || event_id || timestamp || category || severity)
↓
Atomare Transaktion in SQLite:
- INSERT INTO security_events
- INSERT INTO security_audit_chain (event_id, previous_hash, event_hash)
↓
SQLite-Trigger erzwingen Unveränderlichkeit:
  trg_security_events_immutable_update blockiert Modifikationen

```

trg_security_events_no_delete blockiert Löschungen

↓

Live-Warnung an betroffenen Nutzer via `/api/v1/security/me/events`
oder an Node-Betreiber via `SecurityCenter.jsx`

8. Shared & Core Komponenten

Die folgenden Module und Dateien sind fundamentale Pfeiler des Gesamtsystems und werden modulübergreifend wiederverwendet:

1. Frontend API Client (Frontend/frontend/src/api.js)

CORE / SHARED DEPENDENCY - **Zweck:** Kapselt alle REST-Aufrufe, verwaltet Authentifizierungs-Header (Bearer JWT), implementiert automatische Token-Rotation (/api/v1/auth/refresh), Desktop-Erkennung (isDesktopRuntime) und einheitliche Fehlerbehandlung. - **Wird verwendet von:** Fast allen Frontend-Panes, Hooks (useChat , useEmails , useDrive , usePublicChannels , useGsn) und Modals.

2. Frontend Crypto Orchestrator (Frontend/frontend/src/crypto.js & sovereignCrypto.js)

CORE / SHARED DEPENDENCY - **Zweck:** Zentraler Einstiegspunkt für alle kryptografischen Operationen im Client. Kapselt Noble-Kurven, BIP-39 Mnemonic-Wiederherstellung, Web Worker Orchestrierung und Tauri-IPC-Befehle. - **Wird verwendet von:** AuthScreen.jsx , ChatPane.jsx , DrivePane.jsx , ComposerPane.jsx , GsnPane.jsx , useGaiaAuth.js .

3. Shared Rust Multi-Cipher Core (SovereignCryptoCore/src/lib.rs)

CORE / SHARED DEPENDENCY - **Zweck:** Plattformunabhängige, memory-safe Implementierung der authentifizierten Chiffren-Kaskaden. - **Wird verwendet von:** DesktopClient/src-tauri (nativ kompiliert) und SovereignCryptoWasm (nach WebAssembly kompiliert für Browser-Worker).

4. Zero-Dependency Core Primitives (Core/rust/gaiacore/src/lib.rs)

CORE / SHARED DEPENDENCY - **Zweck:** Extrem schlanke, standardbibliothek-reine Rust-Bibliothek für Invarianten: GaiaID-Validierung, konstanter Byte-Vergleich, Envelope-Parsing und TrustMesh-Formeln. - **Wird verwendet von:** Rust-Kernen und dem Android-Kernel via JNI.

5. Backend HTTP Router & Middleware Engine (Backend/httpx/)

CORE / SHARED DEPENDENCY - **Zweck:** Gin-freies, leichtgewichtiges HTTP-Framework auf Basis von Standard-Go net/http . Bietet CORS-Steuerung, strikte Security-Header und JSON-Serialisierung. - **Wird verwendet von:** Backend/routes.go , allen Domain-Handlern und der Operations-Überwachung.

6. Backend Persistence Adapter (Backend/repository/sql_store.go)

CORE / SHARED DEPENDENCY - **Zweck:** Zentraler Wrapper um *sql.DB . Definiert Transaktionsgrenzen, Isolationsebenen und bindet alle 25 domänenspezifischen Repository-Teile an. - **Wird verwendet von:** Allen Backend-Services zur relationalen Datenhaltung.

7. Globales Domänenmodell (Backend/models/models.go)

CORE / SHARED DEPENDENCY - **Zweck:** Enthält sämtliche systemweit geteilten Datenstrukturen, Envelope-Typen, DTOs und Datenbankzeilen-Definitionen. - **Wird verwendet von:** Allen Go-Paketen im Backend (auth , messaging , federation , room , mailbox , etc.).

8. GaiaShield Sicherheits-Subsystem (Backend/internal/security/shield.go)

CORE / SHARED DEPENDENCY - **Zweck:** Zentrale Instanz für Auditierung, Schwärzung sensibler Daten, IDS-Regelüberwachung und Ratenbegrenzung. - **Wird verwendet von:** routes.go , authHandler , messagingHandler , storageHandler , fedHandler .

9. Sichere Serialisierungs- & Typ-Utilities (Frontend/frontend/src/utis/safeJson.js , uuid.js , payload.js)

CORE / SHARED DEPENDENCY - **Zweck:** Schutz vor Prototype Pollution beim Parsen unvertrauenswürdiger Nutzlasten, strikte RFC-4122 UUID-Validierung und sanitierte Datenstrukturen. - **Wird verwendet von:** Nahezu allen Frontend-Komponenten und Hooks.

10. Mehrsprachigkeits-Engine (Frontend/frontend/src/utis/i18n.jsx)

CORE / SHARED DEPENDENCY - **Zweck:** Stellt Sprachkontexte, Übersetzungsfunktionen (t()) und Umschaltung für 10+ Sprachen bereit. - **Wird verwendet von:** Sämtlichen Benutzeroberflächen und Dialogen.

9. Architektur-Relationen & Mermaid-Diagramme

Diagramm 1: Gesamtsystem-Architektur

```
graph TB
    subgraph Client_Layer ["Client Layer (Endgeräte)"]
        subgraph Web_Client ["Web Client (Vite / React 18)"]
            ReactUI["React UI (Panels & Modals)"]
            WebCryptoWorker["Web Worker (liboqs HQC-256)"]
            SovereignWASM["SovereignCryptoWasm (Rust Kaskade)"]
            OPFS["OPFS (Lokaler GaiaDrive Speicher)"]
        end
        subgraph Desktop_Client ["Desktop Client (Tauri 2 Shell)"]
            TauriHost["Tauri 2 Rust Host"]
            NativeHQC["Native pqcrypto-hqc"]
            NativeCascade["Native SovereignCryptoCore"]
        end
        subgraph Mobile_Client ["Mobile Client (Android)"]
            ComposeUI["Jetpack Compose UI"]
            AndroidKernel["Platform Kernel"]
            AndroidKeystore["Keystore / StrongBox"]
            GoMobileNode["Embedded Go Node (Backend/mobileapi)"]
        end
    end

    subgraph Network_Edge ["Network Edge & Ingress"]
        NGINX["NGINX Reverse Proxy (TLS 1.3, CSP, Frame Deny)"]
    end

    subgraph Backend_Application ["Go Backend Application (Port :8080)"]
        HTTPX["httpx Engine (Router, CORS, Security Headers)"]
        GaiaShield["GaiaShield (Audit Chain & Privacy Redactor)"]

        subgraph Core_Services ["Core Services"]
            AuthSvc["Auth & Device Service"]
            MsgSvc["Messaging Service"]
            MailSvc["Mailbox & Scheduler Service"]
            RoomSvc["Room & Channel Service"]
            StorageSvc["Storage Service (Quotas & Chunks)"]
            FedSvc["Federation S2S Service"]
            GovSvc["Governance & Abuse Service"]
            GsnSvc["GSN Social Service"]
        end
    end

    subgraph Persistence_Layer ["Persistenz & Storage Layer"]
        SQLite["SQLite Engine (WAL Mode, Max 4/32 Conns)"]
        LocalFS["Lokaler Chunks-Pfad (uploads/)"]
        S3Store["S3 / MinIO Object Store (Optional)"]
    end

    subgraph External_Network ["Föderiertes Netzwerk"]
        RemoteNode["Entfernter GaiaCom Knoten"]
        SMTPGateway["Klassischer SMTP E-Mail Server"]
    end

    %% Client Interaktionen
    ReactUI --> WebCryptoWorker
    WebCryptoWorker --> SovereignWASM
    ReactUI --> OPFS
    ReactUI -->|HTTPS / API| NGINX
```

```

TauriHost --> NativeHQC
TauriHost --> NativeCascade
TauriHost -->|IPC Bridge| ReactUI

ComposeUI --> AndroidKernel
AndroidKernel --> AndroidKeystore
AndroidKernel --> GoMobileNode

%% Network Routing
NGINX -->|Reverse Proxy :8080| HTTPX
HTTPX --> GaiaShield
GaiaShield --> Core_Services

%% Service Zugriffe
AuthSvc --> SQLite
MsgSvc --> SQLite
MailSvc --> SQLite
RoomSvc --> SQLite
GovSvc --> SQLite
GsnSvc --> SQLite
StorageSvc --> SQLite
StorageSvc --> LocalFS
StorageSvc --> S3Store

%% Föderation & SMTP
FedSvc <-->|Signed PDUs / S2S| RemoteNode
Core_Services -->|SMTP Ingest / Egress| SMTPGateway

```

Diagramm 2: Dashboard- & Navigation-Architektur

```

graph LR
    User["Benutzer Interaktion"] --> NavSidebar["NavigationSidebar.jsx"]

    subgraph Navigation_Sektionen ["Navigations-Kategorien"]
        NavSidebar --> WorkspaceSec["Workspace"]
        NavSidebar --> NetworkSec["Netzwerk"]
        NavSidebar --> PersonalSec["Persönlich"]
    end

    subgraph Workspace_Views ["Workspace Views"]
        WorkspaceSec --> DashboardView["DashboardPane.jsx (Kommandozentrale)"]
        WorkspaceSec --> GaiaMailView["GaiaMail (Inbox, Sent, Drafts, etc.)"]
        WorkspaceSec --> SmtMailView["SMTP Mail (Legacy Ingest / Outbox)"]
        WorkspaceSec --> ChatView["ChatPane.jsx (Quanten-Chat 1-to-1)"]
        WorkspaceSec --> GroupsView["GroupChatPane.jsx (Räume & Kanäle)"]
    end

    subgraph Network_Views ["Netzwerk Views"]
        NetworkSec --> ChannelsView["PublicChannelsPane.jsx (Broadcasts)"]
        NetworkSec --> GsnView["GsnPane.jsx (GaiaSocialNetwork)"]
        NetworkSec --> NetHealthView["NetworkHealthDashboard.jsx"]
        NetworkSec --> SecCenterView["SecurityCenter.jsx (GaiaShield)"]
        NetworkSec --> AbuseCenterView["AbuseCenter.jsx (Meldecenter)"]
    end

    subgraph Personal_Views ["Persönliche Views"]
        PersonalSec --> ContactsView["ListPane.jsx (Adressbuch)"]
        PersonalSec --> ProfileView["ProfilePane.jsx (Schlüssel, Mnemonic, Privacy)"]
        PersonalSec --> DriveView["DrivePane.jsx (GaiaDrive / Vault)"]
        PersonalSec --> DropView["DropPane.jsx (GaiaDrop Inbox)"]
    end

```

```

subgraph Orchestration ["Renderer & Modals"]
  AppMainContent["AppMainContent.jsx (Dynamic View Switcher)"]
  AppModalLayer["AppModalLayer.jsx (Modals & Overlays)"]
end

Workspace_Views --> AppMainContent
Network_Views --> AppMainContent
Personal_Views --> AppMainContent
AppMainContent --> AppModalLayer

```

Diagramm 3: Modul-Abhängigkeitsgraph

```

graph TD
  SovereignCrypto["Modul 1: Sovereign Cryptographic Engine"]
  AuthModule["Modul 2: Authentifizierung & Vault"]
  IdentityModule["Modul 3: Identität & Trust Passport"]
  MessagingModule["Modul 4: Quanten-Chat (Direct E2EE)"]
  RoomModule["Modul 5: Gruppen-Chats (Rooms & Channels)"]
  MailboxModule["Modul 6: GaiaMail & Mailbox States"]
  SmtplibModule["Modul 7: SMTP Legacy Bridge"]
  ChannelsModule["Modul 8: Public Channels"]
  GsnModule["Modul 9: GSN Social Network"]
  DriveModule["Modul 10: GaiaDrive & Chunk Storage"]
  DropModule["Modul 11: GaiaDrop Submissions"]
  FedModule["Modul 12: Federation S2S Engine"]
  RegistryModule["Modul 13: Node Registry"]
  TrustMeshModule["Modul 14: TrustMesh Abuse Scoring"]
  GovModule["Modul 15: Governance & Gaia's Eyes"]
  ShieldModule["Modul 16: GaiaShield Security System"]

  %% Abhängigkeiten
  AuthModule --> SovereignCrypto
  IdentityModule --> AuthModule
  IdentityModule --> SovereignCrypto

  MessagingModule --> SovereignCrypto
  MessagingModule --> IdentityModule
  MessagingModule --> ShieldModule

  RoomModule --> MessagingModule
  RoomModule --> IdentityModule

  MailboxModule --> MessagingModule
  MailboxModule --> IdentityModule

  SmtplibModule --> MailboxModule
  SmtplibModule --> ShieldModule

  ChannelsModule --> IdentityModule
  GsnModule --> IdentityModule
  GsnModule --> SovereignCrypto

  DriveModule --> SovereignCrypto
  DriveModule --> AuthModule

  DropModule --> IdentityModule
  DropModule --> ShieldModule

  FedModule --> MessagingModule
  FedModule --> ShieldModule

  RegistryModule --> FedModule
  TrustMeshModule --> MessagingModule

```

```
GovModule --> TrustMeshModule
GovModule --> ShieldModule
```

Diagramm 4: Backend / API / Datenbank-Pipeline

```
sequenceDiagram
    autonumber
    actor Client as Frontend Client (React/Tauri)
    participant Nginx as NGINX Reverse Proxy
    participant HTTPX as httpx Router & Middleware
    participant Shield as GaiaShield (Edge Guard)
    participant Handler as Domain Handler (e.g. Messaging)
    participant Service as Domain Service (e.g. MessagingService)
    participant Repo as SQLStore (Repository Adapter)
    participant DB as SQLite Database Engine

    Client->>Nginx: HTTPS POST /api/v1/messaging/send (Bearer Token, Ciphertext)
    Nginx->>HTTPX: Proxy Pass 127.0.0.1:8080
    HTTPX->>Shield: SecurityHeaders & EdgeShieldMiddleware
    Shield->>Shield: Rate-Limit Check & Path-Jail Verification
    Shield->>Handler: Weiterleitung an Authentifizierten Handler
    Handler->>Handler: JSON Parsing & Validierung der Identitätsrechte
    Handler->>Service: SendMessage(senderIdentityId, recipientIds, envelope)
    Service->>Service: Proof-Generierung & Transcript-Bindung
    Service->>Repo: CreateEnvelope(envelope) & DeliverToInbox(inbox)
    Repo->>DB: BEGIN IMMEDIATE TRANSACTION
    Repo->>DB: INSERT INTO message_envelopes (...)
    Repo->>DB: INSERT INTO inboxes (...)
    Repo->>DB: INSERT INTO message_proofs (...)
    DB-->>Repo: Commit Transaction Success
    Repo-->>Service: Speicherung bestätigt
    Service-->>Handler: Nachricht erfolgreich geroutet
    Handler-->>HTTPX: httpx.WriteJSON(w, 200, {status: 'delivered'})
    HTTPX-->>Nginx: Response mit Security Headers
    Nginx-->>Client: 200 OK
```

10. Dateireferenzen

Zur technischen Nachvollziehbarkeit sind hier konkrete Implementierungsdetails aufgeführt:

- `Backend/repository/sql_store.go`: Definiert den Datentyp `SQLStore` als konkreten Adapter um `*sql.DB` und implementiert das Interface `repository.Store` aus `Backend/repository/repository.go`.
- `Backend/database/database.go`: Richtet SQLite-WAL-Modus über `PRAGMA journal_mode=WAL` und `PRAGMA foreign_keys(1)` in `sqliteDSN()` ein; führt `migrationStatements` zur Erstellung aller Tabellen aus.
- `Backend/internal/security/shield.go`: Implementiert `SecuritySystem`, führt `EdgeShieldMiddleware()` für alle HTTP-Aufrufe aus und steuert den `RunRetentionSweeper()`.
- `Backend/internal/security/audit.go`: Berechnet über `RecordSecurityAudit()` die kryptografische Kette `event_hash = SHA256(previous_hash || event_id)` und schreibt in `security_audit_chain`.
- `Backend/auth/auth_service.go`: Implementiert `Authenticate()`, erzeugt Passworthashes mittels `crypto.DerivePasswordHash()` und signiert JWTs via `jwt.SignToken()`.
- `Backend/messaging/messaging_service.go`: Kapselt `SendMessage()` und erzeugt in `generateMessageProof()` den SHA-256 Digest über das Chifftrat zur Hinterlegung in `message_proofs`.
- `Backend/storage/storage_service.go`: Koordiniert Chunks in `SaveChunk()`, validiert Quotas via `CheckUserQuota()` und ruft `RunFileRetentionSweeper()` auf.
- `Frontend/frontend/src/sovereignCrypto.js`: Implementiert `sovereignSeal()` und `sovereignOpen()`, prüft Hex-Längen über `assertHex()` und leitet Aufrufe an den Web-Worker oder die Tauri-Bridge weiter.
- `Frontend/frontend/src/sovereign/webCrypto.worker.js`: Lädt `hqc-256.runtime.js` und das WASM-Modul `gaiacom_sovereign_wasm_bg.wasm` zur isolierten Schlüsselberechnung im Browser.
- `DesktopClient/src-tauri/src/sovereign_crypto.rs`: Stellt über `#[tauri::command]` die fünf nativen Operationen `sovereign_hqc256_keypair`, `sovereign_hqc256_encapsulate`, `sovereign_hqc256_decapsulate`, `sovereign_seal` und `sovereign_open` bereit.
- `Core/rust/gaiacore/src/lib.rs`: Validiert Identitäten in `validate_gaia_id(value: &str)` nach dem Muster `@user:domain.tld` und vergleicht Puffer timing-resistent in `constant_time_eq()`.

11. Architektur-Auffälligkeiten & Ungenutzte Dateien

Bei der tiefen Codebasis-Analyse wurden folgende Architektur-Besonderheiten, historische Artefakte und Redundanzen identifiziert:

1. Veraltete Tresor-Implementierung (`useVault.js` vs. `useDrive.jsx`)

- **Auffälligkeit:** Im Verzeichnis `Frontend/frontend/src/components/chat/` existiert die Datei `VaultPane.jsx` sowie in `hooks/` die Datei `useVault.js`.
- **Analyse:** In `AppMainContent.jsx` und `App.jsx` wurde `useVault` vollständig durch `useDrive.jsx` und `DrivePane.jsx` abgelöst. `useDrive.jsx` dokumentiert dies explizit im Header (*"Replaces useVault with GaiaDrive"*). `VaultPane.jsx` und `useVault.js` sind ungenutzte historische Artefakte, die im Code nicht mehr aktiv aufgerufen werden.

2. Doppelte Föderations-Weiterleitungs-Routen

- **Auffälligkeit:** In `Backend/routes.go` (Zeilen 239–240) sind zwei identische Forward-Routen registriert:

```
router.POST("/.well-known/gaiacom/s2s/v1/forward", fedHandler.HandleS2SForward)
router.POST("/_gaiacom/s2s/v1/forward", fedHandler.HandleS2SForward)
```

- **Analyse:** `/_gaiacom/s2s/v1/forward` existiert als Abwärtskompatibilitäts-Alias für ältere Clients, während `/.well-known/gaiacom/...` dem standardisierten RFC-Discovery-Schema entspricht.

3. Statischer Systembenutzer in Datenbankmigrationen

- **Auffälligkeit:** In `Backend/database/database.go` (Zeilen 494–496) wird ein fest verdrahteter Benutzer `00000000-0000-0000-0000-000000000000` (`system_remote_hub`) angelegt:

```
INSERT OR IGNORE INTO users (id, username, ...) VALUES ('00000000-0000-...', 'system_remote_hub', ...)
```

- **Analyse:** Dient als synthetischer Fremdschlüsselanker für Föderationsobjekte und S2S-Hub-Nachrichten, um Fremdschlüssel-Constraints (`ON DELETE CASCADE`) ohne echten Benutzer-Login zu erfüllen.

4. Migrationen: Tabellenänderung vor Deklaration in älteren Blöcken

- **Auffälligkeit:** In `Backend/database/database.go` werden mehrere Tabellen zunächst angelegt und in späteren Migrationen per `ALTER TABLE` um Spalten erweitert (z.B. `public_channel_posts ADD COLUMN pinned_at`, `public_channels ADD COLUMN comments_enabled`).
- **Analyse:** Entstanden durch fortlaufende Software-Iterationen. Das Migrationssystem wendet diese sauber inkrementell an; für Neuinstallationen empfiehlt sich langfristig eine konsolidierte Baseline.

5. Dual-PQC Migrationsgrenze (Legacy v0.1/v0.2 vs. Sovereign v1.0)

- **Auffälligkeit:** In `Backend/messaging/messaging_service.go` und `Frontend/frontend/src/crypto.js` existiert Parsing-Code für Legacy-Algorithmen-Suiten (`GaiaCom/v0.1/hybrid-kem/...`).
- **Analyse:** Gewährleistet, dass während der Beta-v2-Phase ältere Nachrichtenpakete im Posteingang lesbar bleiben, während Neusendungen strikt auf Sovereign v1.0 (Accelerated / Top Secret) bestehen.

6. Mobile Sovereign v1.0 Nachrichtenpfad als definierte Integrationsgrenze

- **Auffälligkeit:** Die native Android-Architektur unter `Android/` besitzt ein vollständiges Modulgerüst und bindet über `Backend/mobileapi` den Go-Knoten ein. Der duale PQC-Nachrichtenpfad mit nativer HQC-256 Kaskade ist für Browser und Desktop (Tauri) produktiv verifiziert; die mobile End-to-End-Integration befindet sich im Übergang (Technical Beta).

7. Administrative PowerShell-Skripte im Projekt-Root

- **Auffälligkeit:** In `scripts/` existieren Hilfsskripte (`accept_android_licenses.ps1` , `build_android_native.ps1`), die für CI/CD-Pipelines und Entwickler gedacht sind, aber keine Laufzeitabhängigkeit für den Serverbetrieb darstellen.

Dokumentation generiert und verifiziert auf Basis des realen GaiaCom-Projektstands. Stand: Sovereign Beta v2.